

Frontend Interview Concepts

Complete Checklist — 143 Sections

Every concept from the Medium checklist — expanded with in-depth theory, real-time use cases, interview Q&A, and hands-on code. React sections excluded.

Source: [Babek Naghiyev — Frontend Interview Concepts \(Medium\)](#)

JavaScript Core | DOM & Browser | HTML | CSS | TypeScript | Performance | Security | Networking | Testing | Build Tools | Accessibility | Design Patterns | DSA | System Design | Angular | RxJS | NgRx | Output Questions | Polyfills | Machine Coding | Coding Challenges | Git | PWA

■ **90-Day FAANG Study Path** — 4 Hours/Day — SDE-2/3 Level A battle-tested, FAANG-caliber roadmap: 360 hours covering all 143 sections, 120+ LeetCode problems, 8 system design mocks, 10+ machine coding exercises, Angular projects, and behavioral rounds. This is what it takes to crack Google, Meta, Amazon, Apple, Microsoft at Senior level.

90Days 4hPer Day 360hTotal Hours 143Sections 12Phases 120+LeetCode

■ **Theory & Concepts**~90 hours — Deep reading of all 143 sections. Not skimming — understanding internals, writing your own explanations, drawing diagrams ■ **Hands-on Coding**~72 hours — All coding challenges (#131–143), polyfills (#122), machine coding (#123), build-from-scratch implementations. Every line typed by hand. ■ **DSA / LeetCode**~72 hours — 120+ curated problems: arrays, strings, trees, graphs, DP, backtracking, sliding window, two pointers, monotonic stack, BFS/DFS, Trie, union-find ■ **System Design**~48 hours — Frontend HLD + LLD, component APIs, state machines, performance budgets, accessibility audits. 8 full mock designs spoken aloud with timer ■ **Angular Deep Dive**~32 hours — All 15 Angular sections + build 3 real projects from scratch (dashboard, e-commerce, realtime chat) ■ **Behavioral + Mock**~30 hours — 15 STAR stories, leadership principles (Amazon LP, Google Googliness), 6 full mock interview simulations, speed rounds ■ **Revision & Weak Areas**~16 hours — Spaced repetition cycles, targeted drilling on weak topics, flashcard review, error pattern analysis

P1 Phase 1 — JavaScript Deep Foundations Days 1–12

48 hours Day 1 2.5h **#1–2:** JS Fundamentals + Scope & Closures — read deeply, make notes on lexical scope, closure gotchas, temporal dead zone

1.5h **#131:** Coding — Closures: counter, once, memoize, private variables — TYPE every solution, do NOT copy-paste Day 2 2h **#3–4:** this Keyword + Prototypes — draw the prototype chain diagram on paper. Understand `\_\_proto\_\_` vs `prototype`

1h **#137:** Coding — myBind, myCall, myApply, myNew — implement from scratch

1h **#138:** Coding — Scope & Hoisting puzzles (IIFE, TDZ, block scoping, var vs let in loops) Day 3 2.5h **#5:** Async JS — Promises deep dive (microtask queue, Promise chaining, error propagation, unhandled rejection). Do NOT rush this.

1.5h **#133:** Coding — promisify, sleep, Promise.allSettled, sequential execution Day 4 2h **#6–7:** ES6+ Features + Advanced Concepts (generators, iterators, symbols, Proxy, Reflect, WeakMap/WeakSet)

1h **#133:** Coding — retry with exponential backoff, concurrency limiter, async queue, cancellable promise

1h LEETCODE 5 easy array problems: Two Sum, Best Time to Buy Stock, Contains Duplicate, Move Zeroes, Merge Sorted Array Day 5 2h **#8–10:** Array Methods, Object Methods, Error Handling — know every trap (sparse arrays, Object.is, custom errors)

1h **#134:** Coding — groupBy, flat, deep clone with circular refs, object diff

1h **#131:** Coding — curry (fixed arity), infinite currying, partial application, compose/pipe Day 6 2h **#11–12:** Modules (ESM vs CJS, tree shaking, circular deps) + Regular Expressions (groups, lookahead, named captures)

1h **#140:** Coding — String compression, template engine, query string parser, regex patterns

1h LEETCODE 5 easy string problems: Valid Anagram, Reverse String, Longest Common Prefix, Valid Palindrome, Roman to Integer Day 7 2.5h **#121:** Tricky Output Questions — solve EVERY question with pen & paper FIRST, then verify in console

1.5h **#139:** Event Loop challenges — predict output for ALL micro/macro task ordering problems. Draw the queue diagram. Day 8 — REST DAY. Revision: re-read your notes for #1–12. Redo 3 tricky output questions without

looking at answers. Day 9 2.5h **#122:** Polyfills Round 1 — Promise.all, Promise.race, Promise.any, Promise.allSettled from scratch

1.5h **#122:** Polyfills Round 2 — Array.map, Array.filter, Array.reduce, Array.flat from scratch Day 10 2h **#132:** Coding — debounce (leading/trailing/both options), throttle (with leading/trailing), requestAnimationFrame throttle

1h **#136:** Coding — EventEmitter (on, off, once, emit), PubSub, Observable pattern

1h LEETCODE 3 medium problems: 3Sum, Group Anagrams, Product of Array Except Self Day 11 2h **#135:** Coding — Recursion & Trees: flatten nested array/object, deep equal, tree traversal (BFS/DFS), serialize/deserialize

1h **#141:** Coding — LRU Cache, Stack, Queue, LinkedList from scratch

1h LEETCODE 3 medium: Valid BST, Level Order Traversal, Invert Binary Tree Day 12 2h **#142:** Design Patterns in JS — Singleton, Factory, Observer, Strategy, Module, Decorator, Command pattern

1h **JS Mastery Test:** Time yourself: implement Promise.all + debounce + curry + EventEmitter in under 40 min total

1h LEETCODE 3 medium: Maximum Subarray, Coin Change, Climbing Stairs DP

P2 Phase 2 — DOM, Browser & HTML Mastery Days 13–20

32 hours Day 13 2h **#13–14:** DOM Manipulation + DOM Traversal — understand reflow vs repaint, DocumentFragment, virtual DOM concept

1h **#143:** Coding — DOM Challenges: build a querySelector, event delegation utility, DOM differ

1h LEETCODE 3 medium: Implement Trie, Word Search, Longest Substring Without Repeating Characters Day 14

2h **#15–16:** Events (bubbling/capturing/delegation) + Browser Storage (localStorage, sessionStorage, IndexedDB, cookies)

1h **#143:** Coding — Virtual scroll implementation, infinite scroll, intersection observer utility

1h PRACTICE Build a lightweight jQuery clone: \$(), .on(), .off(), .css(), .addClass(), .html() Day 15 2h **#17–19:** Browser APIs + Window & Document + Forms — Service Workers, Web Workers, requestIdleCallback, Broadcast Channel

1h **Machine Coding #1:** Build a form validator library from scratch (required, email, minLength, pattern, custom rules)

1h LEETCODE 3 medium: Number of Islands, Rotting Oranges, Flood Fill (BFS/DFS practice) Day 16 2h **#20–24:** Semantic HTML + HTML5 Features + Metadata/SEO + Media + HTML Accessibility

2h **#95–98:** Browser Rendering + How Browsers Work + Progressive Enhancement + Web Standards — understand Critical Rendering Path end-to-end Day 17 2.5h **#123 Part 1:** Machine Coding — Build an autocomplete/typeahead from scratch: debounced API, keyboard nav, highlight matches, accessibility

1.5h LEETCODE Sliding window: Minimum Window Substring, Max Sum Subarray of Size K, Longest Repeating Character Replacement Day 18 2.5h **#123 Part 2:** Machine Coding — Build a drag-and-drop Kanban board (HTML5 DnD API, state management, persistence)

1.5h LEETCODE Two pointers: Container With Most Water, Trapping Rain Water, Sort Colors Day 19 — REST DAY. Revision: Review #13–24. Can you explain the Critical Rendering Path without notes? Redo Event Loop output questions. Day 20 2.5h **#123 Part 3:** Machine Coding — Build a star rating widget, tooltip system, and modal/dialog (focus trapping, ESC close, ARIA)

1.5h LEETCODE Stack/Queue: Valid Parentheses, Min Stack, Daily Temperatures, Next Greater Element

P3 Phase 3 — CSS, TypeScript & Tooling Days 21–28

32 hours Day 21 2h **#25–27:** CSS Selectors + Box Model + Layout — specificity calculation, margin collapse, BFC, stacking contexts

1h **#28–29:** Flexbox + Grid — solve 20 Flexbox/Grid layout challenges. Know every property by heart.

1h PRACTICE Recreate a complex layout: Netflix homepage grid OR a dashboard sidebar+content layout using only CSS Grid Day 22 2h **#30–35:** Responsive Design + Typography + Colors + Transforms & Animations + CSS Variables + Modern CSS

1h **#36–39:** Methodologies (BEM, OOCSS) + CSS-in-JS + Preprocessors + CSS Performance

1h PRACTICE Build a fully responsive card component with hover animations, dark mode support, and container queries Day 23 2.5h **#40–46:** TypeScript — Basics + Advanced Types + Generics + Utility Types + Functions + Enums + Modules. Focus on generics deeply.

1.5h **#125:** Advanced TypeScript Patterns — conditional types, mapped types, template literal types, infer keyword Day 24 2h PRACTICE TypeScript challenge: Type a full REST API client with generics, discriminated unions, and

branded types

1h LEETCODE 3 medium: LRU Cache, Design HashMap, Design Browser History

1h LEETCODE Binary Search: Search in Rotated Array, Find Minimum in Rotated Sorted Array, Koko Eating Bananas Day 25 2h **#68–73**: Build Tools — Webpack (loaders, plugins, code splitting, HMR), Vite, Babel, Package Managers, Linting, Env Variables

2h PRACTICE Set up a project from scratch: Webpack config with code splitting, tree shaking, CSS extraction, source maps. Then the same with Vite — compare. Day 26 2h **#63–67**: Testing — Unit Testing + Integration + E2E + TDD + State Management patterns

1h PRACTICE Write 15 unit tests for your polyfills (Promise.all, debounce, curry) using Jest/Vitest

1h LEETCODE 4 medium: Merge Intervals, Insert Interval, Meeting Rooms II, Non-overlapping Intervals Day 27 — REST DAY. Revision: Re-read CSS selectors specificity, TypeScript generics, Webpack internals. Practice explaining them aloud. Day 28 2h **Cumulative Review**: Speed run through #25–73 headers. Flag any topic you can't explain in 30 seconds. Deep-read those.

1h LEETCODE 3 medium: Longest Palindromic Substring, Decode Ways, Word Break (DP)

1h **Polyfill Speed Test**: Implement these 5 under 5 min each: Promise.all, Array.reduce, debounce, curry, EventEmitter

P4 Phase 4 — Performance, Security & Networking Days 29–36

32 hours Day 29 2h **#47–48**: Loading Performance + Runtime Performance — lazy loading, code splitting, tree shaking, requestIdleCallback

1h **#49–50**: Rendering Performance + Caching — avoid layout thrashing, will-change, composite layers, cache headers

1h **#128**: Performance Case Studies — real-world optimization stories at FAANG scale Day 30 2h **#51–54**: Core Web Vitals + Images/Media + Fonts + Measuring Performance (Lighthouse, WebPageTest, Chrome DevTools)

2h PRACTICE Performance audit: Take any live website, run Lighthouse, identify top 5 issues, write a fix plan. Then actually fix 2 of them. Day 31 2h **#55–57**: Security — Common Vulnerabilities (XSS, CSRF, SSRF) + Prevention + Authentication (OAuth, JWT, PKCE, session management)

1h **#130**: Advanced Security — CSP headers, subresource integrity, supply chain attacks, CORS deep dive

1h LEETCODE Graph: Clone Graph, Course Schedule, Pacific Atlantic Water Flow Day 32 2h **#58–62**: Networking — HTTP/2/3, REST API Design, GraphQL, Data Fetching patterns, WebSockets, SSE

1h PRACTICE Build a real-time chat using WebSockets (vanilla JS, no libraries). Handle reconnection, heartbeat, message queuing.

1h LEETCODE Heap: Top K Frequent Elements, Kth Largest Element, Merge K Sorted Lists Day 33 2h **#74–79**: Accessibility — WCAG, Keyboard Nav, Screen Readers, ARIA, Color/Contrast, Form Accessibility

1h PRACTICE Accessibility audit: Test your autocomplete and Kanban from Phase 2. Fix every a11y issue — screen reader testing, keyboard-only navigation.

1h LEETCODE 3 medium DP: House Robber, Unique Paths, Minimum Path Sum Day 34 2h **#80–86**: Design Patterns — Creational, Structural, Behavioral, Architectural (MVC, MVVM, Flux, Clean Architecture)

2h PRACTICE Implement Observer, Mediator, Strategy, and Command patterns in a real mini-app (e.g., a text editor with undo/redo) Day 35 — REST DAY. Revision: Review security headers, CORS, performance metrics. Practice explaining Core Web Vitals and XSS prevention aloud in 2 min each. Day 36 2h **#87–91**: DSA — Data Structures + Common Algorithms + Complexity Analysis

1h LEETCODE 3 medium: Implement Trie, Design Add and Search Words, Word Search II (Trie+DFS)

1h LEETCODE 2 hard: Serialize/Deserialize Binary Tree, Median of Two Sorted Arrays

P5 Phase 5 — Frontend System Design (FAANG-Level) Days 37–46

40 hours Day 37 2h **#92–93**: Component Design + App Architecture — micro-frontends, module federation, monorepo patterns

2h **#126**: Frontend LLD — component API design, state machine modeling, dependency injection patterns, interface contracts Day 38 2h **#94–96**: Common Components + Full App Designs + Scalability

2h **#127**: System Design Patterns — BFF, API Gateway, caching layers, CDN strategy, edge computing, SSR vs CSR vs ISR decision framework Day 39 4h **MOCK #1 Design Google Search Frontend** — 45 min timed, speak aloud:

- Requirements gathering (5 min) • High-level architecture (10 min) • Component breakdown (10 min) • Data flow & state (10 min) • Performance & edge cases (10 min)

Then spend 1h writing up what you missed and improving your answer. Record yourself if possible. Day 40 4h

MOCK #2 Design a Real-Time Collaborative Document Editor (Google Docs) — 45 min timed:

- Conflict resolution (OT vs CRDT) • WebSocket architecture • Cursor sync • Offline support • Version history

Spend 1.5h improving your answer. Research OT/CRDT trade-offs. This is a classic FAANG question. Day 41 2h

MOCK #3 Design Facebook News Feed — infinite scroll, virtualization, optimistic updates, cache invalidation, real-time updates

2h LEETCODE Backtracking: Subsets, Permutations, Combination Sum, N-Queens, Sudoku Solver Day 42 2h

MOCK #4 Design Figma (Collaborative Design Tool) — Canvas rendering, layer management, real-time multi-user, zoom/pan, asset library

2h PRACTICE Machine Coding: Build a mini drawing canvas app (HTML5 Canvas, undo/redo stack, shape tools, export as PNG) Day 43 — REST DAY. Review all 4 system design answers. Write a 1-page template for your system design approach. Memorize the structure. Day 44 2h **MOCK #5 Design Spotify Web Player** — audio streaming, playlist state, offline mode, search with debounce, responsive player UI

2h LEETCODE Graph advanced: Dijkstra's, Topological Sort, Network Delay Time, Alien Dictionary Day 45 2h

MOCK #6 Design Twitter/X Frontend — tweet timeline, real-time updates, infinite scroll, media previews, notifications, PWA

2h PRACTICE Machine Coding: Build an image carousel with swipe gestures, lazy loading, keyboard nav, touch support, responsive Day 46 2h **System Design Consolidation:** Re-do your weakest system design mock from scratch. Compare with your first attempt.

1h LEETCODE 2 hard: LFU Cache, Trapping Rainwater II

1h LEETCODE 3 medium: Task Scheduler, Reorganize String, Partition Labels (Greedy)

P6 Phase 6 — Angular Deep Dive & Projects Days 47–56

40 hours Day 47 2h **#106–107:** Angular Change Detection + Dependency Injection — zone.js internals, OnPush strategy, hierarchical injectors

2h **#108–109:** Lifecycle Hooks + RxJS Deep Dive — marble diagrams, switchMap vs mergeMap vs concatMap, memory leak patterns Day 48 2h **#110–111:** Router & Guards + Reactive Forms — lazy loading routes, preloading strategies, custom validators, dynamic forms

2h **#112–113:** HTTP & Interceptors + Standalone & Modern APIs — signals, deferrable views, new control flow

Day 49 2h **#114–116:** Pipes & Directives + Content Projection + Angular Performance

2h **#117–120:** Angular Testing + NgRx & SignalStore + Best Practices + Common Mistakes Day 50 4h PRACTICE

Angular Project #1 — Admin Dashboard:

- Standalone components, lazy-loaded routes, reactive forms
- NgRx/SignalStore for state, HTTP interceptors for auth
- Data table with sorting, filtering, pagination
- Dark mode toggle, responsive sidebar, chart integration Day 51 4h PRACTICE **Angular Project #1 continued:**
- Add unit tests (Jasmine/Karma) for 3 components + 2 services
- Add E2E test (Cypress) for login flow
- Performance audit: implement OnPush everywhere, trackBy, lazy images Day 52 — REST DAY. Review Angular notes. Can you explain Change Detection, DI hierarchy, and RxJS operators without looking at notes? Test yourself.

Day 53 4h PRACTICE **Angular Project #2 — E-Commerce Storefront:**

- Product listing with virtual scroll, search with debounce
- Cart with NgRx, optimistic updates
- Checkout form with multi-step reactive forms, validation
- Route guards, HTTP interceptors, error handling Day 54 2h PRACTICE **Angular Project #2 continued:** Add SSR with Angular Universal, SEO meta tags, PWA manifest

2h LEETCODE DP marathon: Longest Increasing Subsequence, Edit Distance, Longest Common Subsequence, 0/1

Knapsack Day 55 4h PRACTICE **Angular Project #3 — Real-Time Chat App:**

- WebSocket service with RxJS (reconnection, heartbeat)

- Message list with virtual scroll, typing indicators
 - File upload with drag-and-drop, image preview
 - Push notifications, online/offline status, unread counts
- Day 56 2h PRACTICE **Angular Project #3 continued:** Testing, performance tuning, a11y audit

2h **Angular Mastery Test:** Explain these without notes in 2 min each: Change Detection, RxJS switchMap vs mergeMap, Standalone components, Signals, NgRx flow, DI resolution

P7 Phase 7 — Miscellaneous, Git & Soft Skills Days 57–62

24 hours Day 57 2h **#99–105:** Mobile/PWA + CSS Architecture + Modern JS + Framework Comparisons + Soft Skills + Implementation Patterns + Additional Topics

2h **#87–91:** Git Basics + Git Workflows + Advanced Git — rebase vs merge, cherry-pick, bisect, reflog, worktrees

Day 58 2h **#124:** Advanced Coding Challenges — implement JSON.stringify, JSON.parse, lodash.get, lodash.set, deep merge

2h LEETCODE Mixed hard: Regular Expression Matching, Merge K Sorted Lists, Sliding Window Maximum,

Word Ladder Day 59 2h **Machine Coding #4:** Build a spreadsheet/grid component — editable cells, formulas, cell references, undo/redo

2h PRACTICE Machine Coding #5: Build a rich text editor with toolbar — bold, italic, list, link insertion, keyboard shortcuts Day 60 2h **Machine Coding #6:** Build a file explorer tree — expand/collapse, lazy loading children, drag to move, right-click context menu, keyboard nav

2h LEETCODE Union-Find: Number of Connected Components, Redundant Connection, Accounts Merge, Smallest String With Swaps Day 61 — REST DAY. Big revision: Skim ALL 143 section headers. Write down every topic where you feel <70% confident. These are your targets for Phase 8. Day 62 2h **Weak Area Deep Dive:** From your Day 61 list, pick the 5 weakest topics and re-read them thoroughly.

1h LEETCODE 3 problems from your weakest DSA pattern

1h **Polyfill Speed Test #2:** All 10 polyfills from memory, aim for <6 min each now. Time yourself strictly.

P8 Phase 8 — Behavioral & Leadership (FAANG-Specific) Days 63–68

24 hours Day 63 2h **#129:** Behavioral Questions — STAR method deep dive. Write your first 5 stories:

- 1. Biggest technical challenge you solved
- 2. Time you disagreed with your manager
- 3. Project you led
- 4. Failure and what you learned
- 5. Time you mentored someone

2h **Amazon Leadership Principles:** Map each of your 5 stories to LPs: Customer Obsession, Ownership, Bias for Action, Dive Deep, Earn Trust, Deliver Results Day 64 2h **Write 5 more STAR stories:**

- 6. Scaling a system under pressure
- 7. Cross-team collaboration
- 8. Ambiguous problem with no clear solution
- 9. Taking initiative beyond your role
- 10. Handling conflicting priorities

2h **Google Googliness + Meta Core Values:** Map stories to "Collaboration," "Transparency," "Move Fast." Write company-specific versions. Day 65 2h **Additional 5 STAR stories for SDE-3:**

- 11. Tech debt reduction strategy
- 12. Architecture decision with trade-offs
- 13. Driving team productivity improvement
- 14. Influencing without authority
- 15. Handling production incident

2h BEHAVIORAL MOCK Practice all 15 stories aloud. Each must be 2 min or less. Record yourself. Watch the recording. Fix filler words. Day 66 2h **SDE-3 Leadership Deep Dive:** Prepare answers for:

- "How do you make technical decisions?"
- "How do you handle underperformers?"
- "Describe your code review philosophy"
- "How do you balance tech debt vs features?"

2h LEETCODE Monotonic Stack: Largest Rectangle in Histogram, Maximal Rectangle, Stock Span, Remove K Digits Day 67 2h **"Why this company?" Prep:** Research 3-5 target companies deeply. Write genuine reasons for each. Prepare 3 questions to ask each interviewer.

2h PRACTICE Machine Coding: Build a notification system — toast stack, auto-dismiss, priority queue, action buttons, ARIA live regions Day 68 4h FULL MOCK INTERVIEW #1 Simulate a complete FAANG interview (find a friend or use Pramp/interviewing.io):

- Round 1: 45 min — DSA (1 medium + 1 hard)
- Round 2: 45 min — Machine coding (timed, unfamiliar component)
- Round 3: 45 min — System design
- Round 4: 45 min — Behavioral

Score yourself 1–5 on each. Be brutally honest.

P9 Phase 9 — LeetCode Intensive Sprint Days 69–76

32 hours Day 69 4h LEETCODE **Arrays & Hashing**: 4Sum, Encode/Decode Strings, Longest Consecutive Sequence, Top K Frequent Words, Subarray Sum Equals K Day 70 4h LEETCODE **Trees & Graphs**: Binary Tree Max Path Sum, Lowest Common Ancestor, Graph Valid Tree, Word Ladder II, All Nodes Distance K Day 71 4h LEETCODE **DP Advanced**: Burst Balloons, Partition Equal Subset Sum, Target Sum, Palindrome Partitioning, Interleaving String Day 72 4h LEETCODE **Intervals & Greedy**: Minimum Number of Arrows, Car Pooling, Jump Game I+II, Gas Station, Candy Day 73 — REST DAY. Review all LeetCode solutions. For every problem you got wrong, write down WHY and the pattern you missed. Day 74 4h LEETCODE **Sliding Window & Two Pointers**: Minimum Window Substring, Subarrays with K Different Integers, Shortest Subarray with Sum at Least K, Fruit into Baskets Day 75 4h LEETCODE **Design Problems**: Design Twitter, Design Hit Counter, Design File System, Design Snake Game, Design Tic-Tac-Toe Day 76 2h LEETCODE **Hard problems**: Wildcard Matching, Minimum Window Substring, Alien Dictionary, Critical Connections
2h **Pattern Review**: Write a cheat sheet of all DSA patterns: Two Pointers, Sliding Window, BFS/DFS, DP table, Backtracking template, Monotonic Stack, Union-Find template

P10 Phase 10 — Advanced Machine Coding Days 77–82

24 hours Day 77 4h PRACTICE **Machine Coding — Multi-Select Dropdown**: Search, keyboard nav, tag display, virtual scroll for 10k options, accessible, async options loading, group headers, create new option Day 78 4h PRACTICE **Machine Coding — Data Table**: Column sorting, multi-column sort, inline editing, row selection, column resizing, pagination, CSV export, custom cell renderers, sticky headers Day 79 4h PRACTICE **Machine Coding — Calendar/Date Picker**: Month/week/day views, event creation with drag, recurring events, timezone support, responsive, range selection Day 80 4h PRACTICE **Machine Coding — Nested Comment Thread**: Infinite nesting, collapse/expand, vote system, lazy load replies, optimistic updates, reply editor with mentions Day 81 — REST DAY. Review all machine coding projects. For each, write down: What you'd improve, accessibility gaps, performance bottlenecks. Day 82 2h PRACTICE **Speed Machine Coding**: Build a countdown timer with lap functionality in 30 min. Build a color picker in 30 min. Build a tabs component in 20 min.
2h PRACTICE **Speed Machine Coding**: Build an accordion in 15 min. Build a tooltip in 15 min. Build a progress stepper in 20 min. Build a tag input in 20 min.

P11 Phase 11 — Full Mock Interview Simulations Days 83–86

16 hours Day 83 4h FULL MOCK #2 — Google Style

- 45 min DSA: 2 problems (medium+hard), optimize for time complexity
- 45 min Frontend: Build accessible autocomplete from scratch with tests
- 45 min System Design: "Design YouTube Frontend" — video player, recommendations, search, comments
- 45 min Googliness: Leadership, collaboration, handling ambiguity stories Day 84 4h FULL MOCK #3 — Meta Style
- 45 min Coding: 2 medium LC problems, clean code, explain trade-offs aloud
- 45 min Frontend: Machine coding — build something UNFAMILIAR in 40 min
- 45 min System Design: "Design Instagram Stories" — upload, feed, expiry, analytics
- 45 min Behavioral: Values (Move Fast, Be Bold, Focus on Impact, Build Social Value) Day 85 4h FULL MOCK #4 — Amazon Style
- 45 min Online Assessment simulation: 2 problems, strict time limit
- 45 min System Design: "Design Amazon Product Page" — recommendations, reviews, buy box, a11y
- 45 min LP Deep Dive: Customer Obsession, Ownership, Bias for Action, Dive Deep, Earn Trust, Deliver Results — 2 stories per LP
- 45 min Bar Raiser: Hardest behavioral questions — "Tell me about a time you failed spectacularly" Day 86 2h **Mock Debrief**: Review recordings of all 4 full mocks. Write down patterns: Where do you stumble? What do you skip? What sounds weak?

2h **Gap Filling**: Spend time on your 3 biggest weaknesses identified from mocks. Deep study + practice.

P12 Phase 12 — Final Polish & Interview Week Days 87–90

16 hours Day 87 2h REVERSE **Grand Review — Theory:** Speed-read ALL 143 section headers + key concepts. Use your personal notes and flashcards.

1h REVERSE **Polyfill Final Run:** All 10 from memory, <5 min each: Promise.all, Array.map, reduce, filter, bind, call, new, debounce, throttle, EventEmitter

1h REVERSE **Angular Rapid Review:** Change Detection, DI, RxJS operators, Signals, NgRx — explain each in 60 seconds Day 88 2h REVERSE **System Design Template:** Review your 8 system designs. Write a universal template:

Requirements → Architecture → Components → Data Flow → Performance → Edge Cases

1h REVERSE **DSA Pattern Sheet:** Review your pattern cheat sheet. Solve 3 random mediums in under 20 min each.

1h REVERSE **Behavioral Stories:** Rehearse your top 10 STAR stories. Each under 2 min. Confident, specific, impactful. Day 89 — REST DAY. No studying. Sleep 8+ hours. Exercise. Eat well. Your brain is consolidating 360 hours of knowledge. Trust the process. Day 90 1h **Final Cheat Sheet:** Write your 1-page personal summary of the 20 most-asked topics. Keep it in your pocket.

1h **Behavioral Stories:** Rehearse your top 5 stories one final time. Confident, concise, 90 seconds each.

1h **Logistics:** Quiet room, camera, mic test, water, notepad, pen, stable internet. Review company info. Know your interviewer's name if available.

1h **Confidence Boost:** Solve 3 EASY coding problems fast. Remind yourself: **You have invested 360 hours. You have built 3 Angular projects, solved 120+ LeetCode problems, done 8 system design mocks, practiced 15 STAR stories, built 10+ machine coding components. You are ready. ■■**

■ FAANG Interview Reality Check:

- **360 hours is the FAANG standard.** Top engineers at Google/Meta/Amazon typically spend 300–500h preparing for Senior (L5/E5/SDE-3) interviews. This plan is calibrated to that level.
- **120+ LeetCode problems** across 12 patterns: Arrays, Strings, Trees, Graphs, DP, Backtracking, Sliding Window, Two Pointers, Stack/Queue, Heap, Trie, Union-Find.
- **8 system design mocks** covering: Search, Docs, News Feed, Figma, Spotify, Twitter, YouTube, Instagram. You should be able to design ANY frontend system.
- **10+ machine coding components** from scratch: autocomplete, Kanban, data table, calendar, comment thread, spreadsheet, file explorer, carousel, notification system, and speed builds.
- **15 STAR stories** covering: Amazon LPs, Google Googliness, Meta values, general SDE-3 leadership. 40% of senior rejections are behavioral — DO NOT skip this.
- **3 Angular projects:** Dashboard, E-Commerce, Real-Time Chat. Each with tests, performance tuning, and a11y audits.
- **4 full mock interviews:** Google-style, Meta-style, Amazon-style, and one general. If you can pass these, you can pass the real thing.
- **If you have only 60 days:** Do Phases 1–5, then pick 10 days from Phases 6–9 based on your weaknesses, then do Phase 12.
- **If you have only 30 days:** Do Days 1–7, 13–15, 21–24, 29–31, 37–42, 63–65, 69–71, 87–90.
- **Record every mock interview.** Watch your recordings. Fix filler words ("um," "like"), long pauses, and vague answers. This single habit can move you from "borderline" to "strong hire."

1. JavaScript Fundamentals

Checklist: Primitive types · type coercion · strict equality · truthy/falsy · var/let/const · TDZ · hoisting · function declarations vs expressions · arrow functions · IIFE

1.1 Primitive Types

JavaScript has **7 primitive types**: string, number, boolean, null, undefined, symbol (ES6), bigint (ES2020). Primitives are **immutable** and compared **by value**. Everything else (Object, Array, Function, Date, Map, etc.) is an **object** — compared by reference.

```
typeof 'hello'      // 'string'
typeof 42           // 'number'
typeof true         // 'boolean'
typeof undefined    // 'undefined'
typeof null         // 'object' ← historical bug, never fixed
typeof Symbol()     // 'symbol'
typeof 42n          // 'bigint'
typeof {}           // 'object'
typeof []           // 'object' ← arrays are objects
typeof function(){} // 'function'

// Check array properly
Array.isArray([1,2,3]); // true
```

1.2 Type Coercion & Strict Equality

`==` (loose equality) converts operands to the same type before comparison. `===` (strict equality) checks both type and value — **always prefer** `===`.

```
// Loose equality surprises
0 == ''           // true ('' → 0)
0 == '0'          // true ('0' → 0)
'' == '0'         // false (string comparison)
false == '0'      // true (both → 0)
false == null     // false (null only == undefined)
null == undefined // true (special rule)
NaN == NaN        // false (NaN is never equal to anything)

// Always use strict equality
0 === ''          // false
null === undefined // false

// Object.is – like === but handles edge cases
Object.is(NaN, NaN) // true
Object.is(+0, -0)   // false (=== says true)
```

1.3 Truthy & Falsy Values

Falsy values (only 8): false, 0, -0, 0n, "", null, undefined, NaN. **Everything else is truthy** — including [], {}, "0", "false", new Boolean(false).

```
// Common interview traps
Boolean([])      // true – empty array is truthy!
Boolean({})      // true – empty object is truthy!
Boolean("0")     // true – non-empty string is truthy!
Boolean(new Boolean(false)) // true – object wrapper is truthy!

// Practical: guard patterns
const name = user.name || 'Anonymous'; // problem: '' is falsy
const name2 = user.name ?? 'Anonymous'; // better: ?? only checks null/undefined
```

1.4 var, let, const & Temporal Dead Zone

Feature	var	let	const
Scope	Function-scoped	Block-scoped	Block-scoped
Hoisting	Yes → initialized as undefined	Yes → but in TDZ	Yes → but in TDZ
Redeclaration	Allowed	■ SyntaxError	■ SyntaxError
Reassignment	■	■	■ (binding is const, not value)
Global object property	Yes (window.x)	No	No

Temporal Dead Zone (TDZ): The period between entering a block scope and reaching the `let/const` declaration. Accessing the variable during TDZ throws `ReferenceError`. This exists to catch bugs — using a variable before it's declared is almost always a mistake.

```
// TDZ demonstration
{
  // TDZ for `x` starts here
  console.log(x); // ReferenceError: Cannot access 'x' before initialization
  let x = 10;     // TDZ ends here
}

// var does NOT have TDZ
console.log(y); // undefined (hoisted)
var y = 20;

// const with objects – the binding is constant, not the value
const arr = [1, 2, 3];
arr.push(4); // ■ mutating the array is fine
// arr = [5, 6]; // ■ TypeError: Assignment to constant variable
```

1.5 Hoisting

JavaScript moves declarations to the top of their scope during the compilation phase. **Only declarations are hoisted, not initializations.**

```
// Function declaration – fully hoisted (you can call before definition)
greet(); // "Hello!" – works!
function greet() { console.log("Hello!"); }

// Function expression – NOT hoisted (only the var declaration is)
// sayHi(); // TypeError: sayHi is not a function
var sayHi = function() { console.log("Hi!"); };

// Arrow function – NOT hoisted
// add(1,2); // ReferenceError (TDZ if using let/const)
const add = (a, b) => a + b;

// Class declarations – hoisted but IN TDZ (like let)
// const p = new Person(); // ReferenceError
class Person { constructor() {} }
```

1.6 Function Declarations vs Expressions vs Arrow Functions

Feature	Declaration	Expression	Arrow
Syntax	<code>function foo() {}</code>	<code>const foo = function() {}</code>	<code>const foo = () => {}</code>

Feature	Declaration	Expression	Arrow
Hoisted?	Yes (fully)	No	No
Has own <code>this</code> ?	Yes (dynamic)	Yes (dynamic)	No (lexical — inherits from parent)
Has <code>arguments</code> ?	Yes	Yes	No (use rest params)
Can be constructor?	Yes (new)	Yes	No (new throws <code>TypeError</code>)
<code>prototype</code> property?	Yes	Yes	No

1.7 IIFE (Immediately Invoked Function Expression)

A function that runs the moment it's defined. Before ES6 modules, IIFEs were the primary way to create private scope and avoid polluting the global namespace.

```
// Classic IIFE
(function() {
  var secret = 42; // not accessible outside
  console.log('Runs immediately');
})();

// Arrow IIFE
(() => {
  const config = { debug: false };
  // ... initialization code
})();

// IIFE returning a module (Module Pattern)
const Counter = (() => {
  let count = 0; // private
  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
})();
Counter.increment(); // 1
Counter.increment(); // 2
Counter.getCount(); // 2
// Counter.count → undefined(private!)
```

■ Interview Questions — JS Fundamentals

Q: What is the output?

```
console.log(typeof null);
console.log(typeof NaN);
console.log(NaN === NaN);
console.log(null == undefined);
console.log(null === undefined);
```

A: 'object', 'number', false, true, false. `typeof null` is a historical bug. `NaN` is type number but never equals itself. `null == undefined` is true by spec, but `===` checks type too.

Q: When would you use `var` over `let`/`const`?

A: Almost never in modern code. The only edge case is when you specifically need function-scoping or want the variable on `window` (global scope). In practice, use `const` by default, `let` when rebinding is needed, `var` never.

Q: What's the difference between `==` and `===`? When would you use `==`?

A: `==` coerces types before comparison, `===` does not. The only acceptable use of `==` is `x == null` which checks both `null` and `undefined` in one expression (equivalent to `x === null || x === undefined`).

2. Scope & Closures

Checklist: Global/function/block scope · lexical scoping · closure creation & use cases · module pattern · private variables · memory implications

2.1 Scope Types

Global scope: Variables declared outside any function/block. Accessible everywhere. In browsers, `var` globals become properties of `window`.

Function scope: `var` is scoped to the nearest enclosing function.

Block scope: `let/const` are scoped to the nearest enclosing `{}` block (if, for, while, etc.).

```
var globalVar = 'global'; // global scope

function outer() {
  var functionVar = 'function'; // function scope

  if (true) {
    var stillFunction = 'var ignores blocks'; // function scope!
    let blockScoped = 'only in this if';      // block scope
    const alsoBlock = 'also only here';      // block scope
  }

  console.log(stillFunction); // ■ 'var ignores blocks'
  // console.log(blockScoped); // ■ ReferenceError
}
```

2.2 Lexical Scoping

A function's scope is determined by **where it is written** in the source code, not where it is called. Inner functions can access variables of outer functions. This is the basis for closures.

2.3 Closures — Deep Dive

A **closure** is created when a function retains access to its outer (enclosing) scope's variables even after the outer function has returned. Every function in JavaScript creates a closure over its surrounding scope.

```
// Closure in action
function createMultiplier(factor) {
  // 'factor' is enclosed (closed over)
  return function(number) {
    return number * factor; // accesses 'factor' from outer scope
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);
double(5); // 10 — 'factor' is 2
triple(5); // 15 — 'factor' is 3
// createMultiplier has already returned, but 'factor' is still accessible!
```

2.4 Closure Use Cases

Private Variables & Data Encapsulation

```
function createBankAccount(initialBalance) {
  let balance = initialBalance; // private — not accessible from outside
  const transactions = [];      // private

  return {
    deposit(amount) {
      if (amount <= 0) throw new Error('Amount must be positive');
      balance += amount;
      transactions.push({ type: 'deposit', amount, date: new Date() });
      return balance;
    },
  },
}
```

```

    withdraw(amount) {
      if (amount > balance) throw new Error('Insufficient funds');
      balance -= amount;
      transactions.push({ type: 'withdraw', amount, date: new Date() });
      return balance;
    },
    getBalance() { return balance; },
    getStatement() { return [...transactions]; } // return copy
  };
}
const account = createBankAccount(1000);
account.deposit(500); // 1500
account.withdraw(200); // 1300
account.getBalance(); // 1300
// account.balance → undefined (truly private!)

```

Function Factories

```

// Creating specialized validators
function createValidator(regex, errorMsg) {
  return function(value) {
    if (!regex.test(value)) return errorMsg;
    return null;
  };
}
const isEmail = createValidator(/^[@]+\.[^@]+$/, 'Invalid email');
const isPhone = createValidator(/^\d{10}$/, 'Must be 10 digits');
isEmail('test@example.com'); // null (valid)
isEmail('bad'); // 'Invalid email'

```

Event Handlers & Callbacks

```

function setupButton(buttonId, message) {
  const button = document.getElementById(buttonId);
  let clickCount = 0; // enclosed in closure
  button.addEventListener('click', () => {
    clickCount++;
    console.log(`${message} – clicked ${clickCount} times`);
  });
}

```

2.5 Classic Closure Trap — Loop with var

```

// THE BUG: all callbacks share the same 'i'
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3 (not 0, 1, 2!)

// FIX 1: Use let (block scoping creates new 'i' per iteration)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2

// FIX 2: IIFE (creates new scope per iteration)
for (var i = 0; i < 3; i++) {
  ((j) => { setTimeout(() => console.log(j), 100); })(i);
}

// FIX 3: Function factory
for (var i = 0; i < 3; i++) {
  setTimeout(((j) => () => console.log(j))(i), 100);
}

```

2.6 Memory Implications of Closures

Closures keep their outer scope variables alive in memory as long as the closure itself exists. This can cause **memory leaks** if not handled carefully — especially with event listeners, timers, and subscriptions.

```
// Memory leak example
function setupHandler() {
  const hugeArray = new Array(1000000).fill('data');
  const element = document.getElementById('btn');
  element.addEventListener('click', () => {
    // This closure keeps 'hugeArray' alive even if we only need its length
    console.log(hugeArray.length);
  });
}
// Fix: only close over what you need
function setupHandlerFixed() {
  const hugeArray = new Array(1000000).fill('data');
  const len = hugeArray.length; // extract what you need
  const element = document.getElementById('btn');
  element.addEventListener('click', () => {
    console.log(len); // only 'len' is enclosed, hugeArray can be GC'd
  });
}
```

■ Interview Questions — Scope & Closures

Q: What is the output and why?

```
for (var i = 0; i < 5; i++) {
  setTimeout(function() { console.log(i); }, i * 1000);
}
```

A: Prints 5 five times (at 0s, 1s, 2s, 3s, 4s intervals). All five callbacks share the same `var i` (function-scoped). By the time any callback runs, the loop has finished and `i === 5`. Fix: use `let i` instead.

Q: How do closures enable the Module Pattern?

A: An IIFE returns an object with methods that close over private variables. The returned object is the public API, while internal state stays hidden. This provides encapsulation without classes — used extensively before ES6 modules.

Q: Can closures cause memory leaks? How?

A: Yes. If a closure references a large object from its outer scope, and the closure lives long (e.g., event listener never removed, timer never cleared), the large object can't be garbage collected. Fix: null out references, remove listeners, extract only needed values.

3. The this Keyword

Checklist: Implicit binding · explicit binding (call/apply/bind) · new binding · arrow function binding · default binding · precedence rules

3.1 The Five Binding Rules

Rule	this equals	Example
Default binding	window (sloppy) / undefined (strict)	foo() — standalone call
Implicit binding	The object before the dot	obj.foo() → this === obj
Explicit binding	Whatever you pass	foo.call(myObj), foo.apply(myObj), foo.bind(myObj)
new binding	The newly created object	new Foo() → this is the new instance
Arrow function	Lexical — inherits from enclosing scope	() => this — this is determined at definition time

3.2 Binding Precedence (highest → lowest)

new → call/apply/bind → implicit (obj.method) → default (standalone)

3.3 call vs apply vs bind

```
function introduce(greeting, punctuation) {
  console.log(`${greeting}, I'm ${this.name}${punctuation}`);
}

const person = { name: 'Sid' };

// call – invokes immediately, args passed individually
introduce.call(person, 'Hello', '!'); // "Hello, I'm Sid!"

// apply – invokes immediately, args passed as array
introduce.apply(person, ['Hi', '.']); // "Hi, I'm Sid."

// bind – returns a NEW function with 'this' permanently set
const boundIntro = introduce.bind(person, 'Hey');
boundIntro('?'); // "Hey, I'm Sid?"
// Can be called later, passed as callback, etc.
```

3.4 Implicit Binding & the "Lost this" Problem

```
const user = {
  name: 'Sid',
  greet() { console.log(`Hi, I'm ${this.name}`); }
};

user.greet(); // "Hi, I'm Sid" – implicit: this = user

const fn = user.greet; // extracting the method
fn(); // "Hi, I'm undefined" – default binding: this = window/undefined

// This is why event handlers lose 'this':
// button.addEventListener('click', user.greet); // 'this' will be the button, not user

// Fixes:
```

```
// 1. bind
// button.addEventListener('click', user.greet.bind(user));
// 2. Arrow wrapper
// button.addEventListener('click', () => user.greet());
// 3. Arrow method (defined in constructor/class field)
class Component {
  name = 'Angular';
  handleClick = () => { console.log(this.name); }; // arrow: lexical this
}
```

3.5 Arrow Functions & Lexical this

```
const team = {
  name: 'Frontend',
  members: ['Alice', 'Bob', 'Charlie'],

  // Arrow function inherits 'this' from team object's method
  listMembers() {
    this.members.forEach(member => {
      console.log(`${member} is on ${this.name}`);
      // Arrow: 'this' is the team object (from listMembers scope)
    });
  },

  // Regular function would have its own 'this' (undefined in forEach)
  listMembersBroken() {
    this.members.forEach(function(member) {
      console.log(`${member} is on ${this.name}`);
      // 'this' is undefined (strict) or window (sloppy) – NOT team!
    });
  }
};
team.listMembers(); // Works: "Alice is on Frontend", etc.
```

■ Interview Questions — this Keyword

Q: What will this code print?

```
const obj = {
  value: 42,
  getValue: function() { return this.value; },
  getValueArrow: () => this.value
};
console.log(obj.getValue());
console.log(obj.getValueArrow());
```

A: 42 and undefined. `getValue` is a regular method — implicit binding gives `this === obj`. `getValueArrow` is an arrow function — it inherits `this` from the surrounding scope (module/global), not from `obj`. Arrow functions should not be used as object methods.

Q: Can you use `call`/`apply` to change the `this` of an arrow function?

A: No. Arrow functions have lexical `this` that is determined at definition time and cannot be changed by `call`, `apply`, `bind`, or `new`.

4. Prototypes & Inheritance

Checklist: Prototype chain · `__proto__` vs `prototype` · `Object.create()` · constructor functions · class syntax · prototypal inheritance · `hasOwnProperty()` · `Object.getPrototypeOf()`

4.1 The Prototype Chain

Every JavaScript object has a hidden internal link called `[[Prototype]]` (accessible via `__proto__` or `Object.getPrototypeOf()`). When you access a property on an object, JS first looks on the object itself, then walks up the prototype chain until it finds the property or reaches `null`.

```
const animal = {
  alive: true,
  speak() { return `${this.name} makes a sound`; }
};

const dog = Object.create(animal); // dog's prototype is animal
dog.name = 'Rex';
dog.bark = function() { return `${this.name} barks!`; };

dog.bark(); // "Rex barks!" – found on dog itself
dog.speak(); // "Rex makes a sound" – found on prototype (animal)
dog.alive; // true – found on prototype

// Chain: dog → animal → Object.prototype → null
Object.getPrototypeOf(dog) === animal; // true
dog.hasOwnProperty('name'); // true – own property
dog.hasOwnProperty('speak'); // false – inherited
```

4.2 `__proto__` vs `prototype`

Term	What it is	Who has it
<code>__proto__</code>	Reference to the object's prototype (the thing it inherits from)	Every object
<code>.prototype</code>	An object that will become <code>__proto__</code> of instances created with <code>new</code>	Only functions (constructors)

```
function Person(name) { this.name = name; }
Person.prototype.greet = function() { return `Hi, I'm ${this.name}`; };

const p = new Person('Sid');
p.__proto__ === Person.prototype; // true
Person.prototype.constructor === Person; // true
Object.getPrototypeOf(p) === Person.prototype; // true (preferred over __proto__)
```

4.3 Constructor Functions & Class Syntax

```
// Constructor function (ES5)
function Vehicle(make, model) {
  this.make = make;
  this.model = model;
}
Vehicle.prototype.describe = function() {
  return `${this.make} ${this.model}`;
};

// Class syntax (ES6) – syntactic sugar over prototypes
class Car extends Vehicle {
  constructor(make, model, year) {
    super(make, model); // calls Vehicle constructor
    this.year = year;
  }
}
```

```

    }
    describe() {
      return `${this.year} ${super.describe()}`;
    }
    static compare(a, b) { return a.year - b.year; }
  }

  const c = new Car('Toyota', 'Camry', 2024);
  c.describe(); // "2024 Toyota Camry"
  c instanceof Car; // true
  c instanceof Vehicle; // true

```

■ Interview Questions — Prototypes

Q: How would you implement inheritance without using classes?

A: Use `Object.create()` to set up the prototype chain and `ParentConstructor.call(this, ...)` in the child constructor to inherit instance properties. Set `Child.prototype = Object.create(Parent.prototype)` then fix `Child.prototype.constructor = Child`.

Q: What does `new` actually do under the hood?

A: Four steps: (1) Creates a new empty object, (2) Sets its `__proto__` to the constructor's `prototype`, (3) Calls the constructor with `this` bound to the new object, (4) Returns the new object (unless constructor explicitly returns an object).

5. Asynchronous JavaScript

Checklist: Callbacks · callback hell · Promises (states, methods) · Promise.all/race/allSettled/any · async/await · try/catch · microtasks vs macrotasks · event loop · call stack · task queue · setTimeout/setInterval · requestAnimationFrame

5.1 The Event Loop

JavaScript is **single-threaded**. The event loop is the mechanism that allows async operations. It works with:

- **Call Stack:** executes synchronous code, one frame at a time.
- **Web APIs:** browser-provided (setTimeout, fetch, DOM events) — run outside the JS engine.
- **Microtask Queue:** Promises, queueMicrotask(), MutationObserver. Processed **after each task, before rendering**.
- **Macrotask Queue:** setTimeout, setInterval, I/O, UI events. Processed **one per loop iteration**.

Priority: Call stack → ALL microtasks → ONE macrotask → ALL microtasks → render → next macrotask...

```
// Classic event loop question
console.log('1'); // sync → call stack
setTimeout(() => console.log('2'), 0); // macrotask queue
Promise.resolve().then(() => console.log('3')); // microtask queue
queueMicrotask(() => console.log('4')); // microtask queue
console.log('5'); // sync → call stack

// Output: 1, 5, 3, 4, 2
// Sync first (1, 5) → microtasks (3, 4) → macrotask (2)
```

5.2 Callbacks & Callback Hell

```
// Callback hell – deeply nested, hard to read/maintain
getUser(userId, function(user) {
  getOrders(user.id, function(orders) {
    getOrderDetails(orders[0].id, function(details) {
      getShipping(details.shippingId, function(shipping) {
        console.log(shipping); // pyramid of doom!
      });
    });
  });
});
```

5.3 Promises — States & Methods

State	Description	Transition
pending	Initial state — operation in progress	→ fulfilled OR rejected
fulfilled	Operation completed successfully	Final (immutable)
rejected	Operation failed	Final (immutable)

```
// Creating promises
const promise = new Promise((resolve, reject) => {
  // async operation...
  if (success) resolve(data);
  else reject(new Error('Failed'));
});

// Chaining – each .then returns a new promise
fetch('/api/user')
  .then(res => res.json()) // returns new promise
  .then(user => fetch(`/api/orders/${user.id}`))
  .then(res => res.json())
```

```
.catch(err => console.error(err)) // catches ANY error in the chain
.finally(() => hideSpinner());    // runs regardless
```

5.4 Promise Static Methods

Method	Resolves when	Rejects when	Use case
<code>Promise.all([])</code>	ALL fulfill	FIRST rejects	Parallel API calls where all are needed
<code>Promise.race([])</code>	FIRST settles	FIRST settles	Timeout pattern, fastest response
<code>Promise.allSettled([])</code>	ALL settle	Never rejects	Batch operations, partial success OK
<code>Promise.any([])</code>	FIRST fulfills	ALL reject (AggregateError)	Fastest CDN, fallback APIs

```
// Promise.allSettled – dashboard loading multiple widgets
const results = await Promise.allSettled([
  fetch('/api/users'),
  fetch('/api/orders'),
  fetch('/api/analytics')
]);
results.forEach((result, i) => {
  if (result.status === 'fulfilled') renderWidget(i, result.value);
  else showWidgetError(i, result.reason);
});

// Promise.race – timeout pattern
function fetchWithTimeout(url, ms) {
  return Promise.race([
    fetch(url),
    new Promise( (_, reject) => setTimeout(() => reject(new Error('Timeout')), ms))
  ]);
}
```

5.5 async/await

```
// Syntactic sugar over Promises – makes async code look synchronous
async function loadDashboard(userId) {
  try {
    const user = await fetchUser(userId); // waits for promise
    const [orders, prefs] = await Promise.all([ // parallel await
      fetchOrders(user.id),
      fetchPreferences(user.id)
    ]);
    return { user, orders, prefs };
  } catch (error) {
    if (error instanceof NetworkError) {
      return getCacheDashboard(userId); // graceful fallback
    }
    throw error; // re-throw unknown errors
  } finally {
    hideLoadingSpinner();
  }
}

// Top-level await (ES2022, in modules)
const config = await fetch('/config.json').then(r => r.json());
```

5.6 requestAnimationFrame


```
// Smooth animation – runs before next repaint (~60fps)
function animateProgress(current, target) {
  if (current < target) {
    current += 2;
    progressBar.style.width = `${current}%`;
    requestAnimationFrame(() => animateProgress(current, target));
  }
}
requestAnimationFrame(() => animateProgress(0, 100));
```

■ Interview Questions — Async JavaScript

Q: What's the output?

```
async function foo() { return 1; }
const p = foo();
console.log(p);
p.then(v => console.log(v));
```

A: First logs `Promise { <fulfilled>: 1 }`, then `1`. `async` always returns a promise. The `.then` callback is a microtask, so it logs after the synchronous `console.log`.

Q: Difference between microtask and macrotask?

A: Microtasks (Promises, `queueMicrotask`) are processed immediately after the current task completes, before any macrotask. Macrotasks (`setTimeout`, `setInterval`) are processed one per event loop iteration. This means one `setTimeout` callback + ALL queued promise callbacks before the next `setTimeout` callback.

Q: How would you implement parallel API calls with error handling for each?

A: Use `Promise.allSettled()` which waits for all promises and returns their statuses, or wrap each `fetch` in its own `try/catch` and use `Promise.all()` on the wrapped versions.

6. ES6+ Features

Checklist: Destructuring · spread/rest · template literals · default params · enhanced object literals · computed property names · for...of · iterators & generators · Symbol · Map/Set · WeakMap/WeakSet · optional chaining · nullish coalescing · logical assignment operators

6.1 Destructuring

```
// Object destructuring – rename, default, nested
const response = { data: { user: { name: 'Sid', age: 28 } }, status: 200 };
const { data: { user: { name: userName, age, role = 'user' } }, status } = response;
// userName='Sid', age=28, role='user' (default), status=200

// Array destructuring – skip elements, rest
const [first, , third, ...remaining] = [1, 2, 3, 4, 5];
// first=1, third=3, remaining=[4,5]

// Swap variables
let a = 1, b = 2;
[a, b] = [b, a]; // a=2, b=1

// Function parameter destructuring
function createUser({ name, email, role = 'viewer' }) {
  return { name, email, role, createdAt: new Date() };
}
```

6.2 Spread & Rest Operators

```
// Spread – expand iterable into individual elements
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3, b: 99 }; // { a:1, b:99, c:3 } – last wins

// Rest – collect remaining elements
function sum(...numbers) { return numbers.reduce((a, b) => a + b, 0); }
sum(1, 2, 3, 4); // 10

// Immutable array operations (critical for Angular signals/state)
const items = [1, 2, 3];
const added = [...items, 4]; // add
const removed = items.filter(x => x !== 2); // remove
const updated = items.map(x => x === 2 ? 99 : x); // update
```

6.3 Template Literals & Tagged Templates

```
// Multi-line strings + interpolation
const html = `
  <div class="card">
    <h3>${user.name}</h3>
    <p>${user.bio || 'No bio provided'}</p>
  </div>
`;

// Tagged templates – custom string processing
function highlight(strings, ...values) {
  return strings.reduce((result, str, i) => {
    return result + str + (values[i] ? `${values[i]}` : '');
  }, '');
}
const name = 'Sid';
highlight`Hello ${name}, welcome!`; // "Hello Sid, welcome!"
```

6.4 Map, Set, WeakMap, WeakSet

```
// Map – keys can be ANY type (not just strings)
const cache = new Map();
cache.set(userObj, { data: 'cached', timestamp: Date.now() });
```

```

cache.get(userObj); // { data: 'cached', ... }
cache.has(userObj); // true
cache.size;         // 1
cache.delete(userObj);

// Set – unique values only
const unique = new Set([1, 2, 2, 3, 3, 3]); // Set {1, 2, 3}
unique.add(4);
unique.has(2); // true
[...unique];    // [1, 2, 3, 4]

// WeakMap – keys must be objects, entries are garbage-collectable
const metadata = new WeakMap();
function process(element) {
  if (!metadata.has(element)) {
    metadata.set(element, { processed: true, timestamp: Date.now() });
  }
}
// When 'element' is GC'd, the WeakMap entry is automatically removed
// → No memory leaks! Perfect for caching DOM node metadata.

```

6.5 Iterators & Generators

```

// Iterator protocol – any object with next() method
const range = {
  from: 1, to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        return current <= last
          ? { value: current++, done: false }
          : { done: true };
      }
    };
  }
};

for (const n of range) console.log(n); // 1, 2, 3, 4, 5

// Generator – lazy evaluation, produces values on demand
function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const fib = fibonacci();
fib.next().value; // 0
fib.next().value; // 1
fib.next().value; // 1
fib.next().value; // 2

```

6.6 Optional Chaining & Nullish Coalescing

```

// Optional chaining (?.) – short-circuits to undefined
const street = user?.address?.street; // property access
const city = user?.addresses?.[0]?.city; // array access
const result = user?.getProfile?.(); // method call

// Nullish coalescing (??) – only checks null/undefined (NOT falsy)
const port = config.port ?? 3000; // uses 3000 only if port is null/undefined
const port2 = config.port || 3000; // uses 3000 if port is 0, '', false, null, undefined
// For port=0: ?? gives 0, || gives 3000

// Logical assignment operators (ES2021)

```

```
let opts = { timeout: 0, retries: null, debug: undefined };
opts.timeout ||= 5000; // stays 0 (0 is falsy, ||= assigns)
opts.timeout ??= 5000; // stays 0 (0 is not null/undefined)
opts.retries ??= 3;     // becomes 3 (null is nullish)
opts.debug ??= false;   // becomes false
opts.name &&= opts.name.trim(); // only trims if name is truthy
```

■ Interview Questions — ES6+

Q: When would you use Map over a plain object?

A: Use Map when: keys are not strings (objects, functions, numbers), you need to know the size (`.size`), you need ordered iteration, or you frequently add/remove entries (Maps are optimized for this). Use plain objects for simple string-key config/data.

Q: What's the difference between `||` and `??`?

A: `||` returns the right side if the left is **falsy** (false, 0, "", null, undefined, NaN). `??` returns the right side only if the left is **null or undefined**. Use `??` when 0, false, or "" are valid values.

7. Advanced JavaScript Concepts

Checklist: Higher-order functions · pure functions · function composition · currying · partial application · memoization · recursion · tail call optimization · debouncing · throttling · Proxy & Reflect · property descriptors · Object.defineProperty · getters/setters · immutability

7.1 Higher-Order Functions & Pure Functions

A **higher-order function** takes a function as argument or returns a function. Examples: `map`, `filter`, `reduce`, `forEach`, `setTimeout`, event listeners.

A **pure function** always returns the same output for the same input and has no side effects (no mutations, no I/O, no global state changes).

```
// Pure function – predictable, testable
const add = (a, b) => a + b; // always returns same result, no side effects

// Impure function – depends on external state
let total = 0;
const addToTotal = (n) => { total += n; return total; }; // side effect!
```

7.2 Currying & Partial Application

```
// Currying – transforms f(a, b, c) into f(a)(b)(c)
const curry = (fn) => {
  const arity = fn.length;
  return function curried(...args) {
    if (args.length >= arity) return fn(...args);
    return (...moreArgs) => curried(...args, ...moreArgs);
  };
};

const multiply = curry((a, b, c) => a * b * c);
multiply(2)(3)(4); // 24
multiply(2, 3)(4); // 24
multiply(2)(3, 4); // 24

// Practical: reusable config
const log = curry((level, timestamp, message) => {
  console.log(`[${level}] ${timestamp}: ${message}`);
});
const errorLog = log('ERROR');
const errorLogNow = errorLog(new Date().toISOString());
errorLogNow('Server unreachable'); // "[ERROR] 2024-...: Server unreachable"
```

7.3 Debounce & Throttle

```
// Debounce – waits for pause in calls, then executes ONCE
function debounce(fn, delay, immediate = false) {
  let timer;
  return function(...args) {
    const callNow = immediate && !timer;
    clearTimeout(timer);
    timer = setTimeout(() => {
      timer = null;
      if (!immediate) fn.apply(this, args);
    }, delay);
    if (callNow) fn.apply(this, args);
  };
}

// Use: search input → API call after user stops typing (300ms)
const search = debounce((term) => fetchResults(term), 300);

// Throttle – max one execution per interval
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
```

```

    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
// Use: scroll handler – fire at most once per 100ms
window.addEventListener('scroll', throttle(updatePosition, 100));

```

7.4 Proxy & Reflect

```

// Proxy – intercept and customize object operations
const validator = new Proxy({}, {
  set(target, prop, value) {
    if (prop === 'age') {
      if (typeof value !== 'number') throw new TypeError('Age must be a number');
      if (value < 0 || value > 150) throw new RangeError('Invalid age');
    }
    target[prop] = value;
    return true;
  },
  get(target, prop) {
    if (prop in target) return target[prop];
    throw new ReferenceError(`Property '${prop}' does not exist`);
  }
});
validator.age = 25; // ■
// validator.age = -5; // RangeError

// Reflect – provides default behavior for Proxy traps
const logged = new Proxy(apiService, {
  get(target, prop, receiver) {
    console.log(`Accessing: ${String(prop)}`);
    return Reflect.get(target, prop, receiver);
  }
});

```

7.5 Property Descriptors & Getters/Setters

```

// Object.defineProperty – fine-grained control
const config = {};
Object.defineProperty(config, 'apiKey', {
  value: 'secret123',
  writable: false, // cannot be changed
  enumerable: false, // won't show in for...in or Object.keys
  configurable: false // cannot be deleted or reconfigured
});
config.apiKey = 'hacked'; // silently fails (throws in strict mode)

// Getters and setters
class Temperature {
  #celsius;
  constructor(celsius) { this.#celsius = celsius; }

  get fahrenheit() { return this.#celsius * 9/5 + 32; }
  set fahrenheit(f) { this.#celsius = (f - 32) * 5/9; }
  get celsius() { return this.#celsius; }
  set celsius(c) { this.#celsius = c; }
}
const temp = new Temperature(100);
temp.fahrenheit; // 212 – computed on access
temp.fahrenheit = 32;
temp.celsius; // 0

```

7.6 Memoization

```
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    const key = JSON.stringify(args);
    if (cache.has(key)) return cache.get(key);
    const result = fn.apply(this, args);
    cache.set(key, result);
    return result;
  };
}
const factorial = memoize(function f(n) {
  return n <= 1 ? 1 : n * f(n - 1);
});
factorial(10); // calculates
factorial(10); // returns from cache instantly
```

■ Interview Questions — Advanced Concepts

Q: Implement a debounce function from scratch.

A: See section 7.3 above. Key points: use `clearTimeout` to cancel previous timer, `setTimeout` to set new one, preserve `this` context with `.apply()`, and support immediate execution for leading-edge debounce.

Q: What's the difference between debounce and throttle?

A: Debounce waits for a pause in activity and then fires once. **Throttle** fires at regular intervals during continuous activity. Debounce for search input (wait for user to stop typing). Throttle for scroll/resize (fire regularly while scrolling).

8. Array Methods

Checklist: map · filter · reduce · forEach · some · every · find · findIndex · slice · splice · concat · join · split · sort · flat · flatMap · Array.from/of · includes · indexOf

8.1 Transforming: map, filter, reduce

```
const products = [
  { name: 'Laptop', price: 1200, inStock: true },
  { name: 'Phone', price: 800, inStock: false },
  { name: 'Tablet', price: 500, inStock: true },
  { name: 'Watch', price: 300, inStock: true }
];

// map – transform each element (returns new array)
const names = products.map(p => p.name); // ['Laptop', 'Phone', 'Tablet', 'Watch']

// filter – keep elements matching condition
const available = products.filter(p => p.inStock); // 3 items

// reduce – accumulate into single value
const totalValue = products.reduce((sum, p) => sum + p.price, 0); // 2800

// Chaining – real-world data pipeline
const avgInStockPrice = products
  .filter(p => p.inStock)
  .map(p => p.price)
  .reduce((sum, price, _, arr) => sum + price / arr.length, 0);
// (1200 + 500 + 300) / 3 = 666.67
```

8.2 Searching: find, findIndex, some, every, includes

```
// find – returns first matching element (or undefined)
const phone = products.find(p => p.name === 'Phone'); // { name: 'Phone', ... }

// findIndex – returns index (or -1)
const idx = products.findIndex(p => p.price > 1000); // 0

// some – returns true if ANY element matches
const hasExpensive = products.some(p => p.price > 1000); // true

// every – returns true if ALL elements match
const allInStock = products.every(p => p.inStock); // false

// includes – simple value check
[1, 2, 3].includes(2); // true
```

8.3 Mutating vs Non-Mutating

Non-mutating (returns new)	Mutating (changes original)
map, filter, reduce	push, pop, shift, unshift
slice, concat, flat	splice, sort, reverse, fill
find, some, every	

```
// slice vs splice
const arr = [1, 2, 3, 4, 5];
arr.slice(1, 3); // [2, 3] – does NOT modify arr
arr.splice(1, 2); // [2, 3] – REMOVES from arr, arr is now [1, 4, 5]

// sort – MUTATES! Always copy first
const prices = [300, 1200, 500, 800];
const sorted = [...prices].sort((a, b) => a - b); // [300, 500, 800, 1200]
```



```
// prices is still [300, 1200, 500, 800]

// flat & flatMap
const nested = [1, [2, 3], [4, [5, 6]]];
nested.flat(); // [1, 2, 3, 4, [5, 6]]
nested.flat(Infinity); // [1, 2, 3, 4, 5, 6]

// flatMap = map + flat(1)
const sentences = ['hello world', 'foo bar'];
sentences.flatMap(s => s.split(' ')); // ['hello', 'world', 'foo', 'bar']

// Array.from – convert iterables/array-like to arrays
Array.from({ length: 5 }, (_, i) => i * 2); // [0, 2, 4, 6, 8]
Array.from(document.querySelectorAll('li')); // NodeList → Array
```

■ Interview Questions — Array Methods

Q: Implement `Array.prototype.reduce` from scratch.

```
Array.prototype.myReduce = function(callback, initialValue) {
  let accumulator = initialValue;
  let startIndex = 0;
  if (accumulator === undefined) {
    if (this.length === 0) throw new TypeError('Reduce of empty array with no initial value');
    accumulator = this[0];
    startIndex = 1;
  }
  for (let i = startIndex; i < this.length; i++) {
    accumulator = callback(accumulator, this[i], i, this);
  }
  return accumulator;
};
```

Q: What's the difference between `map` and `forEach`?

A: `map` returns a new array with transformed values. `forEach` returns `undefined` — it's for side effects only. `map` is chainable, `forEach` is not. If you need the result, use `map`.

9. Object Methods

Checklist: Object.keys/values/entries · Object.assign · Object.freeze/seal · Object.is · spread syntax · shallow vs deep cloning

```
const user = { name: 'Sid', age: 28, role: 'developer' };

// Iteration methods
Object.keys(user); // ['name', 'age', 'role']
Object.values(user); // ['Sid', 28, 'developer']
Object.entries(user); // [['name','Sid'], ['age',28], ['role','developer']]

// Object.fromEntries – reverse of entries
const params = new URLSearchParams('page=1&sort=name');
const obj = Object.fromEntries(params); // { page: '1', sort: 'name' }

// Object.assign – merge objects (shallow)
const defaults = { theme: 'light', lang: 'en', debug: false };
const overrides = { theme: 'dark', debug: true };
const config = Object.assign({}, defaults, overrides);
// { theme: 'dark', lang: 'en', debug: true }

// Object.freeze vs Object.seal
const frozen = Object.freeze({ a: 1, nested: { b: 2 } });
// frozen.a = 99; // ■ silently fails (throws in strict)
// frozen.newProp = 'x'; // ■ cannot add
frozen.nested.b = 99; // ■ freeze is SHALLOW – nested objects still mutable!

const sealed = Object.seal({ x: 1, y: 2 });
sealed.x = 99; // ■ can modify existing
// sealed.z = 3; // ■ cannot add new
// delete sealed.x; // ■ cannot delete

// Deep clone
const original = { a: 1, nested: { b: 2 }, arr: [1, 2] };
const deepCopy = structuredClone(original); // modern JS (2022)
deepCopy.nested.b = 99;
console.log(original.nested.b); // 2 – independent!
```

■ Interview Questions — Object Methods

Q: How do you deep clone an object in JavaScript?

A: (1) `structuredClone(obj)` — modern, handles circular refs, dates, maps. (2) `JSON.parse(JSON.stringify(obj))` — works for simple data but loses functions, undefined, Date objects, and fails on circular refs. (3) Recursive manual clone. (4) Libraries like lodash's `_.cloneDeep`.

10. Error Handling

Checklist: try/catch/finally · error types (TypeError, ReferenceError, SyntaxError) · custom error classes · error propagation · async error handling

```
// Built-in Error types
// TypeError: wrong type operation – e.g., null.toString()
// ReferenceError: accessing undeclared variable
// SyntaxError: invalid code – caught at parse time (can't catch with try/catch)
// RangeError: value out of range – e.g., new Array(-1)
// URIError: malformed URI – e.g., decodeURIComponent('%')

// Custom Error class
class HttpError extends Error {
  constructor(statusCode, message, endpoint) {
    super(message);
    this.name = 'HttpError';
    this.statusCode = statusCode;
    this.endpoint = endpoint;
  }

  get isClientError() { return this.statusCode >= 400 && this.statusCode < 500; }
  get isServerError() { return this.statusCode >= 500; }
}

// Error handling patterns
async function fetchData(url) {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new HttpError(res.status, res.statusText, url);
    return await res.json();
  } catch (error) {
    if (error instanceof HttpError && error.isClientError) {
      console.warn(`Client error at ${error.endpoint}: ${error.statusCode}`);
      return null; // graceful fallback
    }
    throw error; // re-throw server errors and network errors
  } finally {
    hideLoadingSpinner(); // always runs – cleanup
  }
}

// Global error handlers
window.addEventListener('error', (event) => {
  sendToErrorTracking({ message: event.message, stack: event.error?.stack });
});
window.addEventListener('unhandledrejection', (event) => {
  sendToErrorTracking({ message: event.reason?.message || 'Unhandled rejection' });
  event.preventDefault(); // prevent default browser logging
});
```

■ Interview Questions — Error Handling

Q: Does finally always execute?

A: Yes, finally always executes whether try succeeds, fails, or throws. Even if there's a return in try or catch, finally runs before the return actually completes. A return in finally overrides the return value from try/catch.

11. Modules

Checklist: ES6 modules (import/export) · named vs default exports · dynamic imports · CommonJS (require/module.exports) · module bundling concepts

```
// ■■■ Named exports ■■■
// utils.js
export function formatDate(d) { return d.toISOString().split('T')[0]; }
export function formatCurrency(n) { return `$$${n.toFixed(2)}`; }
export const API_URL = '/api/v1';

// Importing named exports
import { formatDate, formatCurrency } from './utils.js';
import { formatDate as fmtDate } from './utils.js'; // rename
import * as Utils from './utils.js'; // namespace

// ■■■ Default export ■■■
// logger.js
export default class Logger {
  log(msg) { console.log(`[LOG] ${msg}`); }
}
import Logger from './logger.js'; // any name works for default

// ■■■ Re-exporting (barrel files) ■■■
// index.js
export { formatDate, formatCurrency } from './utils.js';
export { default as Logger } from './logger.js';

// ■■■ Dynamic imports – lazy loading ■■■
async function loadChart() {
  const { Chart } = await import('./chart-library.js'); // loaded on demand
  return new Chart(canvas, config);
}
// Use case: Angular lazy-loaded routes, heavy libraries

// ■■■ CommonJS (Node.js) ■■■
// math.js
module.exports = { add: (a, b) => a + b };
// app.js
const { add } = require('./math.js');
// Key difference: CommonJS is synchronous, ES modules are async-capable
```

■ Interview Questions — Modules

Q: What's tree shaking and how do modules enable it?

A: Tree shaking removes unused code during bundling. It works with ES modules because their imports/exports are **static** — determined at compile time, not runtime. CommonJS `require()` is dynamic and can't be tree-shaken reliably.

12. Regular Expressions

Checklist: Pattern matching · flags (g, i, m, s, u, y) · character classes · quantifiers · groups & capturing · lookahead/lookbehind · string methods (match, search, replace, split)

```
// Common patterns for interviews
const patterns = {
  email: /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/ ,
  phone: /^[\+]?(\d{1,3})?[-.\s](\d{1,4})?[-.\s](\d{1,4})?[-.\s](\d{1,9})?$/,
  url: /^https?:\/\/(www\.)?[\w.-]+(:\d+)?(\/[\w.%-]*)?(?[\&\w=-]*)?(#[\w-]*)?$/,
  strongPassword: /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/
};

// Named capture groups
const dateStr = '2024-03-15';
const { groups: { year, month, day } } =
  dateStr.match(/(?<=\\$)\\d{4})-(?<=\\$)\\d{2})-(?<=\\$)\\d{2})/);

// Lookahead & lookbehind
'100px 200em 50%'.match(/\\d+(?=px)/g); // ['100'] — followed by 'px'
'$100 €200 £50'.match(/(?<=\\$)\\d+/g); // ['100'] — preceded by '$'

// Replace with function
const camelCase = 'backgroundColor'.replace(/([A-Z])/g, '-$1').toLowerCase();
// 'background-color'

// Template parsing
const template = 'Hello {{name}}, you have {{count}} messages';
const data = { name: 'Sid', count: 42 };
template.replace(/\\{\\(\\w+)\\}\\}/g, (_, key) => data[key] ?? '');
// 'Hello Sid, you have 42 messages'

// Flags
// g: global — find all matches
// i: case-insensitive
// m: multiline — ^ and $ match line boundaries
// s: dotAll — . matches newlines
// u: unicode — correct unicode handling
// y: sticky — match at exact position
```

■ Interview Questions — Regex

Q: Write a regex to validate an email address.

A: `/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/`. Note: true email validation per RFC 5322 is extremely complex. In production, send a verification email instead of relying solely on regex.

13. DOM Manipulation

Checklist: `getElementById` · `querySelector/All` · `createElement` · `appendChild` · `removeChild` · `insertBefore` · `insertAdjacentElement` · `cloneNode` · `innerHTML` vs `textContent` vs `innerText` · `classList` · `setAttribute/getAttribute` · `dataset` · `DocumentFragment`

13.1 Selecting Elements

```
// By ID – returns single element or null
const header = document.getElementById('header');

// CSS selector – returns first match or null
const card = document.querySelector('.product-card');
const activeTab = document.querySelector('[data-active="true"]');

// CSS selector – returns NodeList (static snapshot, NOT live)
const allCards = document.querySelectorAll('.product-card');
allCards.forEach(card => card.classList.add('loaded'));

// LIVE collections – update automatically when DOM changes
const divs = document.getElementsByTagName('div'); // HTMLCollection (live)
const items = document.getElementsByClassName('item'); // HTMLCollection (live)
```

13.2 Creating & Modifying Elements

```
// Create and configure an element
const card = document.createElement('div');
card.className = 'product-card';
card.id = 'card-42';
card.dataset.productId = '42'; // sets data-product-id="42"
card.setAttribute('role', 'article');
card.textContent = 'Product Name'; // SAFE – no HTML injection
// card.innerHTML = '<h3>Title</h3>'; // CAREFUL – can inject scripts!

// Append to DOM
document.getElementById('container').appendChild(card);

// Insert at specific positions
parent.insertBefore(newNode, referenceNode); // before reference
element.insertAdjacentElement('beforebegin', newEl); // before element
element.insertAdjacentElement('afterbegin', newEl); // first child
element.insertAdjacentElement('beforeend', newEl); // last child
element.insertAdjacentElement('afterend', newEl); // after element

// Modern: append, prepend, before, after, replaceWith
parent.append(child1, child2, 'text'); // can append multiple + text
parent.prepend(child); // add as first child
node.before(newNode); // insert before
node.after(newNode); // insert after
node.replaceWith(newNode); // replace
node.remove(); // remove from DOM
```

13.3 innerHTML vs textContent vs innerText

Property	Behavior	XSS Safe?	Performance
<code>textContent</code>	Gets/sets raw text, ignores HTML	■ Yes	Fast
<code>innerText</code>	Gets visible text only (respects CSS), triggers reflow	■ Yes	Slow

Property	Behavior	XSS Safe?	Performance
innerHTML	Gets/sets HTML markup, parses as HTML	■ Dangerous!	Medium

13.4 classList API

```
const el = document.querySelector('.btn');
el.classList.add('loading', 'disabled'); // add multiple
el.classList.remove('loading');          // remove
el.classList.toggle('active');            // add if absent, remove if present
el.classList.toggle('visible', condition); // force add/remove based on boolean
el.classList.contains('active');          // check → true/false
el.classList.replace('old-class', 'new-class'); // swap
```

13.5 DocumentFragment — Batch DOM Updates

```
// BAD: 1000 individual DOM insertions → 1000 reflows
for (let i = 0; i < 1000; i++) {
  const li = document.createElement('li');
  li.textContent = `Item ${i}`;
  list.appendChild(li); // triggers reflow each time!
}

// GOOD: DocumentFragment → single reflow
const fragment = document.createDocumentFragment();
for (let i = 0; i < 1000; i++) {
  const li = document.createElement('li');
  li.textContent = `Item ${i}`;
  fragment.appendChild(li); // fragment is in memory, no reflow
}
list.appendChild(fragment); // ONE reflow for all 1000 items
```

■ Interview Questions — DOM Manipulation

Q: Why should you never use innerHTML with user input?

A: It creates an XSS vulnerability. If userInput contains `<script>alert('hacked')</script>`, setting `element.innerHTML = userInput` executes the script. Always use `textContent` for user-provided data.

Q: Why use DocumentFragment?

A: DOM operations trigger layout recalculations (reflows). Inserting 1000 elements one-by-one causes 1000 reflows. A DocumentFragment exists in memory — you build the entire tree in the fragment, then insert it once = 1 reflow.

14. DOM Traversal

Checklist: parentNode/parentElement · childNodes/children · firstChild/lastChild · firstElementChild/lastElementChild · nextSibling/previousSibling · nextElementSibling/previousElementSibling · closest()

```
// Node vs Element properties
// Node includes text nodes, comments, etc.
// Element only includes element nodes

const parent = document.querySelector('.list');
parent.childNodes; // NodeList – includes text nodes, comments
parent.children; // HTMLCollection – only element children
parent.firstChild; // could be a text node (whitespace!)
parent.firstElementChild; // first actual element

// Sibling navigation
const item = document.querySelector('.item');
item.nextSibling; // might be text node (whitespace)
item.nextElementSibling; // next element
item.previousElementSibling; // previous element

// closest() – searches UP the DOM tree
const btn = document.querySelector('.delete-btn');
const card = btn.closest('.card'); // finds nearest ancestor with .card
const form = btn.closest('form'); // finds nearest ancestor form
const nothing = btn.closest('.nonexistent'); // null

// Walking the DOM tree
function walkDOM(node, callback) {
  callback(node);
  node = node.firstElementChild;
  while (node) {
    walkDOM(node, callback);
    node = node.nextElementSibling;
  }
}
walkDOM(document.body, el => console.log(el.tagName));
```

■ Interview Questions — DOM Traversal

Q: What's the difference between `childNodes` and `children`?

A: `childNodes` returns ALL child nodes including text nodes and comments (NodeList). `children` returns only element children (HTMLCollection). In practice, use `children` to avoid dealing with whitespace text nodes.

15. Events

Checklist: addEventListener/removeEventListener · event object (target, currentTarget, type) · bubbling · capturing · delegation · stopPropagation · preventDefault · keyboard/mouse/touch events · passive listeners · CustomEvent

15.1 Event Flow: Capture → Target → Bubble

```
// Events flow in three phases:
// 1. CAPTURING: window → document → html → body → ... → target (top-down)
// 2. TARGET: event fires on the target element
// 3. BUBBLING: target → ... → body → html → document → window (bottom-up)

// By default, listeners fire during BUBBLING phase
element.addEventListener('click', handler);

// Listen during CAPTURE phase
element.addEventListener('click', handler, { capture: true });
// or shorthand: element.addEventListener('click', handler, true);
```

15.2 target vs currentTarget

```
// target: the element that TRIGGERED the event (what was clicked)
// currentTarget: the element the LISTENER is attached to (where you're listening)

document.querySelector('.list').addEventListener('click', (e) => {
  console.log(e.target);          // the actual <li> or <span> clicked
  console.log(e.currentTarget);   // always the .list element
});
```

15.3 Event Delegation

```
// Instead of 100 listeners on 100 items, use ONE on the parent
document.getElementById('todo-list').addEventListener('click', (e) => {
  // Check if a delete button was clicked
  const deleteBtn = e.target.closest('[data-action="delete"]');
  if (deleteBtn) {
    const todoId = deleteBtn.closest('[data-todo-id]').dataset.todoId;
    deleteTodo(todoId);
    return;
  }

  // Check if a toggle was clicked
  const checkbox = e.target.closest('[data-action="toggle"]');
  if (checkbox) {
    toggleTodo(checkbox.closest('[data-todo-id]').dataset.todoId);
  }
});
// Benefits: works for dynamically added items, single listener, less memory
```

15.4 stopPropagation & preventDefault

```
// preventDefault – stops the browser's default action
form.addEventListener('submit', (e) => {
  e.preventDefault(); // stops form from navigating
  // handle submission via fetch instead
});

link.addEventListener('click', (e) => {
  e.preventDefault(); // stops navigation
  router.navigate(link.href);
});

// stopPropagation – stops event from continuing to parent
child.addEventListener('click', (e) => {
  e.stopPropagation(); // parent's click handler won't fire
});
```

```
// stopImmediatePropagation – stops OTHER handlers on same element too
element.addEventListener('click', handler1); // this fires
element.addEventListener('click', (e) => {
  e.stopImmediatePropagation();
  // handler after this on same element won't fire
});
```

15.5 Custom Events

```
// Create and dispatch custom events
const cartEvent = new CustomEvent('cart:updated', {
  bubbles: true,      // event bubbles up through DOM
  composed: true,     // crosses shadow DOM boundaries
  detail: { items: 5, total: 2499 }
});
element.dispatchEvent(cartEvent);

// Listen for custom events
document.addEventListener('cart:updated', (e) => {
  updateCartBadge(e.detail.items);
});
```

15.6 Passive Event Listeners

```
// Passive listeners tell the browser "I won't call preventDefault()"
// This allows the browser to scroll smoothly without waiting for JS
document.addEventListener('touchstart', handleTouch, { passive: true });
document.addEventListener('wheel', handleWheel, { passive: true });

// Cleanup with AbortController
const controller = new AbortController();
element.addEventListener('click', handler, { signal: controller.signal });
element.addEventListener('keydown', handler2, { signal: controller.signal });
// Remove ALL listeners at once:
controller.abort();
```

■ Interview Questions — Events

Q: Explain event delegation. Why is it useful?

A: Event delegation attaches a single listener to a parent element instead of individual listeners on each child. It works because events bubble up. Benefits: (1) Works for dynamically added elements, (2) Less memory (1 listener vs N), (3) Less setup code. Use `e.target.closest(selector)` to identify which child was interacted with.

Q: What's the difference between stopPropagation and preventDefault?

A: `preventDefault()` stops the browser's default behavior (navigation, form submission) but the event still propagates. `stopPropagation()` stops the event from bubbling to parent elements but the default action still occurs. They solve different problems.

16. Browser Storage

Checklist: localStorage · sessionStorage · storage events · cookies (document.cookie, attributes: expires, max-age, domain, path, secure, SameSite) · IndexedDB basics

Storage	Capacity	Lifetime	Scope	Sent with requests?
localStorage	~5-10 MB	Until manually cleared	Same origin (protocol+host+port)	No
sessionStorage	~5-10 MB	Until tab closes	Same origin + same tab	No
Cookies	~4 KB per cookie	Expires or session	Origin + path	Yes — every HTTP request!
IndexedDB	50 MB+ (unlimited with permission)	Until cleared	Same origin	No

```
// localStorage – persists across browser restarts
localStorage.setItem('theme', JSON.stringify({ mode: 'dark', fontSize: 16 }));
const theme = JSON.parse(localStorage.getItem('theme'));
localStorage.removeItem('theme');
localStorage.clear(); // remove ALL items

// Cross-tab sync via storage event
window.addEventListener('storage', (e) => {
  if (e.key === 'user-session') {
    if (e.newValue === null) logoutCurrentTab(); // logged out in another tab
  }
});

// Cookies – set from JavaScript (NOT for HttpOnly cookies)
document.cookie = 'name=Sid; max-age=86400; path=/; SameSite=Lax; Secure';
// Reading cookies (messy – it's one big string)
const cookies = Object.fromEntries(
  document.cookie.split('; ').map(c => c.split('='))
);

// IndexedDB – async, structured data, large capacity
const request = indexedDB.open('MyDatabase', 1);
request.onupgradeneeded = (e) => {
  const db = e.target.result;
  const store = db.createObjectStore('products', { keyPath: 'id' });
  store.createIndex('category', 'category', { unique: false });
};
```

■ Interview Questions — Browser Storage

Q: Where should you store JWT tokens?

A: HttpOnly cookies are most secure — not accessible from JS (immune to XSS). localStorage is convenient but exposed to XSS attacks. Never store sensitive tokens in sessionStorage or non-HttpOnly cookies if XSS is a concern. Use Secure; SameSite=Strict; HttpOnly cookie attributes.

17. Browser APIs

Checklist: Fetch API · XMLHttpRequest · Geolocation · History API · Web Workers · Service Workers · IntersectionObserver · MutationObserver · ResizeObserver · Performance API · requestIdleCallback · Page Visibility · Clipboard · Notification · Drag & Drop

17.1 IntersectionObserver — Lazy Loading / Infinite Scroll

```
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;    // load real image
      img.classList.add('loaded');
      observer.unobserve(img);      // stop watching
    }
  });
}, {
  root: null,                      // viewport
  rootMargin: '200px',            // start loading 200px before visible
  threshold: 0.1                  // trigger when 10% visible
});
document.querySelectorAll('img[data-src]').forEach(img => observer.observe(img));
```

17.2 MutationObserver — Watch DOM Changes

```
const observer = new MutationObserver((mutations) => {
  mutations.forEach(mutation => {
    if (mutation.type === 'childList') {
      console.log('Children changed:', mutation.addedNodes, mutation.removedNodes);
    }
    if (mutation.type === 'attributes') {
      console.log(`Attribute '${mutation.attributeName}' changed`);
    }
  });
});
observer.observe(targetElement, {
  childList: true,    // watch for added/removed children
  attributes: true,   // watch for attribute changes
  subtree: true       // watch entire subtree
});
```

17.3 ResizeObserver

```
const resizeObserver = new ResizeObserver(entries => {
  for (const entry of entries) {
    const { width, height } = entry.contentRect;
    entry.target.classList.toggle('compact', width < 400);
    entry.target.classList.toggle('wide', width >= 800);
  }
});
resizeObserver.observe(document.querySelector('.dashboard-widget'));
```

17.4 Page Visibility & Clipboard

```
// Pause expensive work when tab is hidden
document.addEventListener('visibilitychange', () => {
  if (document.hidden) { pausePolling(); pauseAnimations(); }
  else { resumePolling(); }
});

// Clipboard API (async, requires HTTPS)
async function copyToClipboard(text) {
  try {
    await navigator.clipboard.writeText(text);
    showToast('Copied!');
  } catch (err) {
    // Fallback for older browsers
  }
}
```

```

    const textarea = document.createElement('textarea');
    textarea.value = text;
    document.body.appendChild(textarea);
    textarea.select();
    document.execCommand('copy');
    textarea.remove();
  }
}

// Performance API – custom timings
performance.mark('api-start');
await fetch('/api/data');
performance.mark('api-end');
performance.measure('API Call', 'api-start', 'api-end');
const duration = performance.getEntriesByName('API Call')[0].duration;

```

17.5 History API

```

// SPA routing fundamentals
history.pushState({ page: 'products' }, '', '/products'); // add to history
history.replaceState({ page: 'home' }, '', '/'); // replace current
window.addEventListener('popstate', (e) => {
  console.log('Navigation:', e.state); // user clicked back/forward
  renderPage(e.state?.page);
});

```

18. Window & Document

Checklist: window.location · window.history · window.scrollTo/scrollBy · document.documentElement · document.readyState · DOMContentLoaded vs load · getBoundingClientRect · viewport dimensions

```
// window.location – URL manipulation
window.location.href;           // full URL
window.location.pathname;       // '/products/42'
window.location.search;         // '?sort=price&order=asc'
window.location.hash;           // '#reviews'
window.location.origin;         // 'https://example.com'
// Redirect:
window.location.href = '/new-page'; // adds to history
window.location.replace('/new-page'); // replaces in history (no back)

// Viewport dimensions
window.innerWidth; // viewport width (includes scrollbar)
window.innerHeight; // viewport height
document.documentElement.clientWidth; // viewport width (excludes scrollbar)

// Scroll
window.scrollTo({ top: 0, behavior: 'smooth' });
window.scrollBy({ top: 100, behavior: 'smooth' });
element.scrollIntoView({ behavior: 'smooth', block: 'center' });

// DOMContentLoaded vs load
document.addEventListener('DOMContentLoaded', () => {
  // DOM is ready – HTML parsed, but images/CSS may still be loading
  initializeApp();
});
window.addEventListener('load', () => {
  // EVERYTHING is loaded – images, stylesheets, iframes, etc.
  hideLoadingScreen();
});
// document.readyState: 'loading' → 'interactive' (DOMContentLoaded) → 'complete' (load)

// getBoundingClientRect – element position relative to viewport
const rect = element.getBoundingClientRect();
// rect.top, rect.left, rect.right, rect.bottom, rect.width, rect.height
const isInViewport = rect.top >= 0 && rect.bottom <= window.innerHeight;
```

19. Forms

Checklist: Form validation · Constraint Validation API · FormData API · input types & attributes · form events (submit, reset, invalid)

```
// FormData API – read form values without manual DOM queries
const form = document.querySelector('#checkout-form');
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const formData = new FormData(form);

  // Convert to object
  const data = Object.fromEntries(formData.entries());
  // { name: 'Sid', email: 'sid@test.com', ... }

  // Send as JSON
  await fetch('/api/checkout', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  });

  // Or send as multipart (for file uploads)
  await fetch('/api/upload', { method: 'POST', body: formData });
});

// Constraint Validation API
const input = document.querySelector('#email');
input.validity.valid;           // is the input valid?
input.validity.typeMismatch;    // wrong type (e.g., invalid email)
input.validity.valueMissing;    // required field is empty
input.validity.tooShort;        // shorter than minlength
input.validity.patternMismatch; // doesn't match pattern attribute

// Custom validation message
input.addEventListener('invalid', () => {
  if (input.validity.typeMismatch) {
    input.setCustomValidity('Please enter a valid email address');
  }
});
input.addEventListener('input', () => input.setCustomValidity(''));

// Programmatic validation
if (!form.checkValidity()) {
  form.reportValidity(); // shows browser validation UI
}
```

20. Semantic HTML

Checklist: header · nav · main · article · section · aside · footer · figure · figcaption · time · mark · progress · details · summary · benefits for SEO & accessibility

```
<!-- Semantic page structure -->
<header>
  <nav aria-label="Main navigation">
    <a href="#main" class="skip-link">Skip to content</a>
    <ul role="list">
      <li><a href="/">Home</a></li>
      <li><a href="/products" aria-current="page">Products</a></li>
    </ul>
  </nav>
</header>

<main id="main">
  <article>
    <h1>Product Review: MacBook Pro 2024</h1>
    <p>Published <time datetime="2024-03-15">March 15, 2024</time></p>

    <section>
      <h2>Performance</h2>
      <p>The M3 chip delivers <mark>40% faster</mark> performance...</p>
      <figure>
        
        <figcaption>Figure 1: Benchmark results</figcaption>
      </figure>
    </section>

    <details>
      <summary>Full Specifications</summary>
      <table>...</table>
    </details>
  </article>

  <aside aria-label="Related articles">
    <h2>Related Reviews</h2>
    <ul>...</ul>
  </aside>
</main>

<footer>
  <p>&copy; 2024 TechReview. All rights reserved.</p>
</footer>
```

Why Semantic HTML matters: (1) **Accessibility:** Screen readers use landmarks (nav, main, article) for navigation. (2) **SEO:** Search engines understand content structure better. (3) **Maintainability:** Code is self-documenting. (4) **Default behaviors:** `<button>` is keyboard-accessible by default, `<div>` is not.

21. HTML5 Features

Checklist: Data attributes · contenteditable · draggable · hidden · input types · required/pattern/min/max · placeholder · autocomplete · output element

```
<!-- Data attributes – store custom data on elements -->
<div class="product-card"
      data-product-id="42"
      data-category="electronics"
      data-price="1299.99">
  ...
</div>

<script>
const card = document.querySelector('.product-card');
card.dataset.productId; // '42' (camelCase conversion)
card.dataset.price;      // '1299.99' (always a string)
card.dataset.inStock = 'true'; // sets data-in-stock="true"
</script>

<!-- HTML5 input types – built-in validation & UI -->
<input type="email" required placeholder="user@example.com">
<input type="tel" pattern="[0-9]{10}" title="10-digit phone number">
<input type="url">
<input type="date" min="2024-01-01" max="2025-12-31">
<input type="number" min="0" max="100" step="5">
<input type="range" min="0" max="100" value="50">
<input type="color" value="#0f6b5f">
<input type="search" list="suggestions">
<datalist id="suggestions">
  <option value="Angular">
  <option value="React">
  <option value="Vue">
</datalist>

<!-- contenteditable – inline editing -->
<div contenteditable="true">Click to edit this text</div>

<!-- output – for calculation results -->
<form oninput="result.value = a.valueAsNumber + b.valueAsNumber">
  <input id="a" type="number"> + <input id="b" type="number">
  = <output name="result" for="a b">0</output>
</form>
```

22. Metadata & SEO

Checklist: meta tags (description, viewport, keywords) · Open Graph · Twitter Cards · canonical URLs · structured data (JSON-LD, Schema.org)

```
<head>
  <!-- Essential meta tags -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Buy premium running shoes online. Free shipping on orders over $50.">
  <title>Running Shoes | SportStore</title>

  <!-- Open Graph (Facebook, LinkedIn) -->
  <meta property="og:title" content="Premium Running Shoes">
  <meta property="og:description" content="Top-rated running shoes for every terrain">
  <meta property="og:image" content="https://example.com/shoes.jpg">
  <meta property="og:url" content="https://example.com/running-shoes">
  <meta property="og:type" content="product">

  <!-- Twitter Cards -->
  <meta name="twitter:card" content="summary_large_image">
  <meta name="twitter:title" content="Premium Running Shoes">

  <!-- Canonical URL – prevent duplicate content -->
  <link rel="canonical" href="https://example.com/running-shoes">

  <!-- Structured Data (JSON-LD) – rich search results -->
  <script type="application/ld+json">
  {
    "@context": "https://schema.org",
    "@type": "Product",
    "name": "Nike Air Max 2024",
    "image": "https://example.com/shoes.jpg",
    "description": "Premium running shoes",
    "offers": {
      "@type": "Offer",
      "price": "8999",
      "priceCurrency": "INR",
      "availability": "https://schema.org/InStock"
    },
    "aggregateRating": {
      "@type": "AggregateRating",
      "ratingValue": "4.5",
      "reviewCount": "128"
    }
  }
  </script>
</head>
```

23. Media Elements

Checklist: video · audio · source · track (captions) · picture · srcset & sizes

```
<!-- Responsive images with picture + srcset -->
<picture>
  <source media="(min-width: 1200px)" srcset="hero-1200.avif" type="image/avif">
  <source media="(min-width: 1200px)" srcset="hero-1200.webp" type="image/webp">
  <source media="(min-width: 600px)" srcset="hero-600.webp" type="image/webp">
  
</picture>

<!-- Video with multiple sources + captions -->
<video controls preload="metadata" poster="thumb.jpg"
  width="640" height="360">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  <track kind="subtitles" src="captions-en.vtt" srclang="en" label="English" default>
  <track kind="subtitles" src="captions-hi.vtt" srclang="hi" label="Hindi">
  Your browser does not support the video element.
</video>

<!-- srcset + sizes – let browser choose optimal image -->

```

24. HTML Accessibility

Checklist: ARIA roles · ARIA attributes (aria-label, labelledby, describedby) · aria-live regions · aria-hidden · landmark roles · alt text · label associations · tabindex · skip links

```
<!-- Skip link – lets keyboard users jump to main content -->
<a href="#main-content" class="skip-link">Skip to main content</a>
<style>
.skip-link {
  position: absolute; left: -9999px;
  &:focus { left: 10px; top: 10px; z-index: 10000; }
}
</style>

<!-- Accessible form -->
<form>
  <fieldset>
    <legend>Shipping Address</legend>

    <label for="street">Street Address <span aria-hidden="true">*</span></label>
    <input id="street" type="text" required aria-required="true"
      aria-describedby="street-help">
    <small id="street-help">Include apartment/suite number</small>

    <label for="city">City</label>
    <input id="city" type="text" required aria-required="true"
      aria-invalid="false">
    </fieldset>
  </form>

<!-- aria-live – announces dynamic content to screen readers -->
<div aria-live="polite" aria-atomic="true" id="status">
  <!-- When JS updates this content, screen reader announces it -->
</div>
<script>
document.getElementById('status').textContent = '3 results found';
// Screen reader: "3 results found"
</script>

<!-- Accessible custom button (prefer native <button>) -->
<div role="button" tabindex="0"
  aria-pressed="false"
  onkeydown="if(event.key==='Enter' || event.key===' ')this.click()">
  Toggle Dark Mode
</div>

<!-- aria-hidden – hide decorative elements from screen readers -->
<button>
  <span aria-hidden="true">■■</span>
  <span class="sr-only">Delete item</span>
</button>

<!-- tabindex values -->
<!-- tabindex="0": adds to natural tab order -->
<!-- tabindex="-1": focusable via JS only, removed from tab order -->
<!-- tabindex="1+": AVOID – creates confusing tab order -->
```

■ Interview Questions — HTML & Accessibility

Q: What is the difference between aria-label, aria-labelledby, and aria-describedby?

A: aria-label provides an invisible label string directly. aria-labelledby references another element's ID whose text content becomes the label (for visible labels). aria-describedby adds supplementary description (help text, error messages) — read after the label. Priority: aria-labelledby > aria-label > native <label>.

Q: Why is `<div onclick>` bad for accessibility?

A: It's not keyboard-accessible (no focus, no Enter/Space handling), has no ARIA role, and screen readers don't announce it as interactive. Use `<button>` instead — it gets focus, keyboard support, and semantics for free.

25. CSS Selectors

Checklist: Type/class/ID selectors · attribute selectors · pseudo-classes (:hover, :focus, :nth-child, :nth-of-type, :is, :where, :has, :not) · pseudo-elements (::before, ::after, ::first-letter, ::first-line) · combinators · specificity calculation

Specificity Calculation

Category	Weight	Examples
Inline styles	1,0,0,0	style="..."
ID selectors	0,1,0,0	#header
Classes, attributes, pseudo-classes	0,0,1,0	.card, [type="text"], :hover
Elements, pseudo-elements	0,0,0,1	div, ::before
:where()	0,0,0,0	Always zero — great for base styles
!important	Overrides everything	Avoid in application code

```
/* :has() – the "parent selector" (game-changer) */
.card:has(img) { grid-template-rows: 200px auto; }
.form-group:has(:invalid) { border-color: red; }
a:has(> img) { border: none; } /* link containing image */

/* :is() – grouping without repeating */
:is(h1, h2, h3, h4):hover { color: var(--accent); }
/* Specificity = highest argument: :is(#id, .class) has ID specificity */

/* :where() – like :is() but ZERO specificity */
:where(.btn) { padding: 8px 16px; } /* easily overridable */

/* :not() */
.list-item:not(:last-child) { border-bottom: 1px solid #ddd; }
input:not([type="submit"]):not([type="button"]) { border: 1px solid #ccc; }

/* :nth-child patterns */
li:nth-child(odd) { background: #f5f5f5; }
li:nth-child(3n) { font-weight: bold; } /* every 3rd */
li:nth-child(n+4) { color: gray; } /* 4th and beyond */
li:nth-last-child(-n+3) { color: red; } /* last 3 items */

/* Attribute selectors */
a[href^="https"] { color: green; } /* starts with */
a[href$=".pdf"] { color: red; } /* ends with */
a[href*="example"] { color: blue; } /* contains */
input[type="text" i] { } /* case-insensitive */

/* Combinators */
div p { } /* descendant – any depth */
div > p { } /* child – direct children only */
h2 + p { } /* adjacent sibling – immediately after */
h2 ~ p { } /* general sibling – all after */
```

■ Interview Questions — CSS Selectors

Q: Calculate the specificity: `div#header .nav li.active a:hover`

A: ID(#header)=1, classes(.nav, .active, :hover)=3, elements(div, li, a)=3 → 0, 1, 3, 3.

26. CSS Box Model

Checklist: Content/padding/border/margin · box-sizing · margin collapse · negative margins

```
/* box-sizing – the single most important CSS reset */
*, ::before, ::after {
  box-sizing: border-box; /* width includes padding + border */
}

/* Without border-box:
   width: 300px + padding: 20px×2 + border: 1px×2 = 342px actual width
   With border-box:
   width: 300px total, content shrinks to accommodate padding + border */

/* Margin collapse – vertical margins of adjacent blocks collapse */
/* The LARGER margin wins, they don't add up */
.box1 { margin-bottom: 30px; }
.box2 { margin-top: 20px; }
/* Gap between them: 30px (not 50px!) */

/* Margin collapse does NOT happen with: */
/* ✗ flexbox/grid items */
/* ✗ floated elements */
/* ✗ absolute/fixed positioned elements */
/* ✗ elements with overflow other than visible */
/* ✗ inline-block elements */

/* Negative margins – pulling elements */
.overlap { margin-top: -20px; } /* pulls element UP */
.bleed { margin-left: -16px; margin-right: -16px; } /* full-width in padded parent */
```

27. CSS Layout

Checklist: display (block/inline/inline-block/none) · position (static/relative/absolute/fixed/sticky) · float & clear · vertical alignment · z-index & stacking context

```
/* Display values */
.block { display: block; }           /* full width, stacks vertically */
.inline { display: inline; }         /* flows with text, no width/height */
.inline-block { display: inline-block; } /* inline + can set width/height */
.none { display: none; }             /* removed from layout entirely */
.contents { display: contents; }     /* box disappears, children remain */

/* Position */
.relative { position: relative; top: 10px; } /* offset from normal position, keeps space */
.absolute { position: absolute; top: 0; right: 0; } /* relative to nearest positioned ancestor */
.fixed { position: fixed; top: 0; } /* relative to viewport, stays on scroll */
.sticky { position: sticky; top: 0; } /* relative → fixed when scrolled past offset */

/* Stacking context – z-index only works within a context */
/* A new stacking context is created by: */
/* position: relative/absolute/fixed + z-index */
/* opacity < 1, transform, filter, will-change */
/* isolation: isolate */
.parent { position: relative; z-index: 1; } /* creates stacking context */
.child { position: absolute; z-index: 999; } /* only counts within parent */

/* Center anything – modern approaches */
.center-grid { display: grid; place-items: center; }
.center-flex { display: flex; justify-content: center; align-items: center; }
.center-abs {
  position: absolute; top: 50%; left: 50%;
  transform: translate(-50%, -50%);
}
.center-margin { margin-inline: auto; } /* horizontal centering for block elements */
```


28. CSS Flexbox

Checklist: flex-direction · flex-wrap · justify-content · align-items · align-content · flex-grow/shrink/basis · flex shorthand · align-self · order · gap

```
/* Flexbox cheat sheet */
.container {
  display: flex;
  flex-direction: row;          /* row | row-reverse | column | column-reverse */
  flex-wrap: wrap;              /* nowrap | wrap | wrap-reverse */
  justify-content: space-between; /* flex-start|end | center | space-between|around|evenly */
  align-items: center;          /* stretch | flex-start|end | center | baseline */
  gap: 16px;                    /* row-gap column-gap */
}

.item {
  flex: 1 0 200px; /* grow:1 shrink:0 basis:200px */
  /* flex-grow: how much extra space to take (0 = don't grow) */
  /* flex-shrink: how much to shrink when space is tight (0 = don't shrink) */
  /* flex-basis: initial size before growing/shrinking (like width) */
  align-self: flex-end; /* override align-items for this item */
  order: -1;            /* move this item first (default order is 0) */
}

/* Common patterns */
/* Navbar: logo left, nav right */
.navbar { display: flex; justify-content: space-between; align-items: center; }

/* Card with footer pushed to bottom */
.card { display: flex; flex-direction: column; height: 100%; }
.card-body { flex: 1; } /* takes remaining space */
.card-footer { margin-top: auto; } /* pushed to bottom */

/* Equal-width columns */
.cols { display: flex; gap: 16px; }
.cols > * { flex: 1; } /* each child gets equal space */

/* Wrap with minimum width */
.grid-flex { display: flex; flex-wrap: wrap; gap: 16px; }
.grid-flex > * { flex: 1 1 300px; } /* min 300px, grows to fill */
```

29. CSS Grid

Checklist: grid-template-columns/rows · grid-template-areas · fr · repeat() · minmax() · auto-fill/fit · grid gap · grid item placement · implicit vs explicit grid · named grid lines · align/justify/place-items

```
/* Responsive grid – no media queries needed */
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 24px;
}
/* auto-fill: creates empty columns if space allows */
/* auto-fit: collapses empty columns, stretches existing items */

/* Dashboard layout with named areas */
.dashboard {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar main aside"
    "footer footer footer";
  grid-template-columns: 250px 1fr 200px;
  grid-template-rows: 60px 1fr 40px;
  min-height: 100vh;
  gap: 0;
}
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.aside { grid-area: aside; }
.footer { grid-area: footer; }

/* Explicit item placement */
.hero {
  grid-column: 1 / -1; /* span full width */
  grid-row: 1 / 3; /* span 2 rows */
}
.featured {
  grid-column: span 2; /* span 2 columns */
}

/* Alignment */
.grid {
  justify-items: center; /* horizontal alignment of items within cells */
  align-items: center; /* vertical alignment of items within cells */
  place-items: center; /* shorthand for both */
  justify-content: space-between; /* horizontal alignment of grid tracks */
  align-content: center; /* vertical alignment of grid tracks */
}

/* Subgrid (children inherit parent grid lines) */
.parent { display: grid; grid-template-columns: repeat(3, 1fr); }
.child { display: grid; grid-template-columns: subgrid; grid-column: span 3; }
```

■ Interview Questions — Flexbox & Grid

Q: When would you choose Flexbox over Grid?

A: Flexbox for one-dimensional layouts (row OR column): navbars, card footers, centering items, flexible wrapping.
Grid for two-dimensional layouts (rows AND columns): page layouts, dashboards, galleries. They can be combined — Grid for the page layout, Flexbox inside each grid cell.

30. Responsive Design

Checklist: Media queries · mobile-first vs desktop-first · breakpoints · fluid typography · container queries · clamp() · min/max · aspect-ratio

```
/* Mobile-first approach – start with mobile, enhance for larger */
.card { padding: 12px; font-size: 14px; }

@media (min-width: 768px) {
  .card { padding: 20px; font-size: 16px; display: flex; }
}
@media (min-width: 1200px) {
  .card { padding: 32px; max-width: 1200px; margin: 0 auto; }
}

/* Fluid typography – no breakpoints needed */
h1 { font-size: clamp(1.5rem, 4vw + 0.5rem, 3.5rem); }
p { font-size: clamp(0.9rem, 1.5vw + 0.5rem, 1.15rem); }
/* clamp(minimum, preferred, maximum) */

/* Container queries – responsive to PARENT, not viewport */
.card-wrapper { container-type: inline-size; container-name: card; }

@container card (min-width: 400px) {
  .card { display: flex; gap: 16px; }
  .card-image { width: 40%; }
}
@container card (max-width: 399px) {
  .card-image { width: 100%; }
}

/* Aspect ratio */
.video-container { aspect-ratio: 16 / 9; width: 100%; }
.avatar { aspect-ratio: 1; border-radius: 50%; object-fit: cover; }

/* min() and max() */
.container { width: min(90vw, 1200px); } /* whichever is smaller */
.sidebar { width: max(200px, 20vw); } /* whichever is larger */
```

31. CSS Typography

Checklist: font-family/size/weight · line-height · letter/word-spacing · text-align/decoration/transform · @font-face · font-display · WOFF/WOFF2 · system font stacks · variable fonts

```
/* System font stack – fastest loading (no downloads) */
body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
               'Helvetica Neue', Arial, sans-serif;
  line-height: 1.5;           /* 1.5× font size – good readability */
  -webkit-font-smoothing: antialiased;
}

/* @font-face with font-display strategy */
@font-face {
  font-family: 'Inter';
  src: url('/fonts/inter-var.woff2') format('woff2');
  font-display: swap;         /* show fallback → swap when loaded */
  font-weight: 100 900;       /* variable font: all weights in one file */
  unicode-range: U+0000-00FF; /* subset: only latin characters */
}

/* font-display values:
  auto:      browser decides
  swap:      show fallback immediately, swap when ready (FOUT)
  block:     invisible text for short period, then show (FOIT)
  fallback:  very short block, then fallback (swap if loads quickly)
  optional:  browser may not swap at all (best for slow connections) */

/* Variable fonts – single file, infinite weights/widths */
.heading {
  font-family: 'Inter', sans-serif;
  font-weight: 750;           /* any value 100-900, not just 400/700 */
  font-variation-settings: 'wght' 750, 'slnt' -5;
}
```

32. Colors & Backgrounds

Checklist: hex/rgb/rgba/hsl/hsla · `currentColor` · gradients (linear/radial/conic) · background-size/position/attachment · multiple backgrounds · opacity vs rgba

```
/* Color formats */
.box {
  color: #0f6b5f;           /* hex */
  color: rgb(15, 107, 95);  /* rgb */
  color: rgba(15, 107, 95, 0.8); /* rgb + alpha */
  color: hsl(170, 75%, 24%); /* hue 0-360, saturation %, lightness % */
  color: hsla(170, 75%, 24%, 0.8); /* hsl + alpha */
  color: oklch(50% 0.15 170); /* modern: perceptually uniform */
}

/* currentColor – inherits the text color */
.btn {
  color: #0f6b5f;
  border: 2px solid currentColor; /* border matches text color */
  box-shadow: 0 2px 0 currentColor; /* shadow matches text color */
}
.btn:hover { color: #0a4f45; } /* everything updates together! */

/* Gradients */
.hero { background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); }
.radial { background: radial-gradient(circle at top right, #fff, #eee); }
.conic { background: conic-gradient(red, yellow, green, blue, red); }

/* Multiple backgrounds – layered (first on top) */
.card {
  background:
    linear-gradient(rgba(0,0,0,0.3), rgba(0,0,0,0.7)),
    url('bg.jpg') center/cover no-repeat;
}

/* opacity vs rgba */
.overlay { opacity: 0.5; } /* affects ENTIRE element + children */
.overlay-bg { background: rgba(0, 0, 0, 0.5); } /* only background is transparent */
```

33. Transforms & Animations

Checklist: transform (translate/rotate/scale/skew) · transform-origin · 3D transforms · transitions · @keyframes · animation properties · will-change · GPU acceleration

```
/* Performant animations – ONLY transform + opacity trigger GPU layer */
.card {
  transition: transform 0.2s ease-out, box-shadow 0.2s ease-out;
}
.card:hover {
  transform: translateY(-4px) scale(1.01);
  box-shadow: 0 12px 24px rgba(0,0,0,0.12);
}

/* Transition shorthand */
.btn {
  transition: all 0.2s ease;
  /* property | duration | timing-function | delay */
  /* timing: ease | linear | ease-in | ease-out | ease-in-out | cubic-bezier() */
}

/* @keyframes – complex multi-step animation */
@keyframes slideInUp {
  from {
    transform: translateY(30px);
    opacity: 0;
  }
  to {
    transform: translateY(0);
    opacity: 1;
  }
}
.animate-in {
  animation: slideInUp 0.4s ease-out forwards;
  /* name | duration | timing | fill-mode */
  /* fill-mode: forwards = keeps final state */
}

/* Loading spinner */
@keyframes spin {
  from { transform: rotate(0deg); }
  to { transform: rotate(360deg); }
}
.spinner {
  width: 24px; height: 24px;
  border: 3px solid #e0e0e0;
  border-top-color: var(--accent);
  border-radius: 50%;
  animation: spin 0.6s linear infinite;
}

/* GPU acceleration – will-change */
.animated-element {
  will-change: transform, opacity; /* hint: these will animate soon */
  /* WARNING: don't overuse – each element with will-change creates a GPU layer */
}

/* 3D transforms */
.flip-card {
  perspective: 1000px;
}
.flip-card-inner {
  transition: transform 0.6s;
  transform-style: preserve-3d;
}
.flip-card:hover .flip-card-inner {
```

```
    transform: rotateY(180deg);  
}
```

34. CSS Variables

Checklist: Custom properties declaration · var() function · scope & inheritance · fallback values · runtime manipulation with JavaScript

```
/* Declaration – scoped to the element and its descendants */
:root {
  --color-primary: #0f6b5f;
  --color-bg: #fffd8;
  --radius: 8px;
  --spacing-sm: 8px;
  --spacing-md: 16px;
  --spacing-lg: 24px;
  --font-sans: 'Inter', system-ui, sans-serif;
}

/* Usage with fallback */
.btn {
  background: var(--color-primary);
  border-radius: var(--radius);
  padding: var(--spacing-sm) var(--spacing-md);
  font-family: var(--font-heading, var(--font-sans)); /* nested fallback */
}

/* Scoped overrides – theming */
[data-theme="dark"] {
  --color-primary: #8bf0cc;
  --color-bg: #1a1a2e;
}

/* All components using these variables automatically update! */

/* Component-level variables */
.card {
  --card-padding: 16px;
  padding: var(--card-padding);
}
.card.compact { --card-padding: 8px; } /* override for variant */

/* JS manipulation – dynamic theming */
document.documentElement.style.setProperty('--color-primary', '#e74c3c');
const value = getComputedStyle(document.documentElement)
  .getPropertyValue('--color-primary').trim();
```


35. Modern CSS

Checklist: Subgrid · aspect-ratio · gap (flexbox/grid) · logical properties · scroll-snap · backdrop-filter · mix-blend-mode · filter · clip-path · shape-outside

```
/* Logical properties – work with any writing direction (LTR/RTL) */
.card {
  margin-inline: auto;          /* replaces margin-left + margin-right */
  padding-block: 16px;         /* replaces padding-top + padding-bottom */
  border-inline-start: 4px solid var(--accent); /* left in LTR, right in RTL */
}

/* Scroll snap – carousel without JavaScript */
.carousel {
  display: flex;
  overflow-x: auto;
  scroll-snap-type: x mandatory; /* snap required after scrolling */
  gap: 16px;
  scrollbar-width: none;
}
.carousel-item {
  scroll-snap-align: start;      /* snap to start of each item */
  flex: 0 0 80%;
}

/* backdrop-filter – frosted glass effect */
.modal-overlay {
  backdrop-filter: blur(12px) saturate(180%);
  background: rgba(0, 0, 0, 0.25);
}

/* clip-path – non-rectangular shapes */
.hero-section {
  clip-path: polygon(0 0, 100% 0, 100% 85%, 0 100%);
}
.circle-avatar {
  clip-path: circle(50%);
}

/* filter effects */
.grayscale { filter: grayscale(100%); }
.blur { filter: blur(4px); }
.image:hover { filter: brightness(1.1) contrast(1.05); }

/* :has() – style parent based on child state */
.form-group:has(input:focus) {
  border-color: var(--accent);
  box-shadow: 0 0 0 3px var(--accent-soft);
}
```

36. CSS Methodologies

Checklist: BEM · SMACSS · OOCSS · Atomic CSS · CSS Modules · utility-first CSS

```
/* BEM — Block__Element--Modifier */
.product-card {}                                /* Block */
.product-card__title {}                        /* Element */
.product-card__title--highlighted {}          /* Modifier */
.product-card__image {}                       /* Element */
.product-card--featured {}                    /* Block modifier */

/* SCSS with BEM */
.product-card {
  border: 1px solid #ddd;

  &__title {
    font-size: 18px;
    &--highlighted { color: red; }
  }
  &__image { width: 100%; border-radius: 8px; }
  &--featured { border-color: gold; }
}

/* Utility-first (Tailwind-style) */
/* <div class="flex items-center gap-4 p-4 rounded-lg bg-white shadow-md"> */

/* CSS Modules — local scope (generated class names) */
/* styles.module.css */
.card { padding: 16px; } /* → .card_a1b2c3 in output */
/* import styles from './styles.module.css'; */
/* <div className={styles.card}> */
```

■ Interview — CSS Methodologies

Q: What is BEM and why use it?

A: BEM (Block-Element-Modifier) is a naming convention: `.block__element--modifier`. It makes CSS predictable, reusable, and avoids specificity wars. Each component (block) is self-contained, elements are parts of the block, modifiers are variations. Flat specificity (one class per rule) makes overrides predictable.

37. CSS-in-JS

Checklist: Styled Components · Emotion · CSS Modules · inline styles · trade-offs & performance

```
// Styled Components (React ecosystem – understand for comparisons)
const Button = styled.button`
  background: ${props => props.primary ? '#0f6b5f' : 'white'};
  color: ${props => props.primary ? 'white' : '#0f6b5f'};
  padding: 8px 16px;
  border-radius: 6px;
  &:hover { opacity: 0.9; }
`;

// Angular equivalent: component styles with ViewEncapsulation
@Component({
  styles: [
    :host { display: block; }
    .btn { background: var(--color-primary); }
    .btn--primary { background: var(--accent); color: white; }
  ],
  encapsulation: ViewEncapsulation.Emulated // default – scoped styles
})
```

Approach	Pros	Cons
CSS-in-JS	Scoped, dynamic theming, co-located	Runtime cost, bundle size, framework-specific
CSS Modules	Scoped, no runtime, works with any framework	No dynamic styles without classes
Utility CSS (Tailwind)	No naming, small bundle (purging), fast	Verbose HTML, learning curve
BEM + SCSS	No tooling needed, clear structure	Naming discipline required, global scope

38. CSS Preprocessors

Checklist: SASS/SCSS (variables, nesting, mixins, extends) · Less · PostCSS · Autoprefixer

```
// SCSS – most widely used preprocessor
// Variables (legacy – prefer CSS custom properties)
$primary: #0f6b5f;
$spacing: 8px;

// Nesting
.card {
  padding: $spacing * 2;

  &:hover { box-shadow: 0 4px 12px rgba(0,0,0,0.1); }

  &__title { font-weight: bold; }

  @media (min-width: 768px) {
    display: flex;
  }
}

// Mixins – reusable style blocks
@mixin respond-to($breakpoint) {
  @if $breakpoint == 'tablet' { @media (min-width: 768px) { @content; } }
  @if $breakpoint == 'desktop' { @media (min-width: 1200px) { @content; } }
}
.card {
  padding: 12px;
  @include respond-to('tablet') { padding: 24px; }
}

// Extend – share styles (use sparingly)
%flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}
.modal { @extend %flex-center; }
.popup { @extend %flex-center; }

// PostCSS – CSS transformations via plugins
// Popular plugins: autoprefixer, postcss-preset-env, cssnano
// Angular CLI uses PostCSS under the hood
```

39. CSS Performance

Checklist: Critical CSS · CSS containment · content-visibility · reducing reflows & repaints · will-change · layer separation

```
/* Critical CSS – inline above-fold styles for faster FCP */
<head>
  <style>
    /* ONLY styles needed for first viewport render */
    .header, .hero, .nav { ... }
  </style>
  <!-- Load full CSS asynchronously -->
  <link rel="stylesheet" href="full.css" media="print" onload="this.media='all'">
  <noscript><link rel="stylesheet" href="full.css"></noscript>
</head>

/* CSS containment – isolate subtree from rest of page */
.widget {
  contain: layout style paint; /* browser can optimize rendering of this subtree */
  /* layout: size doesn't affect parent */
  /* style: counters/quotes don't leak */
  /* paint: content doesn't render outside bounds */
}

/* content-visibility – skip rendering of off-screen content */
.section {
  content-visibility: auto; /* render only when in/near viewport */
  contain-intrinsic-size: 0 500px; /* estimated height for scrollbar */
}
/* Can save massive rendering time for long pages! */

/* Reducing reflows (layout recalculations) */
/* ■ Causes reflow: changing width/height, padding, margin, font-size,
   adding/removing DOM nodes, reading offsetHeight/offsetWidth */
/* ■ Cheap: transform, opacity, visibility, color, background-color */

/* Layer separation – promote animated elements to GPU layer */
.animated {
  will-change: transform; /* creates separate composite layer */
  transform: translateZ(0); /* force GPU layer (old hack) */
}

/* will-change best practices */
/* ■ Add before animation starts, remove after */
element.addEventListener('mouseenter', () => {
  element.style.willChange = 'transform';
});
element.addEventListener('animationend', () => {
  element.style.willChange = 'auto';
});
/* ■ Don't set will-change on everything – each layer costs GPU memory */
```

■ Interview Questions — CSS Performance

Q: What triggers reflow vs repaint?

A: Reflow (expensive): changes to geometry — width, height, padding, margin, font-size, adding/removing elements. **Repaint** (cheaper): changes to appearance — color, background, visibility, box-shadow. **Neither** (cheapest): transform, opacity — these run on the compositor thread (GPU).

Q: What is content-visibility: auto and when would you use it?

A: It tells the browser to skip rendering elements that are off-screen. The browser doesn't compute layout, paint, or style for those elements until they're near the viewport. Use on long pages with many sections — can reduce initial render time by 50%+ on content-heavy pages.

40. TypeScript Basics

Checklist: Type annotations · inference · any/unknown/never/void · arrays/tuples · interfaces · type vs interface · optional/readonly

```
// Type annotations vs inference
let name: string = 'Sid';    // explicit annotation
let age = 28;                // inferred as number (no annotation needed)

// any vs unknown vs never
let risky: any = 42;          // disables type checking – avoid!
let safe: unknown = 42;       // must narrow before use
if (typeof safe === 'number') { safe.toFixed(2); } // OK after narrowing

function throwError(msg: string): never {
  throw new Error(msg);      // never returns – function always throws
}

// Tuples – fixed-length, typed arrays
type UserTuple = [string, number, boolean]; // [name, age, active]
const user: UserTuple = ['Sid', 28, true];
user[0].toUpperCase(); // TS knows index 0 is string

// Interface vs Type
interface User {
  readonly id: number;
  name: string;
  email?: string;    // optional
}
type UserOrAdmin = User | Admin; // unions only with type
interface AdminUser extends User { permissions: string[]; } // extends with interface
// Rule of thumb: use interface for object shapes, type for unions/intersections
```

41. TypeScript Advanced Types

Checklist: Union/intersection · literal types · discriminated unions · type guards (typeof/instanceof/user-defined) · type assertions · type narrowing

```
// Discriminated unions – type-safe state machines
type RequestState<T> =
  | { status: 'idle' }
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; error: string };

function handleState<T>(state: RequestState<T>): string {
  switch (state.status) {
    case 'idle': return 'Ready';
    case 'loading': return 'Loading...';
    case 'success': return `Got: ${JSON.stringify(state.data)}`; // TS knows .data exists
    case 'error': return `Error: ${state.error}`; // TS knows .error exists
  } // exhaustive – TS ensures all cases are handled
}

// User-defined type guard
interface Cat { meow(): void; }
interface Dog { bark(): void; }
function isCat(pet: Cat | Dog): pet is Cat {
  return 'meow' in pet; // narrows type within if block
}
const pet: Cat | Dog = getPet();
if (isCat(pet)) { pet.meow(); } // TS knows it's Cat
else { pet.bark(); } // TS knows it's Dog

// Intersection types – combine types
type Timestamped = { createdAt: Date; updatedAt: Date };
type SoftDeletable = { deletedAt: Date | null };
type AuditableUser = User & Timestamped & SoftDeletable;

// Literal types
type Direction = 'up' | 'down' | 'left' | 'right';
type HttpStatus = 200 | 201 | 400 | 401 | 403 | 404 | 500;

// Template literal types
type HttpMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';
type ApiRoute = `/${string}`;
type Endpoint = `${HttpMethod} ${ApiRoute}`;
// Valid: 'GET /users', 'POST /orders'
```

42. TypeScript Generics

Checklist: Generic functions · interfaces · classes · constraints · default parameters · utility types

```
// Generic function – works with any type while maintaining type safety
function first<T>(arr: T[]): T | undefined {
    return arr[0];
}
first([1, 2, 3]);           // returns number
first(['a', 'b']);          // returns string
// TS infers the type from the argument!

// Generic with constraint
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}
const user = { name: 'Sid', age: 28 };
getProperty(user, 'name'); // string
// getProperty(user, 'invalid'); // ■ Compile error!

// Generic interface
interface ApiResponse<T> {
    data: T;
    status: number;
    message: string;
    timestamp: Date;
}
type UserResponse = ApiResponse<User>;
type OrderListResponse = ApiResponse<Order[]>;

// Generic class
class TypedStorage<T> {
    private items = new Map<string, T>();
    set(key: string, value: T): void { this.items.set(key, value); }
    get(key: string): T | undefined { return this.items.get(key); }
}
const userStore = new TypedStorage<User>();
userStore.set('u1', { id: 1, name: 'Sid' });
const u = userStore.get('u1'); // User | undefined

// Default generic parameter
interface PaginatedResponse<T, M = Record<string, unknown>> {
    data: T[];
    meta: M;
    total: number;
    page: number;
}
```


43. TypeScript Utility Types

Checklist: Partial/Required/Readonly · Pick/Omit · Record · Exclude/Extract · NonNullable · ReturnType/Parameters · Awaited · mapped types · conditional types

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
  role: 'admin' | 'user';
}

// Built-in utility types – CRITICAL for interviews
type CreateUser = Omit<User, 'id'>; // no id for creation
type UpdateUser = Partial<Omit<User, 'id'>>; // all fields optional
type PublicUser = Pick<User, 'id' | 'name' | 'email'>; // no password
type ReadonlyUser = Readonly<User>; // all readonly
type RequiredUser = Required<User>; // all required
type UserRecord = Record<string, User>; // dictionary

// Exclude/Extract – filter union types
type StringOrNumber = string | number | boolean;
type OnlyStrNum = Exclude<StringOrNumber, boolean>; // string | number
type OnlyBool = Extract<StringOrNumber, boolean>; // boolean

// ReturnType & Parameters
function createUser(name: string, email: string): User { /* ... */ }
type CreateReturn = ReturnType<typeof createUser>; // User
type CreateParams = Parameters<typeof createUser>; // [string, string]

// Awaited – unwrap Promise type
type UserPromise = Promise<User>;
type ResolvedUser = Awaited<UserPromise>; // User

// Custom mapped type – make all properties nullable
type Nullable<T> = { [K in keyof T]: T[K] | null };
type NullableUser = Nullable<User>; // { id: number | null; name: string | null; ... }

// Conditional type
type IsArray<T> = T extends Array<any> ? true : false;
type Test1 = IsArray<string[]>; // true
type Test2 = IsArray<string>; // false

// Deep Partial – recursive
type DeepPartial<T> = {
  [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K];
};
```

44. TypeScript Functions

Checklist: Function type expressions · call signatures · overload signatures · rest params with types · optional params · return types

```
// Function type expression
type MathFn = (a: number, b: number) => number;
const add: MathFn = (a, b) => a + b;

// Call signatures (in interfaces)
interface Formatter {
  (input: string): string;
  locale: string; // function with properties
}

// Overload signatures – different return types based on input
function parse(input: string): string[];
function parse(input: string, limit: number): string[];
function parse(input: number): number;
function parse(input: string | number, limit?: number): string[] | number {
  if (typeof input === 'string') return input.split(',', limit);
  return input;
}
const result = parse('a,b,c'); // TS infers string[]
const result2 = parse(42);     // TS infers number

// Rest parameters
function merge<T>(...objects: Partial<T>[]): T {
  return Object.assign({}, ...objects) as T;
}
```

45. TypeScript Enums

Checklist: Numeric enums · string enums · const enums · heterogeneous enums

```
// String enum – most recommended
enum OrderStatus {
    Pending = 'PENDING',
    Processing = 'PROCESSING',
    Shipped = 'SHIPPED',
    Delivered = 'DELIVERED',
    Cancelled = 'CANCELLED'
}
function getStatusColor(status: OrderStatus): string {
    switch (status) {
        case OrderStatus.Pending: return 'yellow';
        case OrderStatus.Shipped: return 'blue';
        case OrderStatus.Delivered: return 'green';
        case OrderStatus.Cancelled: return 'red';
        default: return 'gray';
    }
}

// const enum – inlined at compile time (no runtime object!)
const enum Direction { Up = 0, Down = 1, Left = 2, Right = 3 }
const move = Direction.Up; // compiled to: const move = 0;
// No Direction object exists at runtime = smaller bundle

// Modern alternative: union of literals (recommended over enum)
type Status = 'pending' | 'active' | 'closed';
// Advantages over enum: no runtime code, simpler, tree-shakeable
```

46. TypeScript Modules & Configuration

Checklist: ES6 module syntax · declaration files (.d.ts) · ambient declarations · triple-slash directives · tsconfig.json · strict mode · target/lib · module resolution · path mapping

```
// Declaration files – type definitions for JS libraries
// types.d.ts
declare module 'legacy-library' {
  export function doSomething(input: string): number;
  export interface Config { timeout: number; }
}

// Ambient declarations – global types
declare global {
  interface Window {
    analytics: { track: (event: string, data: object) => void };
  }
}

// tsconfig.json – key options explained
{
  "compilerOptions": {
    "target": "ES2022",           // output JS version
    "lib": ["ES2022", "DOM"],    // available APIs
    "module": "ESNext",         // module system
    "moduleResolution": "bundler", // how imports are resolved
    "strict": true,              // enables ALL strict checks:
    // strictNullChecks, strictFunctionTypes, strictBindCallApply,
    // strictPropertyInitialization, noImplicitAny, noImplicitThis
    "esModuleInterop": true,     // import default from CommonJS
    "skipLibCheck": true,       // skip checking .d.ts files (faster)
    "paths": {                  // path aliases
      "@shared/*": ["src/shared/*"],
      "@models/*": ["src/models/*"]
    },
    "baseUrl": "./",
    "declaration": true,        // generate .d.ts files
    "sourceMap": true           // generate source maps
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "dist"]
}
```

■ Interview Questions — TypeScript

Q: What's the difference between any and unknown?

A: any disables type checking entirely — you can do anything with it (unsafe). unknown is the type-safe version — you must narrow the type before using it (with typeof, instanceof, or type guards). Use unknown for values of uncertain type (e.g., API responses, user input).

Q: When would you use type vs interface?

A: interface: object shapes, class contracts, declaration merging (extensions). **type:** unions, intersections, mapped types, conditional types, primitives. If it's an object shape that might be extended, use interface. If it involves unions or complex type manipulation, use type.

Q: Implement a type that makes certain keys required and the rest optional.

```
type RequireKeys<T, K extends keyof T> = Required<Pick<T, K>> & Partial<Omit<T, K>>;
type CreateOrder = RequireKeys<Order, 'productId' | 'quantity'>;
// productId and quantity are required, everything else is optional
```

47. Loading Performance

Checklist: Code splitting · dynamic imports · tree shaking · bundle analysis · minification/compression · lazy loading images · preload/prefetch/preconnect · resource hints · HTTP/2-3 · CDN

```
<!-- Resource hints – tell the browser what's coming -->
<link rel="preconnect" href="https://api.example.com">           <!-- establish connection early -->
<link rel="dns-prefetch" href="https://cdn.example.com">        <!-- resolve DNS early -->
<link rel="preload" href="/fonts/inter.woff2" as="font" type="font/woff2" crossorigin> <!-- load
<link rel="prefetch" href="/js/checkout.chunk.js">              <!-- speculative next page -->
<link rel="modulepreload" href="/js/utils.mjs">                 <!-- preload ES module -->

// Code splitting with dynamic imports
const AdminModule = lazy(() => import('./admin/admin.module'));
// Angular: loadChildren in routes
{ path: 'admin', loadChildren: () => import('./admin/admin.routes') }

// Tree shaking – works with ES modules (static imports)
import { formatDate } from './utils'; // only formatDate is bundled
// ■ import * as Utils from './utils'; // may prevent tree shaking

// Bundle analysis commands
// Angular: ng build --stats-json → webpack-bundle-analyzer
// Vite: npx vite-bundle-visualizer
```

48. Runtime Performance

Checklist: Debounce/throttle · requestAnimationFrame · avoiding layout thrashing · read-write batching · virtual scrolling · infinite scroll optimization · memoization · Web Workers · OffscreenCanvas

```
// Layout thrashing – BAD: read-write-read-write forces multiple reflows
elements.forEach(el => {
  const height = el.offsetHeight;          // READ → forces layout
  el.style.height = height * 2 + 'px';    // WRITE → invalidates layout
  // Next iteration: READ forces layout AGAIN!
});

// GOOD: Batch all reads, THEN all writes
const heights = elements.map(el => el.offsetHeight); // batch READ
elements.forEach((el, i) => {
  el.style.height = heights[i] * 2 + 'px'; // batch WRITE
});

// Virtual scrolling – only render visible items
// For a list of 100,000 items, only render ~20-30 visible in viewport
// Angular CDK VirtualScroll:
// <cdk-virtual-scroll-viewport itemSize="48">
//   <div *cdkVirtualFor="let item of items">{{item.name}}</div>
// </cdk-virtual-scroll-viewport>

// Web Worker – offload heavy computation to background thread
// worker.ts
self.addEventListener('message', (e) => {
  const sorted = e.data.sort((a, b) => a.price - b.price); // heavy work
  self.postMessage(sorted);
});
// main.ts
const worker = new Worker(new URL('./worker', import.meta.url));
worker.postMessage(hugeDataSet);
worker.onmessage = ({ data }) => renderTable(data);
// Main thread stays responsive!
```

49. Rendering Performance

Checklist: Critical rendering path · reflow vs repaint · composite layers · will-change · transform/opacity for animations · reducing DOM size · CSS containment

```
// Critical rendering path
// 1. HTML → DOM tree
// 2. CSS → CSSOM tree
// 3. DOM + CSSOM → Render tree (skip display:none)
// 4. Layout – calculate positions and sizes
// 5. Paint – fill in pixels
// 6. Compositing – combine layers, render to screen

// Cheap properties (compositor only – no layout/paint):
// transform, opacity, filter, will-change

// Medium properties (paint only – no layout):
// color, background, box-shadow, visibility

// Expensive properties (trigger layout + paint):
// width, height, padding, margin, top, left, font-size, display

// Reduce DOM size
// ■ Virtual scrolling for long lists
// ■ Remove hidden elements (not just visibility:hidden)
// ■ Use CSS for decorative elements (::before, ::after)
// ■ Flatten deeply nested structures
// Target: under 1500 DOM nodes for good performance
```

50. Caching Strategies

Checklist: Browser caching headers · Service Worker caching · Cache-Control · ETag · stale-while-revalidate · application cache · memoization

```
// HTTP Cache headers
Cache-Control: public, max-age=31536000, immutable // static assets with hash
Cache-Control: no-cache // always revalidate
Cache-Control: no-store // never cache (sensitive data)
Cache-Control: public, max-age=0, must-revalidate // use ETag

// ETag – content-based validation
// Server sends: ETag: "abc123"
// Browser sends: If-None-Match: "abc123"
// Server returns 304 Not Modified (no body) if unchanged

// stale-while-revalidate – serve stale, update in background
Cache-Control: max-age=3600, stale-while-revalidate=86400
// Serve cached for 1hr, then serve stale for 24hrs while revalidating

// Service Worker caching strategies
// 1. Cache First: check cache → fallback to network (static assets)
// 2. Network First: check network → fallback to cache (API data)
// 3. Stale While Revalidate: serve cache → update from network
// 4. Cache Only: only from cache (offline resources)
// 5. Network Only: never cache (real-time data)

// Application-level memoization
const apiCache = new Map();
async function cachedFetch(url, ttl = 60000) {
  const cached = apiCache.get(url);
  if (cached && Date.now() - cached.timestamp < ttl) return cached.data;
  const data = await fetch(url).then(r => r.json());
  apiCache.set(url, { data, timestamp: Date.now() });
  return data;
}
```


51. Core Web Vitals

Checklist: LCP · FID/INP · CLS · FCP · TTI · TBT

Metric	What	Good	Poor	How to fix
LCP	Largest Contentful Paint — when biggest element renders	< 2.5s	> 4.0s	Optimize hero image, preload fonts, SSR
INP	Interaction to Next Paint — responsiveness to ALL interactions	< 200ms	> 500ms	Break long tasks, yield to main thread, Web Workers
CLS	Cumulative Layout Shift — visual stability	< 0.1	> 0.25	Set image dimensions, avoid dynamic content insertion
FCP	First Contentful Paint — first pixel	< 1.8s	> 3.0s	Inline critical CSS, eliminate render-blocking resources
TBT	Total Blocking Time — sum of long task blocking time	< 200ms	> 600ms	Code split, defer non-critical JS, requestIdleCallback

```
// Measure in your app
import { onLCP, onINP, onCLS } from 'web-vitals';
onLCP(metric => sendToAnalytics('LCP', metric));
onINP(metric => sendToAnalytics('INP', metric));
onCLS(metric => sendToAnalytics('CLS', metric));

// Yield to main thread – prevent long tasks
async function processLargeArray(items) {
  for (let i = 0; i < items.length; i++) {
    processItem(items[i]);
    if (i % 100 === 0) {
      // Yield every 100 items so browser can handle user interactions
      await new Promise(resolve => setTimeout(resolve, 0));
    }
  }
}
```

52. Images & Media Performance

Checklist: Responsive images (srcset/sizes) · modern formats (WebP/AVIF) · compression · lazy loading · image sprites · SVG optimization · video optimization

```
<!-- Responsive images with modern formats -->
<picture>
  <source srcset="hero-400.avif 400w, hero-800.avif 800w, hero-1200.avif 1200w"
    sizes="(max-width:600px) 400px, (max-width:1000px) 800px, 1200px"
    type="image/avif">
  <source srcset="hero-400.webp 400w, hero-800.webp 800w"
    type="image/webp">
   <!-- high priority for LCP image -->
</picture>

<!-- Always set width & height to prevent CLS -->


// SVG optimization checklist:
// 1. Remove metadata, comments (SVGO tool)
// 2. Use viewBox instead of width/height for responsiveness
// 3. Inline small SVGs (saves HTTP request)
// 4. Use <symbol> + <use> for repeated icons (sprite sheet)
```

53. Font Performance

Checklist: Font loading strategies · font-display · FOUT/FOIT/FOFT · subsetting · system font fallbacks

```
/* Font loading behavior:
  FOUT (Flash of Unstyled Text): text shows in fallback font, then swaps
  FOIT (Flash of Invisible Text): text is invisible until font loads
  FOFT (Flash of Faux Text): show faux bold/italic, then swap */

@font-face {
  font-family: 'Inter';
  src: url('/fonts/inter-var.woff2') format('woff2');
  font-display: swap;          /* FOUT – best for body text */
  font-weight: 100 900;       /* variable font */
  unicode-range: U+0000-00FF; /* only Latin characters = smaller file */
}

/* font-display values:
  swap:      show fallback immediately, swap when loaded (recommended)
  optional:  browser may skip loading if connection is slow
  fallback:  tiny invisible period (100ms), then fallback
  block:     invisible for up to 3s, then fallback */

/* Preload your critical font */
<link rel="preload" href="/fonts/inter-var.woff2" as="font"
      type="font/woff2" crossorigin>

/* System font stack – zero download time */
body {
  font-family: system-ui, -apple-system, BlinkMacSystemFont,
               'Segoe UI', Roboto, sans-serif;
}
```

54. Measuring Performance

Checklist: Performance API · Navigation Timing · Resource Timing · User Timing marks/measures · Lighthouse · WebPageTest · Chrome DevTools Performance panel

```
// Performance API – custom timing
performance.mark('component-render-start');
renderComponent();
performance.mark('component-render-end');
performance.measure('Component Render', 'component-render-start', 'component-render-end');

const entry = performance.getEntriesByName('Component Render')[0];
console.log(`Render took: ${entry.duration.toFixed(2)}ms`);

// Navigation Timing – page load breakdown
const nav = performance.getEntriesByType('navigation')[0];
console.log({
  dns: nav.domainLookupEnd - nav.domainLookupStart,
  tcp: nav.connectEnd - nav.connectStart,
  ttfb: nav.responseStart - nav.requestStart,
  download: nav.responseEnd - nav.responseStart,
  domParsing: nav.domInteractive - nav.responseEnd,
  domComplete: nav.domComplete - nav.domInteractive,
  total: nav.loadEventEnd - nav.startTime
});

// Resource Timing – individual resource metrics
const resources = performance.getEntriesByType('resource');
resources.forEach(r => {
  if (r.duration > 500) {
    console.warn(`Slow resource: ${r.name} took ${r.duration}ms`);
  }
});

// PerformanceObserver – watch for long tasks
const observer = new PerformanceObserver((list) => {
  list.getEntries().forEach(entry => {
    console.warn(`Long task detected: ${entry.duration}ms`);
    sendToMonitoring({ type: 'long-task', duration: entry.duration });
  });
});
observer.observe({ type: 'longtask', buffered: true });
```

■ Interview Questions — Performance

Q: A page has an LCP of 5 seconds. How do you diagnose and fix it?

A: (1) Identify the LCP element (DevTools → Performance → LCP marker). (2) If it's an image: preload it, use modern formats (AVIF/WebP), set `fetchpriority="high"`, ensure no lazy-loading on LCP image. (3) If it's text: inline critical CSS, preload fonts, use `font-display: swap`. (4) Check server TTFB — implement SSR, CDN, caching. (5) Remove render-blocking resources.

Q: How do you prevent layout shifts (CLS)?

A: (1) Always set width and height on images/videos. (2) Use `aspect-ratio` CSS for dynamic containers. (3) Reserve space for ads/embeds. (4) Avoid inserting content above existing content. (5) Use CSS `contain` for dynamic widgets. (6) Use `font-display: optional` to prevent font-swap shifts.

55. Common Vulnerabilities

Checklist: XSS (reflected, stored, DOM-based) · CSRF · SQL injection (API context) · clickjacking · MITM attacks

XSS (Cross-Site Scripting)

Type	How it works	Example
Reflected	Malicious script in URL, reflected back in response	?search=<script>alert(1)</script>
Stored	Script saved in DB, served to other users	Comment containing
DOM-based	Script modifies DOM via client-side JS (never hits server)	innerHTML = location.hash

CSRF (Cross-Site Request Forgery)

Tricks an authenticated user into making unwanted requests. The attacker creates a form on their site that submits to YOUR API — the browser automatically sends cookies, so the request looks legitimate.

```
// Attacker's page
<form action="https://bank.com/transfer" method="POST" id="evil">
  <input name="to" value="attacker">
  <input name="amount" value="10000">
</form>
<script>document.getElementById('evil').submit();</script>
// Prevention: CSRF tokens, SameSite cookies, check Origin header
```

Clickjacking

Loading your site in an invisible iframe and tricking users into clicking buttons. Prevention: X-Frame-Options: DENY or CSP frame-ancestors 'none'.

56. Prevention Techniques

Checklist: Input sanitization · output encoding · CSP · HTTPS · secure cookies · CORS · SRI · X-Frame-Options · X-Content-Type-Options

```
// XSS Prevention
// 1. ALWAYS use.textContent (not innerHTML) for user data
element.textContent = userInput; // ■ SAFE
element.innerHTML = userInput;   // ■ DANGEROUS

// 2. Content Security Policy (CSP) – whitelists allowed resources
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-abc123'; // only scripts with this nonce
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https://cdn.example.com;
  connect-src 'self' https://api.example.com;
  frame-ancestors 'none';           // prevents clickjacking
  base-uri 'self';
  form-action 'self';

// 3. Subresource Integrity – verify CDN files
<script src="https://cdn.example.com/lib.js"
  integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6..."
  crossorigin="anonymous"></script>

// 4. Security headers
X-Frame-Options: DENY           // no iframing
X-Content-Type-Options: nosniff // prevent MIME sniffing
X-XSS-Protection: 0           // disable old XSS filter (CSP is better)
Strict-Transport-Security: max-age=31536000; includeSubDomains // force HTTPS
Referrer-Policy: strict-origin-when-cross-origin

// 5. CORS – server controls which origins can access your API
Access-Control-Allow-Origin: https://my-app.com // NOT *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true

// 6. Secure cookies
Set-Cookie: session=abc123;
  HttpOnly;           // not accessible from JavaScript
  Secure;             // only sent over HTTPS
  SameSite=Strict;    // not sent with cross-site requests
  Max-Age=86400;      // expires in 24 hours
  Path=/;             // applies to entire site
```

57. Authentication & Authorization

Checklist: JWT · OAuth 2.0 · token storage · refresh token rotation · session management · password hashing · MFA

```
// JWT structure: header.payload.signature
// header: { "alg": "HS256", "typ": "JWT" }
// payload: { "sub": "user123", "name": "Sid", "iat": 1710000000, "exp": 1710003600 }
// signature: HMACSHA256(base64(header) + "." + base64(payload), secret)

// Token refresh pattern
class AuthService {
  private refreshing = false;
  private refreshQueue: Function[] = [];

  async fetchWithAuth(url: string, options?: RequestInit): Promise {
    let token = this.getAccessToken();
    let response = await fetch(url, {
      ...options,
      headers: { ...options?.headers, Authorization: `Bearer ${token}` }
    });

    if (response.status === 401) {
      token = await this.refreshToken();
      response = await fetch(url, {
        ...options,
        headers: { ...options?.headers, Authorization: `Bearer ${token}` }
      });
    }
    return response;
  }

  private async refreshToken(): Promise {
    if (this.refreshing) {
      return new Promise(resolve => this.refreshQueue.push(resolve));
    }
    this.refreshing = true;
    const res = await fetch('/api/auth/refresh', {
      method: 'POST',
      credentials: 'include' // send HttpOnly refresh token cookie
    });
    const { accessToken } = await res.json();
    this.setAccessToken(accessToken);
    this.refreshQueue.forEach(cb => cb(accessToken));
    this.refreshQueue = [];
    this.refreshing = false;
    return accessToken;
  }
}

// OAuth 2.0 Authorization Code Flow (with PKCE for SPAs)
// 1. App generates code_verifier + code_challenge
// 2. Redirect user to authorization server with code_challenge
// 3. User logs in, server redirects back with authorization code
// 4. App exchanges code + code_verifier for tokens
// 5. PKCE prevents authorization code interception attacks
```

■ Interview Questions — Security

Q: How do you prevent XSS in a frontend application?

A: (1) Never use `innerHTML` with user input — use `textContent`. (2) Implement Content Security Policy (CSP). (3) Sanitize HTML if you must render it (DOMPurify library). (4) Use frameworks' built-in escaping (Angular sanitizes by default). (5) Use HttpOnly cookies for tokens. (6) Validate and encode on both client and server.

Q: Where should you store tokens in a SPA?

A: Access token: In memory (JS variable) — short-lived, lost on refresh. **Refresh token:** In HttpOnly, Secure, SameSite=Strict cookie — not accessible from JS (immune to XSS). Avoid localStorage for sensitive tokens. This pattern gives security (HttpOnly cookie) + SPA convenience (in-memory access token).

58. HTTP Basics

Checklist: HTTP methods · status codes · request/response headers · Content-Type · Accept · Cache-Control · CORS headers

Method	Use	Idempotent?	Body?
GET	Read data	Yes	No
POST	Create resource	No	Yes
PUT	Replace entire resource	Yes	Yes
PATCH	Partial update	No	Yes
DELETE	Remove resource	Yes	Optional

Status Codes to Know

```
// 2xx – Success
200 OK // successful request
201 Created // resource created (POST)
204 No Content // success, no body (DELETE)

// 3xx – Redirection
301 Moved Permanently // SEO redirect (permanent)
302 Found // temporary redirect
304 Not Modified // use cached version (ETag/If-Modified-Since)

// 4xx – Client Errors
400 Bad Request // malformed request / validation error
401 Unauthorized // not authenticated (need to login)
403 Forbidden // authenticated but no permission
404 Not Found // resource doesn't exist
409 Conflict // resource conflict (e.g., duplicate)
422 Unprocessable Entity // validation failed (semantic error)
429 Too Many Requests // rate limited

// 5xx – Server Errors
500 Internal Server Error // generic server failure
502 Bad Gateway // upstream server error
503 Service Unavailable // server overloaded / maintenance
504 Gateway Timeout // upstream server timeout
```

59. REST API Design

Checklist: Resource naming · endpoint structure · versioning · HATEOAS · idempotency · pagination (offset/cursor) · filtering/sorting · rate limiting

```
// RESTful URL conventions
GET    /api/v1/users           // list all users
GET    /api/v1/users/42       // get user by ID
POST   /api/v1/users          // create user
PUT    /api/v1/users/42       // replace user 42
PATCH /api/v1/users/42       // partial update user 42
DELETE /api/v1/users/42       // delete user 42
GET    /api/v1/users/42/orders // user's orders (sub-resource)

// Pagination – cursor-based (better for real-time data)
GET /api/v1/products?cursor=eyJpZCI6NDJ9&limit=20
Response: {
  "data": [...],
  "pagination": {
    "next_cursor": "eyJpZCI6NjJ9",
    "has_more": true
  }
}

// Filtering + sorting
GET /api/v1/products?category=electronics&price_min=100&sort=-price&fields=name,price

// Rate limiting headers
X-RateLimit-Limit: 100           // max requests per window
X-RateLimit-Remaining: 42       // remaining in current window
X-RateLimit-Reset: 1710000060    // when window resets (Unix timestamp)
Retry-After: 30                 // when rate limited (429)
```

60. GraphQL

Checklist: Queries · mutations · fragments · variables · aliases · schema · resolvers · Apollo Client · over-fetching/under-fetching

```
// Query – fetch exactly what you need (no over-fetching)
query GetUserWithOrders($userId: ID!) {
  user(id: $userId) {
    name
    email
    orders(limit: 5) {
      id
      total
      status
      items {
        name
        quantity
      }
    }
  }
}

// Mutation – modify data
mutation CreateOrder($input: OrderInput!) {
  createOrder(input: $input) {
    id
    total
    status
  }
}

// Fragments – reusable field sets
fragment UserFields on User {
  id
  name
  email
  avatar
}
query { users { ...UserFields } }

// GraphQL vs REST trade-offs
// ■ GraphQL: single endpoint, no over-fetching, typed schema, self-documenting
// ■ GraphQL: complexity, caching harder, potential N+1 queries, learning curve
// ■ REST: simple, cacheable (HTTP), well-understood, tools ecosystem
// ■ REST: over-fetching, multiple endpoints, versioning complexity
```

61. Data Fetching

Checklist: Fetch API · Axios · AbortController · retry logic · error handling · loading states · optimistic updates · polling vs WebSocket vs SSE · request deduplication

```
// Fetch with AbortController – cancel on unmount
const controller = new AbortController();
try {
  const response = await fetch('/api/data', {
    signal: controller.signal,
    headers: { 'Content-Type': 'application/json' }
  });
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const data = await response.json();
} catch (error) {
  if (error.name === 'AbortError') return; // intentional cancel
  throw error;
}
// Cleanup: controller.abort();

// Retry with exponential backoff
async function fetchWithRetry(url, maxRetries = 3) {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      const res = await fetch(url);
      if (res.ok) return await res.json();
      if (res.status < 500) throw new Error(`Client error ${res.status}`);
    } catch (error) {
      if (attempt === maxRetries - 1) throw error;
      const delay = Math.min(1000 * Math.pow(2, attempt), 10000);
      await new Promise(r => setTimeout(r, delay + Math.random() * 1000));
    }
  }
}

// Optimistic update pattern
function addTodo(newTodo) {
  const tempId = crypto.randomUUID();
  // 1. Update UI instantly
  todos.update(list => [...list, { ...newTodo, id: tempId, status: 'pending' }]);
  // 2. Send to server
  api.createTodo(newTodo).then(
    saved => todos.update(list => list.map(t => t.id === tempId ? saved : t)),
    error => {
      todos.update(list => list.filter(t => t.id !== tempId)); // rollback
      showError('Failed to add todo');
    }
  );
}

// Polling vs WebSocket vs SSE
// Polling: Simple, works everywhere, wasteful if no updates
// WebSocket: Full-duplex, low latency, complex connection management
// SSE: Server → client only, auto-reconnect, simpler than WebSocket
```

62. WebSockets

Checklist: Connection establishment · message sending/receiving · Socket.io · reconnection strategies · heartbeat/ping-pong

```
// Native WebSocket
const ws = new WebSocket('wss://api.example.com/ws');

ws.addEventListener('open', () => {
  console.log('Connected');
  ws.send(JSON.stringify({ type: 'subscribe', channel: 'prices' }));
});

ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);
  updatePriceDisplay(data);
});

ws.addEventListener('close', (event) => {
  console.log(`Disconnected: ${event.code} ${event.reason}`);
  if (event.code !== 1000) reconnect(); // abnormal close → reconnect
});

ws.addEventListener('error', () => { ws.close(); });

// Reconnection with exponential backoff
class ReconnectingWebSocket {
  private ws: WebSocket;
  private retryCount = 0;
  private maxRetries = 10;

  connect() {
    this.ws = new WebSocket(this.url);
    this.ws.onopen = () => { this.retryCount = 0; this.startHeartbeat(); };
    this.ws.onclose = (e) => {
      if (e.code !== 1000 && this.retryCount < this.maxRetries) {
        const delay = Math.min(1000 * Math.pow(2, this.retryCount++), 30000);
        setTimeout(() => this.connect(), delay);
      }
    };
  }

  // Heartbeat – detect broken connections
  private startHeartbeat() {
    setInterval(() => {
      if (this.ws.readyState === WebSocket.OPEN) {
        this.ws.send(JSON.stringify({ type: 'ping' }));
      }
    }, 30000);
  }
}
```

63. State Management Patterns

Checklist: Store/actions/reducers · middleware · selectors · State types (server/client/local/global/URL/derived) · data fetching libraries · caching strategies · stale-while-revalidate

State Types

Type	Where it lives	Example
Server state	Remote, cached locally	User profile, product list, orders
Client state	Only in browser	UI toggles, theme, form input
Local state	Single component	Dropdown open/closed, form fields
Global state	Shared across app	Auth user, cart, notifications
URL state	In the URL	Filters, pagination, search query
Derived state	Computed from other state	Total price (items × quantities), filtered lists

```
// Redux pattern (framework-agnostic concept)
// Action → Reducer → New State → View updates

// 1. Action – describes WHAT happened
const addItem = (product) => ({ type: 'cart/addItem', payload: product });

// 2. Reducer – describes HOW state changes (pure function!)
function cartReducer(state = { items: [] }, action) {
  switch (action.type) {
    case 'cart/addItem':
      return { ...state, items: [...state.items, action.payload] };
    case 'cart/removeItem':
      return { ...state, items: state.items.filter(i => i.id !== action.payload) };
    default:
      return state;
  }
}

// 3. Selector – derives data from state (memoized for performance)
const selectCartTotal = (state) =>
  state.cart.items.reduce((sum, item) => sum + item.price * item.quantity, 0);

// Angular equivalent: NgRx / signals
// signals are Angular's built-in state management (Angular 16+)
const count = signal(0);
const doubled = computed(() => count() * 2); // derived state
count.set(5); // doubled() === 10
```

64. Unit Testing

Checklist: Jest test runner · describe/it/test · assertions & matchers · setup/teardown · mocking (jest.fn, jest.mock) · spies · timer mocks · coverage

```
// Test structure
describe('CartService', () => {
  let service: CartService;

  beforeEach(() => {
    service = new CartService(); // fresh instance for each test
  });

  afterEach(() => {
    jest.restoreAllMocks(); // cleanup mocks
  });

  describe('addItem', () => {
    it('should add item to cart', () => {
      const item = { id: '1', name: 'Laptop', price: 1200, quantity: 1 };
      service.addItem(item);
      expect(service.getItems()).toHaveLength(1);
      expect(service.getItems()[0]).toEqual(item);
    });

    it('should increment quantity for duplicate items', () => {
      const item = { id: '1', name: 'Laptop', price: 1200, quantity: 1 };
      service.addItem(item);
      service.addItem(item);
      expect(service.getItems()).toHaveLength(1);
      expect(service.getItems()[0].quantity).toBe(2);
    });

    it('should throw for invalid price', () => {
      expect(() => service.addItem({ id: '1', price: -10 }))
        .toThrow('Price must be positive');
    });
  });

  describe('getTotal', () => {
    it('should calculate total with tax', () => {
      service.addItem({ id: '1', price: 100, quantity: 2 });
      service.addItem({ id: '2', price: 50, quantity: 1 });
      expect(service.getTotal(0.1)).toBe(275); // (200 + 50) * 1.1
    });
  });
});

// Mocking
jest.mock('./api', () => ({
  fetchUser: jest.fn()
}));
import { fetchUser } from './api';

test('handles API errors gracefully', async () => {
  (fetchUser as jest.Mock).mockRejectedValue(new Error('Network error'));
  const result = await getUserSafe('123');
  expect(result).toBeNull();
  expect(fetchUser).toHaveBeenCalledWith('123');
});

// Timer mocking
jest.useFakeTimers();
test('debounce waits before calling', () => {
  const fn = jest.fn();
  const debounced = debounce(fn, 300);
```

```
    debounced(); debounced(); debounced();  
    expect(fn).not.toHaveBeenCalled();  
    jest.advanceTimersByTime(300);  
    expect(fn).toHaveBeenCalledTimes(1);  
  });
```


65. Integration Testing

Checklist: Testing component interactions · multi-component flows · API integration tests · testing with real dependencies

```
// Integration test – multiple components working together
describe('Checkout Flow', () => {
  it('should complete purchase flow', async () => {
    // Setup: mock API but test real component interaction
    server.use(
      rest.get('/api/products/:id', (req, res, ctx) =>
        res(ctx.json({ id: req.params.id, name: 'Laptop', price: 1200 })))
    ),
      rest.post('/api/orders', (req, res, ctx) =>
        res(ctx.json({ id: 'order-1', status: 'created' })))
    )
  });

  // Render full checkout page (multiple child components)
  render(<CheckoutPage />);

  // Wait for product to load
  await screen.findByText('Laptop');

  // Fill shipping form
  await userEvent.type(screen.getByLabelText('Street'), '123 Main St');
  await userEvent.type(screen.getByLabelText('City'), 'Bengaluru');

  // Fill payment
  await userEvent.type(screen.getByLabelText('Card Number'), '4242424242424242');

  // Submit
  await userEvent.click(screen.getByRole('button', { name: /place order/i }));

  // Verify success
  await screen.findByText('Order placed successfully');
});
```

66. E2E Testing (0 → Advanced)

Checklist: Cypress · Playwright · selectors/locators · network mocking · screenshots/video · parallel execution · page object model

66.1 Playwright — Modern E2E Framework

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';
export default defineConfig({
  testDir: './e2e',
  use: {
    baseURL: 'http://localhost:4200',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    trace: 'on-first-retry'
  },
  projects: [
    { name: 'chromium', use: { browserName: 'chromium' } },
    { name: 'firefox', use: { browserName: 'firefox' } },
    { name: 'webkit', use: { browserName: 'webkit' } }
  ],
  webServer: {
    command: 'ng serve',
    port: 4200,
    reuseExistingServer: !process.env.CI
  }
});

// Basic test
import { test, expect } from '@playwright/test';

test('user can search for products', async ({ page }) => {
  await page.goto('/products');
  await page.getByPlaceholder('Search products').fill('laptop');
  await page.getByRole('button', { name: 'Search' }).click();
  await expect(page.getByTestId('results-count')).toHaveText('5 results');
  await expect(page.getByRole('listitem')).toHaveCount(5);
});
```

66.2 Page Object Model — Scalable E2E

```
// pages/login.page.ts
class LoginPage {
  constructor(private page: Page) {}

  get emailInput() { return this.page.getByLabel('Email'); }
  get passwordInput() { return this.page.getByLabel('Password'); }
  get submitButton() { return this.page.getByRole('button', { name: 'Login' }); }
  get errorMessage() { return this.page.getByTestId('error-message'); }

  async goto() { await this.page.goto('/login'); }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.submitButton.click();
  }
}

// Using the page object
test('login with valid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('user@example.com', 'password123');
  await expect(page).toHaveURL('/dashboard');
});
```

```
test('shows error for invalid credentials', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('user@example.com', 'wrong');
  await expect(loginPage.errorMessage).toBeVisible();
  await expect(loginPage.errorMessage).toHaveText('Invalid credentials');
});
```

66.3 Network Mocking in E2E

```
// Playwright – intercept API calls
test('shows empty state when no products', async ({ page }) => {
  await page.route('/api/products', route =>
    route.fulfill({ status: 200, body: JSON.stringify([]) })
  );
  await page.goto('/products');
  await expect(page.getByText('No products found')).toBeVisible();
});

// Cypress equivalent
cy.intercept('GET', '/api/products', { body: [] }).as('getProducts');
cy.visit('/products');
cy.wait('@getProducts');
cy.contains('No products found').should('be.visible');
```

66.4 Advanced E2E Patterns

```
// Visual regression testing
test('homepage matches snapshot', async ({ page }) => {
  await page.goto('/');
  await expect(page).toHaveScreenshot('homepage.png', { maxDiffPixels: 100 });
});

// Testing accessibility
test('page should be accessible', async ({ page }) => {
  await page.goto('/');
  const violations = await new AxeBuilder({ page }).analyze();
  expect(violations.violations).toHaveLength(0);
});

// Testing with authentication state
test.describe('authenticated user', () => {
  test.use({
    storageState: 'e2e/.auth/user.json' // pre-saved login state
  });

  test('can access dashboard', async ({ page }) => {
    await page.goto('/dashboard');
    await expect(page.getByText('Welcome, Sid')).toBeVisible();
  });
});

// Parallel execution best practices
// - Each test should be independent (no shared state)
// - Use unique test data (UUIDs) to avoid conflicts
// - Use test fixtures for common setup
// - Keep tests fast: mock external services
```

67. Test-Driven Development

Checklist: Red-Green-Refactor cycle · writing tests first · benefits & trade-offs

```
// TDD: Red → Green → Refactor

// Step 1: RED – Write a failing test
test('isValidEmail returns true for valid emails', () => {
  expect(isValidEmail('user@example.com')).toBe(true);
  expect(isValidEmail('a.b@c.co')).toBe(true);
});
test('isValidEmail returns false for invalid emails', () => {
  expect(isValidEmail('invalid')).toBe(false);
  expect(isValidEmail('@no-local.com')).toBe(false);
  expect(isValidEmail('no-domain@')).toBe(false);
  expect(isValidEmail('')).toBe(false);
});
// Tests fail because isValidEmail doesn't exist yet!

// Step 2: GREEN – Write minimal code to pass
function isValidEmail(email) {
  return /^[^\s]+@[^\s]+\.[^\s]+$/.test(email);
}
// Tests pass!

// Step 3: REFACTOR – improve without changing behavior
function isValidEmail(email: string): boolean {
  if (!email || typeof email !== 'string') return false;
  return /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/.test(email);
}
// Tests still pass → refactoring is safe

// TDD benefits:
// ■ Forces you to think about API design before implementation
// ■ High test coverage by default
// ■ Confidence when refactoring
// ■ Tests serve as documentation
// ■ Slower initial development
// ■ Can lead to over-testing implementation details
```

■ Interview Questions — Testing

Q: What's the testing pyramid?

A: Bottom: **Unit tests** (many, fast, cheap — test individual functions/classes). Middle: **Integration tests** (fewer, test component interactions). Top: **E2E tests** (fewest, slow, expensive — test full user flows). Most tests should be unit tests; E2E for critical paths only.

Q: What makes a good unit test?

A: (1) **Fast** — runs in milliseconds. (2) **Isolated** — no external dependencies (mock them). (3) **Deterministic** — same result every time. (4) **Self-validating** — pass or fail, no manual inspection. (5) **Thorough** — covers happy path, edge cases, and error cases. (6) **Readable** — test name describes the behavior being tested.

68. Webpack

Checklist: Entry/output · loaders · plugins · code splitting · tree shaking · dev server · HMR · Module Federation

```
// webpack.config.js
const path = require('path');
const HtmlWebpackPlugin = require('html-webpack-plugin');
const MiniCssExtractPlugin = require('mini-css-extract-plugin');
const { BundleAnalyzerPlugin } = require('webpack-bundle-analyzer');

module.exports = (env, argv) => ({
  mode: argv.mode || 'development',

  // Entry – where Webpack starts building the dependency graph
  entry: {
    main: './src/index.ts',
    vendor: './src/vendor.ts' // separate bundle for 3rd-party code
  },

  // Output – where bundles go
  output: {
    path: path.resolve(__dirname, 'dist'),
    filename: '[name].[contenthash:8].js', // cache-busting
    chunkFilename: '[name].[contenthash:8].chunk.js',
    clean: true // clear dist/ on build
  },

  // Loaders – transform files before bundling
  module: {
    rules: [
      { test: /\.tsx?$/, use: 'ts-loader', exclude: /node_modules/ },
      {
        test: /\.scss$/,
        use: [
          MiniCssExtractPlugin.loader, // extract CSS to separate file
          'css-loader', // resolve @import, url()
          'postcss-loader', // autoprefixer, etc.
          'sass-loader' // compile SCSS → CSS
        ]
      },
      {
        test: /\.(png|jpg|gif|svg)$/,
        type: 'asset', // Webpack 5: auto inline/file based on size
        parser: { dataUrlCondition: { maxSize: 8 * 1024 } } // inline < 8KB
      }
    ]
  },

  // Plugins – extend Webpack functionality
  plugins: [
    new HtmlWebpackPlugin({ template: './src/index.html' }),
    new MiniCssExtractPlugin({ filename: '[name].[contenthash:8].css' }),
    new BundleAnalyzerPlugin({ analyzerMode: 'static', openAnalyzer: false })
  ],

  // Code splitting
  optimization: {
    splitChunks: {
      chunks: 'all',
      cacheGroups: {
        vendor: {
          test: /[\\/]node_modules[\\/]/,
          name: 'vendors',
          chunks: 'all'
        }
      }
    }
  }
});
```

```
    },
    runtimeChunk: 'single' // separate Webpack runtime
  },

  devServer: {
    port: 3000,
    hot: true, // Hot Module Replacement
    proxy: { '/api': 'http://localhost:8080' }
  },

  resolve: {
    extensions: ['.ts', '.tsx', '.js'],
    alias: { '@': path.resolve(__dirname, 'src') }
  }
});
```

69. Vite

Checklist: ES modules in dev · Rollup for production · config · plugins · CSS handling · env variables · SSR

```
// vite.config.ts
import { defineConfig } from 'vite';

export default defineConfig({
  root: 'src',
  build: {
    outDir: '../dist',
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['lodash-es', 'date-fns'], // separate vendor chunk
          charts: ['chart.js']
        }
      }
    },
    target: 'es2020',
    sourcemap: true,
    minify: 'terser'
  },
  server: {
    port: 3000,
    proxy: {
      '/api': { target: 'http://localhost:8080', changeOrigin: true }
    }
  },
  css: {
    preprocessorOptions: {
      scss: { additionalData: '@use "src/styles/vars" as *;' }
    },
    modules: { localsConvention: 'camelCase' }
  },
  resolve: {
    alias: { '@': '/src' }
  }
});

// Why Vite is faster than Webpack in development:
// 1. Uses native ES modules – no bundling in dev!
//    Browser requests modules → Vite serves them directly
// 2. Pre-bundles dependencies with esbuild (100x faster than JS-based tools)
// 3. HMR only updates the changed module (not full rebuild)
// 4. Production uses Rollup – optimized, tree-shaken bundles
```

70. Babel & Transpilation

Checklist: Presets · plugins · polyfills · browserslist · source maps · babel.config.json vs .babelrc

```
// babel.config.json
{
  "presets": [
    ["@babel/preset-env", {
      "targets": "> 0.5%, last 2 versions, not dead",
      "useBuiltIns": "usage",      // auto-import polyfills only when used
      "corejs": 3,                // polyfill library version
      "modules": false            // keep ES modules for tree-shaking
    }],
    "@babel/preset-typescript"
  ],
  "plugins": [
    ["@babel/plugin-proposal-decorators", { "version": "2023-11" }],
    "@babel/plugin-transform-runtime" // reuse Babel helpers
  ]
}

// .browserslistrc – shared target for Babel, PostCSS, etc.
> 0.5%
last 2 versions
not dead
not ie 11

// What Babel transforms:
// ES2015+ → ES5 (arrow functions, classes, async/await, optional chaining)
// TypeScript → JavaScript (strips types, no type checking)
// JSX → createElement calls
// Decorators → wrapper functions

// What Babel does NOT do:
// ■ Type checking (use tsc for that)
// ■ Bundling (that's Webpack/Rollup/Vite)
// ■ Minification (that's Terser/esbuild)
```


71. Package Managers

Checklist: npm vs yarn vs pnpm · lockfiles · semantic versioning · peer dependencies · workspaces · publishing

Feature	npm	yarn	pnpm
Speed	Moderate	Fast	Fastest
Disk usage	Duplicates	Duplicates	Content-addressable store (shared)
Lock file	package-lock.json	yarn.lock	pnpm-lock.yaml
Workspaces	Yes (npm 7+)	Yes	Yes
Strictness	Hoists all	Hoists all	Strict (symlinks, no phantom deps)

```
// Semantic Versioning: MAJOR.MINOR.PATCH
// ^1.2.3 → >=1.2.3 <2.0.0 (caret: compatible with minor updates)
// ~1.2.3 → >=1.2.3 <1.3.0 (tilde: compatible with patch updates)
// 1.2.3 → exactly 1.2.3

// Peer dependencies – "I expect the consumer to have this"
// Used by libraries to avoid bundling framework they extend
{
  "peerDependencies": {
    "@angular/core": ">=16.0.0"    // consumer must install Angular 16+
  },
  "peerDependenciesMeta": {
    "@angular/core": { "optional": false }
  }
}

// Monorepo workspaces (pnpm)
// pnpm-workspace.yaml
packages:
- 'packages/*'
- 'apps/*'
// Shared node_modules, symlinked packages, faster installs
```

72. Linting & Formatting

Checklist: ESLint configuration · custom rules · Prettier · Stylelint · pre-commit hooks (husky + lint-staged) · EditorConfig

```
// .eslintrc.json
{
  "root": true,
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:@angular-eslint/recommended",
    "prettier" // disables formatting rules (Prettier handles those)
  ],
  "rules": {
    "no-console": "warn",
    "@typescript-eslint/no-explicit-any": "error",
    "@typescript-eslint/no-unused-vars": ["error", { "argsIgnorePattern": "^_" } ]
  },
  "overrides": [{
    "files": ["*.spec.ts"],
    "rules": { "@typescript-eslint/no-explicit-any": "off" }
  }]
}

// .prettierrc
{
  "semi": true,
  "singleQuote": true,
  "printWidth": 100,
  "trailingComma": "all",
  "arrowParens": "always",
  "bracketSpacing": true
}

// Pre-commit hooks with Husky + lint-staged
// package.json
{
  "lint-staged": {
    "*.ts,js": ["eslint --fix", "prettier --write"],
    "*.css,scss": ["stylelint --fix", "prettier --write"],
    "*.html": ["prettier --write"]
  }
}

// .husky/pre-commit
#!/bin/sh
npx lint-staged
```

73. Environment Variables

Checklist: .env files · dotenv · environment-specific configs · never commit secrets

```
// Angular environment files
// src/environments/environment.ts (dev)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:3000/api',
  featureFlags: { darkMode: true, newCheckout: true }
};

// src/environments/environment.prod.ts (prod)
export const environment = {
  production: true,
  apiUrl: 'https://api.example.com',
  featureFlags: { darkMode: true, newCheckout: false }
};

// Vite — .env files
// .env                → loaded in all cases
// .env.local           → loaded in all cases, ignored by git
// .env.development     → loaded in dev mode
// .env.production      → loaded in production build

// .env
VITE_API_URL=http://localhost:3000
VITE_APP_TITLE=My App

// Access in code (only VITE_ prefixed vars are exposed!)
const apiUrl = import.meta.env.VITE_API_URL;

// Security rules:
// ■ NEVER put secrets in frontend .env files (they're in the bundle!)
// ■ NEVER commit .env.local or files with real credentials
// ■ Use backend environment variables for secrets
// ■ Use CI/CD environment variables for build-time config
// ■ Add .env*.local to .gitignore
```

■ Interview Questions — Build Tools

Q: Why is Vite faster than Webpack in development?

A: Vite uses native ES modules in development — the browser loads modules individually, so there's no bundling step. Dependencies are pre-bundled with esbuild (written in Go, 100x faster than JS-based bundlers). HMR only replaces the changed module, not rebuilds. In production, Vite uses Rollup for optimized output. Webpack bundles everything even in dev.

Q: What is tree shaking and how does it work?

A: Tree shaking eliminates dead code (unused exports) from the final bundle. It works by analyzing ES module static imports at build time. Requirements: (1) Use ES modules (import/export, not require). (2) Set "sideEffects": false in package.json. (3) Avoid side effects at module top level. (4) Use named imports, not import *. Webpack, Rollup, and esbuild all support it.

74. WCAG Guidelines

Checklist: WCAG 2.1 / 2.2 levels (A, AA, AAA) · POUR principles · conformance requirements · common violations

Four Principles — POUR

Principle	Meaning	Key Guidelines
Perceivable	Users can perceive content	Alt text, captions, color contrast, resizable text
Operable	UI can be operated	Keyboard accessible, enough time, no seizures, navigable
Understandable	Content is understandable	Readable, predictable, input assistance
Robust	Works with assistive tech	Valid HTML, ARIA used correctly, status messages

```
// Common WCAG violations and fixes
// 1. Missing alt text
 // ■ No alt
 // ■ Decorative (intentionally empty)
 // ■ Informative

// 2. Insufficient color contrast
// AA requires: 4.5:1 for normal text, 3:1 for large text (18px+ or 14px+ bold)
// AAA requires: 7:1 for normal text, 4.5:1 for large text
// Use: chrome DevTools → Inspect element → Color picker shows contrast ratio

// 3. Missing form labels
<input type="email"> // ■ No label
<label for="email">Email</label>
<input id="email" type="email"> // ■ Explicit label
<label>Email <input type="email"></label> // ■ Implicit label

// 4. Missing page language
<html> // ■
<html lang="en"> // ■

// 5. Missing skip navigation
<a href="#main-content" class="skip-link">Skip to main content</a>
<main id="main-content">...</main>
```

75. Keyboard Navigation

Checklist: Focus management · tab order · focus trapping (modals) · skip links · visible focus indicators · roving tabindex

```
// Focus management for SPA route changes
router.events.subscribe(event => {
  if (event instanceof NavigationEnd) {
    const heading = document.querySelector('h1');
    heading?.focus(); // move focus to new page heading
  }
});

// Focus trap for modals
class FocusTrap {
  private focusableElements: HTMLElement[];
  private firstFocusable: HTMLElement;
  private lastFocusable: HTMLElement;

  constructor(private container: HTMLElement) {
    this.focusableElements = Array.from(
      container.querySelectorAll(
        'a[href], button:not([disabled]), input:not([disabled]), ' +
        'select:not([disabled]), textarea:not([disabled]), [tabindex]:not([tabindex="-1"])'
      )
    );
    this.firstFocusable = this.focusableElements[0];
    this.lastFocusable = this.focusableElements[this.focusableElements.length - 1];
    this.handleKeyDown = this.handleKeyDown.bind(this);
    container.addEventListener('keydown', this.handleKeyDown);
    this.firstFocusable.focus();
  }

  private handleKeyDown(e: KeyboardEvent) {
    if (e.key !== 'Tab') return;
    if (e.shiftKey && document.activeElement === this.firstFocusable) {
      e.preventDefault();
      this.lastFocusable.focus();
    } else if (!e.shiftKey && document.activeElement === this.lastFocusable) {
      e.preventDefault();
      this.firstFocusable.focus();
    }
  }

  destroy() { this.container.removeEventListener('keydown', this.handleKeyDown); }
}

// Roving tabindex – arrow key navigation for widget groups (tabs, toolbars)
<div role="tablist">
  <button role="tab" tabindex="0" aria-selected="true">Tab 1</button>
  <button role="tab" tabindex="-1" aria-selected="false">Tab 2</button>
  <button role="tab" tabindex="-1" aria-selected="false">Tab 3</button>
</div>
// Only active tab has tabindex="0", others have tabindex="-1"
// Arrow keys move focus between tabs, Tab key moves out of the group
```

76. Screen Readers

Checklist: How screen readers parse DOM · landmark regions · heading hierarchy · hidden content · live regions

```
// Landmark regions – screen readers can jump between these
<header>...</header>           // banner
<nav>...</nav>                 // navigation
<main>...</main>               // main content
<aside>...</aside>             // complementary
<footer>...</footer>           // contentinfo

// Heading hierarchy – must be sequential, no skipping
<h1>Page Title</h1>            // one per page
  <h2>Section</h2>
    <h3>Subsection</h3>
    <h2>Another Section</h2>    // ■ back to h2 is okay
    <h4>Subsection</h4>        // ■ skipped h3!

// Visually hidden but accessible to screen readers
.sr-only {
  position: absolute;
  width: 1px; height: 1px;
  padding: 0; margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}
// display:none and visibility:hidden hide from screen readers too – don't use for a11y text

// Hiding from screen readers (decorative only)
<span aria-hidden="true">■</span>

```

77. ARIA Attributes

Checklist: Roles · states · properties · live regions · aria-label vs aria-labelledby · aria-describedby · aria-expanded · aria-hidden

```
// Rule #1: Don't use ARIA if native HTML works!
<div role="button" tabindex="0">Click</div> // ■ Fake button
<button>Click</button>                    // ■ Real button

// ARIA labels
<button aria-label="Close dialog">X</button>
<input aria-labelledby="name-label" />
<h2 id="name-label">Full Name</h2>

// ARIA states
<button aria-expanded="false" aria-controls="menu">Menu</button>
<ul id="menu" hidden>...</ul>

// Live regions – announce dynamic content changes
<div aria-live="polite">                                // announce when convenient
  {{ statusMessage }}
</div>
<div aria-live="assertive">                             // announce immediately
  {{ errorMessage }}
</div>
<div role="alert">                                       // equivalent to aria-live="assertive"
  Form submission failed!
</div>
<div role="status">                                     // equivalent to aria-live="polite"
  3 results found
</div>

// ARIA for custom widgets
<div role="progressbar" aria-valuenow="60" aria-valuemin="0" aria-valuemax="100"
  aria-label="Upload progress">
  60%
</div>

// Common ARIA mistakes
// ■ aria-label on a <div> (non-interactive elements)
// ■ role="button" without keyboard support
// ■ aria-hidden="true" on focusable elements
// ■ Redundant: <button role="button"> (button already has that role)
```

78. Color & Contrast

Checklist: Contrast ratios (AA/AAA) · color-blind friendly design · not relying on color alone · dark mode accessibility

```
// Contrast requirements (WCAG 2.1)
// AA → 4.5:1 normal text, 3:1 large text, 3:1 UI components
// AAA → 7:1 normal text, 4.5:1 large text

// Never use color alone to convey information
// ■ Red text for errors (color-blind users can't see it)
<span style="color: red">Error</span>

// ■ Color + icon + text
<span class="error">
  <svg aria-hidden="true">...</svg> <!-- error icon -->
  Error: Email is required
</span>

// ■ Color + pattern for charts
// Use different patterns (dots, stripes, crosshatch) in addition to colors

// Dark mode – maintain contrast
:root {
  --text-primary: #1a1a1a; // dark text on light bg
  --bg-primary: #ffffff;
}
@media (prefers-color-scheme: dark) {
  :root {
    --text-primary: #e0e0e0; // light text on dark bg
    --bg-primary: #121212;
  }
}

// Test tools: Chrome DevTools → Rendering → Emulate vision deficiencies
```


79. Form Accessibility

Checklist: Labels · error messages · field descriptions · required fields · autocomplete · input types

```
// Accessible form pattern
<form novalidate (ngSubmit)="onSubmit()">
  <div class="form-group">
    <label for="email">
      Email address
      <span class="required" aria-hidden="true">*</span>
    </label>
    <input
      id="email"
      type="email"
      autocomplete="email"
      required
      aria-required="true"
      [attr.aria-invalid]="emailInvalid"
      [attr.aria-describedby]="emailInvalid ? 'email-error' : 'email-hint'"
    >
    <span id="email-hint" class="hint">We'll never share your email.</span>
    <span id="email-error" class="error" *ngIf="emailInvalid" role="alert">
      Please enter a valid email address.
    </span>
  </div>

  <fieldset>
    <legend>Notification preferences</legend>
    <label><input type="checkbox" name="email-notif"> Email</label>
    <label><input type="checkbox" name="sms-notif"> SMS</label>
  </fieldset>

  <button type="submit">Submit</button>
</form>

// Key patterns:
// 1. Every input has a <label> with matching for/id
// 2. Required fields use aria-required + visual indicator
// 3. Errors use role="alert" or aria-live to announce
// 4. aria-describedby links input to hint/error text
// 5. aria-invalid indicates validation state
// 6. Use <fieldset> + <legend> for related inputs
// 7. autocomplete attribute helps autofill
```

■ Interview Questions — Accessibility

Q: How do you make a custom dropdown accessible?

A: Use ARIA: role="listbox" on container, role="option" on items. Add aria-expanded, aria-activedescendant (points to focused option), aria-labelledby. Support keyboard: Enter/Space to open, Arrow keys to navigate, Escape to close. Focus trap while open. Announce selection changes with aria-live. Better approach: use native <select> when possible.

Q: What is the difference between aria-label, aria-labelledby, and aria-describedby?

A: aria-label: inline text label (no visible element). aria-labelledby: points to element ID whose text IS the label (overrides aria-label + native label). aria-describedby: points to element providing EXTRA description (read after the label). Priority: aria-labelledby > aria-label > <label> > title.

80. Creational Patterns

Checklist: Singleton · Factory · Abstract Factory · Builder · Prototype

```
// SINGLETON – one instance for the entire application
class Logger {
  private static instance: Logger;
  private logs: string[] = [];

  private constructor() {} // prevent external instantiation

  static getInstance(): Logger {
    if (!Logger.instance) {
      Logger.instance = new Logger();
    }
    return Logger.instance;
  }

  log(message: string) {
    this.logs.push(`${new Date().toISOString()} ${message}`);
    console.log(message);
  }
}

// Angular services are singletons by default (providedIn: 'root')
@Injectable({ providedIn: 'root' })
export class LoggerService {
  log(msg: string) { console.log(msg); }
}

// FACTORY – create objects without specifying exact class
interface Notification { send(to: string, message: string): void; }

class EmailNotification implements Notification {
  send(to: string, message: string) { /* send email */ }
}
class SmsNotification implements Notification {
  send(to: string, message: string) { /* send SMS */ }
}
class PushNotification implements Notification {
  send(to: string, message: string) { /* send push */ }
}

function createNotification(type: 'email' | 'sms' | 'push'): Notification {
  const map = {
    email: EmailNotification,
    sms: SmsNotification,
    push: PushNotification
  };
  return new map[type]();
}

// BUILDER – construct complex objects step by step
class QueryBuilder {
  private table = '';
  private conditions: string[] = [];
  private orderByField = '';
  private limitCount = 0;

  from(table: string) { this.table = table; return this; }
  where(condition: string) { this.conditions.push(condition); return this; }
  orderBy(field: string) { this.orderByField = field; return this; }
  limit(n: number) { this.limitCount = n; return this; }

  build(): string {
    let query = `SELECT * FROM ${this.table}`;
  }
}
```

```

        if (this.conditions.length) query += ` WHERE ${this.conditions.join(' AND ')} `;
        if (this.orderByField) query += ` ORDER BY ${this.orderByField} `;
        if (this.limitCount) query += ` LIMIT ${this.limitCount} `;
        return query;
    }
}

const query = new QueryBuilder()
    .from('users')
    .where('age > 18')
    .where('active = true')
    .orderBy('name')
    .limit(10)
    .build();

```

81. Structural Patterns

Checklist: Adapter · Decorator · Facade · Proxy · Composite

```
// ADAPTER – make incompatible interfaces work together
// Old analytics library
class OldAnalytics {
  track(eventName: string, data: object) { /* old API */ }
}
// New analytics library
class NewAnalytics {
  send(event: { name: string; properties: object; timestamp: number }) { /* new API */ }
}
// Adapter
class AnalyticsAdapter {
  constructor(private newAnalytics: NewAnalytics) {}
  track(eventName: string, data: object) {
    this.newAnalytics.send({
      name: eventName,
      properties: data,
      timestamp: Date.now()
    });
  }
}

// DECORATOR – add behavior without modifying original
function withLogging<T> extends (...args: any[]) => any>(fn: T): T {
  return function(...args: any[]) {
    console.log(`Calling ${fn.name} with`, args);
    const result = fn.apply(this, args);
    console.log(`Result:`, result);
    return result;
  } as T;
}
const add = (a: number, b: number) => a + b;
const loggedAdd = withLogging(add);
loggedAdd(2, 3); // logs: Calling add with [2, 3] → Result: 5

// FACADE – simple interface for complex subsystem
class OrderFacade {
  constructor(
    private inventory: InventoryService,
    private payment: PaymentService,
    private shipping: ShippingService,
    private notification: NotificationService
  ) {}

  async placeOrder(order: Order): Promise<OrderResult> {
    await this.inventory.reserve(order.items);
    const payment = await this.payment.charge(order.total, order.paymentMethod);
    const tracking = await this.shipping.createLabel(order.address, order.items);
    await this.notification.sendConfirmation(order.email, tracking);
    return { orderId: order.id, tracking, payment };
  }
}

// PROXY – control access to an object
class CachedApiProxy {
  private cache = new Map<string, { data: any; expires: number }>();

  constructor(private api: ApiService, private ttl = 60000) {}

  async get(url: string) {
    const cached = this.cache.get(url);
    if (cached && cached.expires > Date.now()) return cached.data;
    const data = await this.api.get(url);
```

```
        this.cache.set(url, { data, expires: Date.now() + this.ttl });  
        return data;  
    }  
}
```

82. Behavioral Patterns

Checklist: Observer · Strategy · Command · Mediator · Iterator

```
// OBSERVER – event-driven communication
class EventEmitter {
  private listeners = new Map<string, Set<Function>>();

  on(event: string, callback: Function) {
    if (!this.listeners.has(event)) this.listeners.set(event, new Set());
    this.listeners.get(event)!.add(callback);
    return () => this.listeners.get(event)!.delete(callback); // unsubscribe
  }

  emit(event: string, ...args: any[]) {
    this.listeners.get(event)?.forEach(cb => cb(...args));
  }
}

// Angular: RxJS Subjects ARE the Observer pattern
const subject = new Subject<string>();
const sub = subject.subscribe(value => console.log(value));
subject.next('Hello');
sub.unsubscribe();

// STRATEGY – swap algorithms at runtime
interface SortStrategy<T> {
  sort(data: T[]): T[];
}
class QuickSort<T> implements SortStrategy<T> { sort(data: T[]) { /* ... */ } }
class MergeSort<T> implements SortStrategy<T> { sort(data: T[]) { /* ... */ } }

class DataProcessor<T> {
  constructor(private strategy: SortStrategy<T>) {}
  setStrategy(s: SortStrategy<T>) { this.strategy = s; }
  process(data: T[]) { return this.strategy.sort(data); }
}

// COMMAND – encapsulate actions for undo/redo
interface Command {
  execute(): void;
  undo(): void;
}
class AddTextCommand implements Command {
  constructor(private editor: Editor, private text: string, private position: number) {}
  execute() { this.editor.insert(this.text, this.position); }
  undo() { this.editor.delete(this.position, this.position + this.text.length); }
}
class CommandHistory {
  private history: Command[] = [];
  private pointer = -1;
  execute(cmd: Command) { cmd.execute(); this.history[++this.pointer] = cmd; }
  undo() { if (this.pointer >= 0) this.history[this.pointer--].undo(); }
  redo() { if (this.pointer < this.history.length - 1) this.history[++this.pointer].execute(); }
}
```

83. Architectural Patterns

Checklist: MVC · MVVM · MVP · Flux/Redux · Clean Architecture · Hexagonal Architecture

```
// MVC (Model-View-Controller)
// Model: data + business logic
// View: UI rendering
// Controller: handles user input, updates model → view
// Used in: classic web frameworks (Express, Ruby on Rails)

// MVVM (Model-View-ViewModel) – Angular's pattern
// Model: data/entities
// View: template (HTML)
// ViewModel: component class (data binding, event handling)
@Component({
  template: `
    <input [ngModel]="name" (ngModelChange)="name = $event">
    <p>Hello, {{ name }}</p>
  `
})
class GreetComponent {
  name = 'World'; // ViewModel – two-way binding to view
}

// Flux/Redux – unidirectional data flow
// View → Action → Dispatcher → Store → View
// Benefits: predictable state, time-travel debugging, easy testing

// Clean Architecture layers (inside → outside)
// 1. Entities: business rules, domain objects
// 2. Use Cases: application-specific logic
// 3. Interface Adapters: controllers, presenters, gateways
// 4. Frameworks: Angular, database, HTTP
// Rule: dependencies point INWARD only

// Frontend Clean Architecture
src/
  domain/           // entities, value objects (no framework imports!)
    user.model.ts
    order.model.ts
  application/      // use cases, ports (interfaces)
    create-order.usecase.ts
    order.repository.ts // interface
  infrastructure/   // adapters (implementations)
    http-order.repository.ts // implements order.repository.ts
    local-storage.adapter.ts
  presentation/     // Angular components, pages
    order-list.component.ts
    order-detail.component.ts
```

■ Interview Questions — Design Patterns

Q: Which patterns do you use most in frontend development?

A: Observer (RxJS, event emitters, pub/sub for component communication). **Strategy** (swappable validators, sort algorithms, rendering strategies). **Facade** (service layer abstracting complex API calls). **Singleton** (Angular services with providedIn: root). **Decorator** (Angular decorators, function wrappers for logging/caching). **Factory** (dynamic component creation, object creation based on config).

84. Data Structures

Checklist: Arrays · Linked Lists · Stacks · Queues · Hash Tables · Trees · Graphs · Heaps · Tries

```
// STACK – LIFO (Last In, First Out)
class Stack<T> {
  private items: T[] = [];
  push(item: T) { this.items.push(item); }
  pop(): T | undefined { return this.items.pop(); }
  peek(): T | undefined { return this.items[this.items.length - 1]; }
  get size() { return this.items.length; }
  get isEmpty() { return this.items.length === 0; }
}
// Use cases: undo/redo, browser history, balanced parentheses, call stack

// QUEUE – FIFO (First In, First Out)
class Queue<T> {
  private items: T[] = [];
  enqueue(item: T) { this.items.push(item); }
  dequeue(): T | undefined { return this.items.shift(); }
  peek(): T | undefined { return this.items[0]; }
}
// Use cases: task scheduling, BFS, print queue, message queues

// LINKED LIST
class ListNode<T> {
  constructor(public value: T, public next: ListNode<T> | null = null) {}
}
class LinkedList<T> {
  head: ListNode<T> | null = null;

  prepend(value: T) {
    this.head = new ListNode(value, this.head);
  }

  append(value: T) {
    if (!this.head) { this.head = new ListNode(value); return; }
    let current = this.head;
    while (current.next) current = current.next;
    current.next = new ListNode(value);
  }

  delete(value: T) {
    if (!this.head) return;
    if (this.head.value === value) { this.head = this.head.next; return; }
    let current = this.head;
    while (current.next && current.next.value !== value) current = current.next;
    if (current.next) current.next = current.next.next;
  }
}

// HASH MAP – O(1) average lookup, insert, delete
const map = new Map<string, number>();
map.set('apple', 3);
map.get('apple'); // 3
map.has('apple'); // true
// Collision resolution: chaining (linked lists) or open addressing (linear probing)

// BINARY SEARCH TREE
class BSTNode {
  constructor(
    public value: number,
    public left: BSTNode | null = null,
    public right: BSTNode | null = null
  ) {}
}
```



```

class BST {
    root: BSTNode | null = null;

    insert(value: number) {
        if (!this.root) { this.root = new BSTNode(value); return; }
        let current = this.root;
        while (true) {
            if (value < current.value) {
                if (!current.left) { current.left = new BSTNode(value); return; }
                current = current.left;
            } else {
                if (!current.right) { current.right = new BSTNode(value); return; }
                current = current.right;
            }
        }
    }
}

// In-order traversal → sorted output
inOrder(node = this.root, result: number[] = []): number[] {
    if (node) {
        this.inOrder(node.left, result);
        result.push(node.value);
        this.inOrder(node.right, result);
    }
    return result;
}

// TRIE – prefix tree for autocomplete, spell-check
class TrieNode {
    children = new Map<string, TrieNode>();
    isEnd = false;
}
class Trie {
    root = new TrieNode();

    insert(word: string) {
        let node = this.root;
        for (const char of word) {
            if (!node.children.has(char)) node.children.set(char, new TrieNode());
            node = node.children.get(char)!;
        }
        node.isEnd = true;
    }

    search(word: string): boolean {
        let node = this.root;
        for (const char of word) {
            if (!node.children.has(char)) return false;
            node = node.children.get(char)!;
        }
        return node.isEnd;
    }

    startsWith(prefix: string): string[] {
        let node = this.root;
        for (const char of prefix) {
            if (!node.children.has(char)) return [];
            node = node.children.get(char)!;
        }
        return this.collectWords(node, prefix);
    }

    private collectWords(node: TrieNode, prefix: string): string[] {
        const results: string[] = [];

```

```
    if (node.isEnd) results.push(prefix);
    for (const [char, child] of node.children) {
      results.push(...this.collectWords(child, prefix + char));
    }
    return results;
  }
}
```

85. Common Algorithms

Checklist: Sorting (bubble, merge, quick) · Binary search · BFS · DFS · Two pointers · Sliding window · Dynamic programming basics

```
// BINARY SEARCH –  $O(\log n)$ 
function binarySearch(arr: number[], target: number): number {
  let left = 0, right = arr.length - 1;
  while (left <= right) {
    const mid = Math.floor((left + right) / 2);
    if (arr[mid] === target) return mid;
    if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1; // not found
}

// MERGE SORT –  $O(n \log n)$ , stable
function mergeSort(arr: number[]): number[] {
  if (arr.length <= 1) return arr;
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));
  return merge(left, right);
}
function merge(a: number[], b: number[]): number[] {
  const result: number[] = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    result.push(a[i] <= b[j] ? a[i++] : b[j++]);
  }
  return result.concat(a.slice(i), b.slice(j));
}

// BFS – level-order traversal, shortest path (unweighted)
function bfs(root: TreeNode): number[][] {
  const levels: number[][] = [];
  const queue: TreeNode[] = [root];
  while (queue.length) {
    const level: number[] = [];
    const size = queue.length;
    for (let i = 0; i < size; i++) {
      const node = queue.shift()!;
      level.push(node.value);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    levels.push(level);
  }
  return levels;
}

// DFS – three traversals
function dfsPreOrder(node: TreeNode | null): number[] {
  if (!node) return [];
  return [node.value, ...dfsPreOrder(node.left), ...dfsPreOrder(node.right)];
}
function dfsInOrder(node: TreeNode | null): number[] {
  if (!node) return [];
  return [...dfsInOrder(node.left), node.value, ...dfsInOrder(node.right)];
}
function dfsPostOrder(node: TreeNode | null): number[] {
  if (!node) return [];
  return [...dfsPostOrder(node.left), ...dfsPostOrder(node.right), node.value];
}

// TWO POINTERS – find pair with target sum in sorted array
```

```

function twoSum(arr: number[], target: number): [number, number] | null {
  let left = 0, right = arr.length - 1;
  while (left < right) {
    const sum = arr[left] + arr[right];
    if (sum === target) return [left, right];
    if (sum < target) left++;
    else right--;
  }
  return null;
}

// SLIDING WINDOW – max sum subarray of size k
function maxSumSubarray(arr: number[], k: number): number {
  let windowSum = arr.slice(0, k).reduce((a, b) => a + b, 0);
  let maxSum = windowSum;
  for (let i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i - k]; // slide: add right, remove left
    maxSum = Math.max(maxSum, windowSum);
  }
  return maxSum;
}

// DYNAMIC PROGRAMMING – Fibonacci (bottom-up)
function fibonacci(n: number): number {
  if (n <= 1) return n;
  let prev2 = 0, prev1 = 1;
  for (let i = 2; i <= n; i++) {
    [prev2, prev1] = [prev1, prev2 + prev1];
  }
  return prev1;
}

```

86. Complexity Analysis

Checklist: Big O notation · time vs space · common complexities · amortized analysis

Big O	Name	Example
$O(1)$	Constant	Array access, hash map get/set
$O(\log n)$	Logarithmic	Binary search, balanced BST operations
$O(n)$	Linear	Array iteration, linear search
$O(n \log n)$	Linearithmic	Merge sort, quick sort (average), heap sort
$O(n^2)$	Quadratic	Bubble sort, nested loops, selection sort
$O(2^n)$	Exponential	Recursive Fibonacci (naive), power set
$O(n!)$	Factorial	Permutations, traveling salesman (brute force)

```
// JavaScript built-in complexities
// Array:
//   push/pop:  $O(1)$  amortized
//   shift/unshift:  $O(n)$  ← moves all elements!
//   splice:  $O(n)$ 
//   indexOf/includes:  $O(n)$ 
//   sort:  $O(n \log n)$ 
//   map/filter/reduce:  $O(n)$ 

// Map/Set:
//   get/set/has/delete:  $O(1)$  average
//   forEach:  $O(n)$ 

// Object:
//   property access:  $O(1)$ 
//   Object.keys():  $O(n)$ 

// Space complexity examples:
//  $O(1)$  – in-place sorting, fixed variables
//  $O(n)$  – creating a copy of array, hash map of  $n$  items
//  $O(n^2)$  – 2D matrix of size  $n \times n$ 
//  $O(\log n)$  – recursive binary search (call stack depth)
```

■ Interview Questions — DSA

Q: What's the time complexity of searching in a hash map vs a sorted array?

A: Hash map: $O(1)$ average, $O(n)$ worst case (hash collisions). Sorted array: $O(\log n)$ with binary search. Hash map is faster for single lookups. Sorted array is better when you need range queries or sorted output.

Q: When would you choose a linked list over an array?

A: When you need frequent insertions/deletions at arbitrary positions ($O(1)$ for linked list vs $O(n)$ for array shifting). In practice, arrays are almost always faster due to cache locality (contiguous memory). Linked lists are rarely used directly in frontend — but the concept is used in LRU caches, undo history, and task queues.

87. Component Design

Checklist: Reusable components · props/inputs design · composition vs inheritance · compound components · render props / content projection · single responsibility

```
// Designing a reusable DataTable component
@Component({
  selector: 'app-data-table',
  template: `
    <table>
      <thead>
        <tr>
          <th *ngFor="let col of columns"
            (click)="sortBy(col.key)"
            [class.sorted]="sortColumn === col.key">
            {{ col.label }}
            <span *ngIf="sortColumn === col.key">
              {{ sortDirection === 'asc' ? '▲' : '▼' }}
            </span>
          </th>
        </tr>
      </thead>
      <tbody>
        <tr *ngFor="let row of paginatedData; trackBy: trackByFn">
          <td *ngFor="let col of columns">
            <ng-container *ngIf="col.template; else defaultCell">
              <ng-container *ngTemplateOutlet="col.template; context: { $implicit: row }">
            </ng-container>
            </ng-container>
            <ng-template #defaultCell>{{ row[col.key] }}</ng-template>
          </td>
        </tr>
      </tbody>
    </table>
    <app-pagination [total]="data.length" [pageSize]="pageSize"
      (pageChange)="currentPage = $event">
    </app-pagination>
  `
})
export class DataTableComponent {
  @Input() columns: Column[] = [];
  @Input() data: any[] = [];
  @Input() pageSize = 10;
  @Input() trackByFn = (i: number, item: any) => item.id;

  sortColumn = '';
  sortDirection: 'asc' | 'desc' = 'asc';
  currentPage = 1;

  get paginatedData() {
    const sorted = this.sortedData;
    const start = (this.currentPage - 1) * this.pageSize;
    return sorted.slice(start, start + this.pageSize);
  }
}

// Design principles for reusable components:
// 1. Single Responsibility – one component, one concern
// 2. Open/Closed – extensible (ng-template, content projection) without modifying source
// 3. Configurability – reasonable defaults, override-able via inputs
// 4. Type safety – strong typing for inputs/outputs
// 5. Accessibility – ARIA attributes, keyboard support built-in
```

88. Application Architecture

Checklist: Folder structure · feature modules · core/shared modules · lazy loading · state management placement · service layer · API layer

```
// Scalable Angular project structure
src/
  app/
    core/                                // singleton services, global guards, interceptors
      auth/
        auth.service.ts
        auth.guard.ts
        auth.interceptor.ts
      http/
        api.service.ts                 // base API service
        error.interceptor.ts
      core.module.ts                   // imported ONCE in AppModule

    shared/                             // reusable components, pipes, directives
      components/
        data-table/
        modal/
        pagination/
        loading-spinner/
      pipes/
        date-format.pipe.ts
        currency.pipe.ts
      directives/
        click-outside.directive.ts
        tooltip.directive.ts
      shared.module.ts                 // imported by feature modules

    features/                           // lazy-loaded feature modules
      dashboard/
        components/
          dashboard-chart.component.ts
          dashboard-stats.component.ts
        pages/
          dashboard.component.ts
        services/
          dashboard.service.ts
          dashboard-routing.module.ts
          dashboard.module.ts

      orders/
        components/
        pages/
        models/
          order.model.ts
        services/
          order.service.ts
          order.resolver.ts
        store/                          // feature-level state (if using NgRx)
          order.actions.ts
          order.reducer.ts
          order.effects.ts
          order.selectors.ts
        orders-routing.module.ts
        orders.module.ts

    app-routing.module.ts              // lazy-load routes
    app.module.ts
    app.component.ts

// Lazy loading routes
const routes: Routes = [
```

```
{ path: '', redirectTo: 'dashboard', pathMatch: 'full' },  
{ path: 'dashboard', loadChildren: () =>  
  import('./features/dashboard/dashboard.module').then(m => m.DashboardModule) },  
{ path: 'orders', loadChildren: () =>  
  import('./features/orders/orders.module').then(m => m.OrdersModule) }  
];
```


89. Common Components to Design

Checklist: Autocomplete · infinite scroll list · modal system · toast notifications · form builder · file uploader · drag & drop

```
// AUTOCOMPLETE – typeahead search
class AutocompleteController {
  private searchSubject = new Subject<string>();
  results$: Observable<SearchResult[]>;

  constructor(private api: SearchService) {
    this.results$ = this.searchSubject.pipe(
      debounceTime(300), // wait 300ms after typing stops
      distinctUntilChanged(), // skip if same query
      filter(q => q.length >= 2), // min 2 chars
      switchMap(query => // cancel previous request
        this.api.search(query).pipe(
          catchError(() => of([])) // graceful error handling
        )
      )
    );
  }

  onInput(query: string) { this.searchSubject.next(query); }
}

// INFINITE SCROLL – with IntersectionObserver
class InfiniteScrollList {
  private page = 1;
  private loading = false;
  private hasMore = true;

  constructor(private sentinel: HTMLElement) {
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting && !this.loading && this.hasMore) {
          this.loadMore();
        }
      },
      { rootMargin: '200px' } // trigger 200px before sentinel is visible
    );
    observer.observe(this.sentinel);
  }

  async loadMore() {
    this.loading = true;
    const items = await api.getItems({ page: this.page, limit: 20 });
    this.renderItems(items);
    this.hasMore = items.length === 20;
    this.page++;
    this.loading = false;
  }
}

// TOAST NOTIFICATION SYSTEM
interface Toast {
  id: string;
  type: 'success' | 'error' | 'warning' | 'info';
  message: string;
  duration?: number;
}

@Injectable({ providedIn: 'root' })
class ToastService {
  private toasts = new BehaviorSubject<Toast[]>([]);
  toasts$ = this.toasts.asObservable();
}
```

```

show(type: Toast['type'], message: string, duration = 5000) {
  const toast: Toast = { id: crypto.randomUUID(), type, message, duration };
  this.toasts.next([...this.toasts.value, toast]);
  if (duration > 0) {
    setTimeout(() => this.dismiss(toast.id), duration);
  }
}

dismiss(id: string) {
  this.toasts.next(this.toasts.value.filter(t => t.id !== id));
}
}

```

90. Full Application Designs

Checklist: E-commerce · chat application · social media feed · dashboard with real-time data · collaborative editor

```
// E-COMMERCE ARCHITECTURE
```

```
// High-level architecture:
```

```
// CDN → Load Balancer → Web Servers → API Gateway → Microservices
```

```
//
```

```
■■■ Auth Service
```

```
//
```

```
■■■ Product Service (catalog + search)
```

```
//
```

```
■■■ Cart Service (Redis for speed)
```

```
//
```

```
■■■ Order Service (with saga pattern)
```

```
//
```

```
■■■ Payment Service (Stripe/Razorpay)
```

```
//
```

```
■■■ Notification Service (email/SMS/push)
```

```
//
```

```
■■■ Inventory Service
```

```
// Frontend considerations:
```

```
// 1. Product listing: virtual scrolling, lazy image loading, skeleton screens
```

```
// 2. Search: debounced input, faceted filters, URL state for shareability
```

```
// 3. Cart: persist in localStorage + sync with server, optimistic updates
```

```
// 4. Checkout: multi-step form with validation, address autocomplete
```

```
// 5. Payment: PCI compliance (never handle raw card data, use Stripe Elements)
```

```
// 6. Real-time: stock alerts via WebSocket, price updates
```

```
// CHAT APPLICATION ARCHITECTURE
```

```
// WebSocket connection for real-time messaging
```

```
// Message states: sending → sent → delivered → read
```

```
// Offline support: queue messages in IndexedDB, sync on reconnect
```

```
// Typing indicators: debounced WebSocket events
```

```
// Message pagination: cursor-based (load more on scroll up)
```

```
// Features to design:
```

```
// - Message types: text, image, file, reply, forward
```

```
// - Group chat: member management, admin controls
```

```
// - Read receipts: track per-user read position
```

```
// - Search: full-text search across conversations
```

```
// - Push notifications: Service Worker + Push API
```

```
// - Encryption: end-to-end (Signal protocol)
```

91. Scalability Topics

Checklist: CDN · load balancing · caching layers · database sharding · message queues · micro-frontends · SSR/SSG

```
// Frontend scalability strategies

// 1. CDN – serve static assets from edge locations
// Put HTML, CSS, JS, images on CDN (CloudFront, Cloudflare)
// Cache-Control: public, max-age=31536000, immutable (for hashed files)
// Cache-Control: no-cache (for index.html – always revalidate)

// 2. Code splitting – load only what's needed
const routes = [
  { path: 'admin', loadChildren: () => import('./admin/admin.module') },
  // Admin module (50KB) only loaded for admin users
];

// 3. SSR (Server-Side Rendering) with Angular Universal
// Benefits: faster FCP, better SEO, works without JS
// Use for: marketing pages, product pages, blog posts
// Trade-offs: server cost, TTFB latency, hydration complexity

// 4. SSG (Static Site Generation) for content pages
// Pre-render at build time → deploy as static files
// Best for: docs, blogs, landing pages, product catalogs

// 5. Service Worker caching
// Cache critical resources for offline use
// Strategies: cache-first (assets), network-first (API), stale-while-revalidate

// 6. Micro-frontends – independently deployable frontend modules
// Module Federation (Webpack 5)
// Each team owns a feature, builds/deploys independently
// Shell app loads feature modules at runtime
new ModuleFederationPlugin({
  name: 'shell',
  remotes: {
    products: 'products@https://products.example.com/remoteEntry.js',
    checkout: 'checkout@https://checkout.example.com/remoteEntry.js'
  }
});

// 7. Performance budgets
// Set limits: main bundle < 200KB, page load < 3s
// Enforce in CI: fail build if budget exceeded
// angular.json budgets:
"budgets": [
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" },
  { "type": "anyComponentStyle", "maximumWarning": "4kb" }
]
```

92. Git Basics

Checklist: add / commit / push / pull · branching · merging · stashing · .gitignore · commit messages

```
// Essential Git commands
git init // initialize repo
git clone <url> // clone remote repo
git add . // stage all changes
git commit -m "feat: add login" // commit with message
git push origin main // push to remote
git pull origin main // fetch + merge

// Branching
git branch feature/login // create branch
git checkout feature/login // switch to branch
git checkout -b feature/login // create + switch (shortcut)
git branch -d feature/login // delete merged branch
git branch -D feature/login // force delete unmerged branch

// Merging
git merge feature/login // merge branch into current
git merge --no-ff feature/login // create merge commit even if fast-forward

// Stashing
git stash // save uncommitted changes
git stash list // list stashes
git stash pop // apply + delete latest stash
git stash apply stash@{1} // apply specific stash (keep it)

// Conventional commit messages
// type(scope): description
// feat: new feature
// fix: bug fix
// docs: documentation
// style: formatting (no code change)
// refactor: restructuring (no feature/fix)
// perf: performance improvement
// test: adding/fixing tests
// chore: tooling, dependencies, CI

git commit -m "feat(auth): add OAuth 2.0 login"
git commit -m "fix(cart): prevent duplicate items"
git commit -m "perf(images): lazy-load below-fold images"
```

93. Git Workflows

Checklist: Git Flow · GitHub Flow · trunk-based development · feature branches · release branches

```
// GITHUB FLOW (simple, recommended for most teams)
// 1. Create feature branch from main
// 2. Make commits
// 3. Open Pull Request
// 4. Code review + CI checks
// 5. Merge to main
// 6. Deploy from main

// GIT FLOW (for versioned releases)
// main – production code, tagged releases
// develop – integration branch
// feature/* – new features (branch from develop)
// release/* – release prep (branch from develop)
// hotfix/* – urgent fixes (branch from main)

// TRUNK-BASED DEVELOPMENT (for CI/CD teams)
// Everyone commits to main (or short-lived branches < 1 day)
// Feature flags control incomplete features in production
// Key: small, frequent commits, excellent CI pipeline

// Pull Request best practices:
// - Small PRs (< 400 lines changed)
// - Clear title following conventional commits
// - Description with context, screenshots, testing steps
// - Link to issue/ticket
// - Self-review before requesting review
// - Address all comments before merging
```

94. Advanced Git

Checklist: Rebase vs merge · interactive rebase · cherry-pick · bisect · reflog · hooks

```
// Rebase – rewrite history for clean linear history
git checkout feature/login
git rebase main // replay feature commits on top of main
// ■■ Never rebase public/shared branches!

// Interactive rebase – clean up commits before PR
git rebase -i HEAD~3 // edit last 3 commits
// pick abc1234 feat: add login form
// squash def5678 fix: typo in login → combine into previous
// reword ghi9012 add validation → rename commit

// Cherry-pick – apply specific commit to another branch
git cherry-pick abc1234 // apply commit abc1234 to current branch

// Bisect – find which commit introduced a bug
git bisect start
git bisect bad // current commit is bad
git bisect good v1.0.0 // last known good commit
// Git checks out middle commit, you test, mark good/bad
// Repeats binary search until the bad commit is found

// Reflog – recover "lost" commits
git reflog // shows ALL recent HEAD movements
git checkout abc1234 // go back to a "lost" commit
git branch recovered abc1234 // create branch from lost commit

// Git hooks (with Husky)
// pre-commit: lint, format, run tests
// commit-msg: validate commit message format
// pre-push: run full test suite
```

95. Browser Rendering Pipeline

Checklist: DOM construction · CSSOM · render tree · layout · paint · compositing · critical rendering path

```
// Browser rendering pipeline:
// 1. Parse HTML → DOM tree
// 2. Parse CSS → CSSOM tree
// 3. Combine DOM + CSSOM → Render tree (visible elements only)
// 4. Layout (Reflow) – calculate position and size of each element
// 5. Paint – fill pixels (colors, borders, shadows, text)
// 6. Composite – combine painted layers, handle transforms, opacity

// What triggers each stage:
// Layout (most expensive):
//   width, height, padding, margin, top/left, font-size, display, position
// Paint:
//   color, background, border-radius, box-shadow, visibility
// Composite only (cheapest – GPU accelerated):
//   transform, opacity, will-change, filter

// Optimization strategies:
// 1. Minimize DOM depth and size (<1500 nodes ideal)
// 2. Use CSS containment: contain: layout style paint;
// 3. Batch DOM reads/writes (avoid layout thrashing)
// 4. Use transform instead of top/left for animations
// 5. Use content-visibility: auto for off-screen content
// 6. Use requestAnimationFrame for visual updates

// Layout thrashing example:
// ■ BAD – reads and writes interleaved
for (const el of elements) {
  const width = el.offsetWidth;    // FORCE LAYOUT (read)
  el.style.width = width * 2 + 'px'; // write
  // next iteration forces layout again!
}

// ■ GOOD – batch reads, then batch writes
const widths = elements.map(el => el.offsetWidth); // all reads
elements.forEach((el, i) => {
  el.style.width = widths[i] * 2 + 'px';           // all writes
});
```


96. How Browsers Work

Checklist: URL to pixels · DNS resolution · TCP/TLS · HTTP request · parsing · rendering · JavaScript execution

```
// What happens when you type a URL and press Enter:

// 1. URL PARSING
//   Browser parses URL: protocol + domain + path + query + fragment

// 2. DNS RESOLUTION
//   Browser cache → OS cache → Router cache → ISP DNS → Root DNS
//   → TLD server (.com) → Authoritative DNS → IP address

// 3. TCP CONNECTION
//   Three-way handshake: SYN → SYN-ACK → ACK
//   If HTTPS: TLS handshake (certificate exchange, key agreement)

// 4. HTTP REQUEST
//   GET /index.html HTTP/2
//   Host: example.com
//   Accept: text/html
//   Cookie: session=abc123

// 5. SERVER RESPONSE
//   HTTP/2 200 OK
//   Content-Type: text/html
//   Set-Cookie: session=abc123

// 6. HTML PARSING
//   Parser builds DOM tree token by token
//   When it hits <link rel="stylesheet"> → parallel CSS download + CSSOM build
//   When it hits <script> → BLOCKS parsing (unless async/defer)

// 7. SCRIPT LOADING
//   <script>           → blocks parsing, download + execute immediately
//   <script async>     → download in parallel, execute ASAP (blocks parsing)
//   <script defer>     → download in parallel, execute AFTER HTML parsing
//   <script type="module"> → deferred by default

// 8. RENDERING
//   DOM + CSSOM → Render Tree → Layout → Paint → Composite → Pixels on screen

// 9. HYDRATION (for SSR apps)
//   Attach event listeners to server-rendered HTML
//   Make the page interactive
```

97. Progressive Enhancement

Checklist: Core functionality without JS · feature detection · graceful degradation · polyfills

```
// Progressive enhancement: build from baseline up
// 1. HTML – semantic, works without CSS/JS
// 2. CSS – enhanced layout and visuals
// 3. JavaScript – enhanced interactivity

// Feature detection (prefer over browser sniffing)
if ('IntersectionObserver' in window) {
  // use IntersectionObserver for lazy loading
} else {
  // fallback: load all images immediately
}

if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js');
}

if (CSS.supports('display', 'grid')) {
  // use CSS Grid layout
}

// @supports in CSS
@supports (display: grid) {
  .container { display: grid; }
}
@supports not (display: grid) {
  .container { display: flex; flex-wrap: wrap; }
}

// Modern progressive enhancement
// - <noscript> message for JS-disabled users
// - Server-side rendering as baseline
// - Client-side hydration for interactivity
// - Feature flags for browser capabilities
```

98. Web Standards

Checklist: W3C · WHATWG · TC39 · ECMAScript proposal stages · Web APIs · browser compatibility

```
// TC39 ECMAScript Proposal Stages
// Stage 0: Strawperson – idea proposed
// Stage 1: Proposal – problem described, solution outlined
// Stage 2: Draft – formal spec language, experimental implementations
// Stage 3: Candidate – complete spec, needs real-world feedback
// Stage 4: Finished – ready for inclusion in ECMAScript standard

// Recent notable proposals:
// ■ Stage 4: Array.prototype.findLast(), Object.groupBy()
// ■ Stage 4: Promise.withResolvers()
// ■ Stage 3: Temporal (date/time replacement for Date)
// ■ Stage 3: Decorators
// ■ Stage 2: Pattern Matching

// Checking browser support:
// - caniuse.com – browser support tables
// - MDN – compatibility tables per API
// - @babel/preset-env – auto-polyfill based on browserslist target
// - core-js – polyfill library for ES features
```

99. Mobile & Progressive Web Apps

Checklist: Responsive techniques · mobile-specific concerns · touch events · viewport · PWA: manifest · service worker · push notifications · offline support · install prompt

```
// PWA – Progressive Web App essentials

// 1. Web App Manifest
// manifest.json
{
  "name": "My Interview App",
  "short_name": "Interview",
  "start_url": "/",
  "display": "standalone",          // fullscreen, standalone, minimal-ui, browser
  "theme_color": "#1a5c2e",
  "background_color": "#ffffff",
  "icons": [
    { "src": "/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}

// 2. Service Worker – offline support
// sw.js
const CACHE_NAME = 'v1';
const ASSETS = ['/', '/index.html', '/styles.css', '/app.js'];

self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => cache.addAll(ASSETS))
  );
});

self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(cached => {
      return cached || fetch(event.request).then(response => {
        const clone = response.clone();
        caches.open(CACHE_NAME).then(cache => cache.put(event.request, clone));
        return response;
      });
    })
  );
});

// 3. Install prompt
let deferredPrompt;
window.addEventListener('beforeinstallprompt', (e) => {
  e.preventDefault();
  deferredPrompt = e;
  showInstallButton();
});
installBtn.addEventListener('click', () => {
  deferredPrompt.prompt();
});

// Mobile touch events
element.addEventListener('touchstart', handler, { passive: true }); // passive improves scroll p
// Touch gestures: touchstart → touchmove → touchend
// Pointer events (modern): pointerdown → pointermove → pointerup (unified mouse+touch+pen)

// Viewport meta tag (required for mobile)
<meta name="viewport" content="width=device-width, initial-scale=1">
```

100. CSS Architecture

Checklist: Specificity management · cascade & inheritance · CSS organization · naming conventions · utility-first · component-scoped

```
// Specificity scoring: (inline, ID, class, element)
// *           → 0,0,0,0
// div         → 0,0,0,1
// .class      → 0,0,1,0
// #id         → 0,1,0,0
// style=""    → 1,0,0,0
// !important  → overrides everything (avoid!)

// div.active   → 0,0,1,1 (element + class)
// #nav .item a → 0,1,1,1 (id + class + element)
// :is(.a, #b)  → uses HIGHEST specificity of arguments (0,1,0,0)
// :where(.a, #b) → ALWAYS 0 specificity (great for defaults)

// Cascade layers (CSS @layer) – control specificity at scale
@layer reset, base, components, utilities;

@layer reset {
  * { margin: 0; padding: 0; box-sizing: border-box; }
}
@layer base {
  body { font-family: system-ui; line-height: 1.6; }
}
@layer components {
  .card { padding: 1rem; border: 1px solid #ddd; }
}
@layer utilities {
  .mt-4 { margin-top: 1rem; } // always wins over components
}

// Angular ViewEncapsulation
// Emulated (default): scopes CSS to component via attribute selectors
// None: styles are global
// ShadowDom: uses native Shadow DOM (true isolation)

// CSS organization for large projects
styles/
_variables.scss // design tokens
_mixins.scss   // reusable patterns
_reset.scss    // normalize/reset
_typography.scss // base type styles
_utilities.scss // utility classes
components/    // component-specific styles
pages/         // page-specific styles
main.scss      // imports everything
```

101. Modern JavaScript Features

Checklist: ES modules · dynamic import · top-level await · decorators · structuredClone · AbortController · Temporal (upcoming)

```
// Dynamic imports – code splitting at runtime
const module = await import('./heavy-chart-lib.js');
const chart = new module.ChartRenderer(data);

// Top-level await (ES2022)
const config = await fetch('/config.json').then(r => r.json());
export { config };

// structuredClone – deep clone (ES2022)
const original = { a: 1, b: { c: 2 }, d: new Date() };
const clone = structuredClone(original);
// ■ Handles: nested objects, arrays, dates, maps, sets, RegExp, ArrayBuffer
// ■ Cannot clone: functions, DOM nodes, Error objects, symbols

// Iterator helpers (Stage 3/4)
const iter = [1, 2, 3, 4, 5].values();
const result = iter.filter(x => x % 2).map(x => x * 10).toArray();
// [10, 30, 50] – lazy evaluation, no intermediate arrays

// Set methods (ES2025)
const setA = new Set([1, 2, 3, 4]);
const setB = new Set([3, 4, 5, 6]);
setA.intersection(setB); // Set {3, 4}
setA.union(setB);        // Set {1, 2, 3, 4, 5, 6}
setA.difference(setB);    // Set {1, 2}
setA.symmetricDifference(setB); // Set {1, 2, 5, 6}

// Promise.withResolvers() (ES2024)
const { promise, resolve, reject } = Promise.withResolvers();
// Useful when you need to resolve/reject from outside the constructor

// Object.groupBy() (ES2024)
const people = [
  { name: 'Alice', department: 'eng' },
  { name: 'Bob', department: 'sales' },
  { name: 'Carol', department: 'eng' }
];
const byDept = Object.groupBy(people, p => p.department);
// { eng: [Alice, Carol], sales: [Bob] }
```

102. Framework Comparisons

Checklist: Angular vs React vs Vue · change detection strategies · state management · when to use what

Feature	Angular	React	Vue
Type	Full framework	UI library	Progressive framework
Language	TypeScript	JavaScript/TypeScript	JavaScript/TypeScript
Change Detection	Zone.js / Signals	Virtual DOM diffing	Reactive proxies
State Mgmt	NgRx / Signals / Services	Redux / Zustand / Context	Pinia / Vuex
Routing	Built-in (@angular/router)	React Router (3rd party)	Vue Router (official)
Forms	Built-in (template/reactive)	Controlled/uncontrolled (+ libs)	v-model (built-in)
Learning Curve	Steep	Moderate	Gentle
Best For	Enterprise, large teams	Flexible, ecosystem	Quick setup, simplicity
Bundle Size	~50KB gzipped (core)	~40KB gzipped	~30KB gzipped

```
// Change detection comparison:
// Angular: Zone.js patches async APIs, triggers change detection on ANY async event
//   → Angular 16+ Signals: fine-grained reactivity, no Zone.js overhead
// React: setState/useState triggers re-render, Virtual DOM diffing
//   → React 18 concurrent features: transitions, suspense
// Vue: Reactive proxies track dependencies, only re-render what changed
//   → Most precise reactivity out of the box

// When to choose:
// Angular: large enterprise teams, need full batteries-included framework,
//   strict patterns, long-term maintainability
// React: flexible team, want ecosystem choice, need React Native for mobile
// Vue: smaller team, quick prototyping, gradual adoption
```

103. Soft Skills & Interview Process

Checklist: Code review skills · communication · problem-solving approach · project discussion

```
// Problem-solving framework for interview coding questions:
// 1. UNDERSTAND – repeat the problem, clarify inputs/outputs/edge cases
// 2. PLAN – discuss approach BEFORE coding, talk about trade-offs
// 3. CODE – write clean code, explain as you go
// 4. TEST – walk through with examples, check edge cases
// 5. OPTIMIZE – discuss time/space complexity, can we do better?

// Talking about past projects (STAR method):
// Situation: Context and challenge
// Task: Your specific responsibility
// Action: What YOU did (technical decisions, leadership)
// Result: Measurable outcome (performance improvement, team velocity)

// Code review checklist:
// ■ Does it solve the stated problem?
// ■ Are edge cases handled?
// ■ Is the code readable and maintainable?
// ■ Are there performance concerns?
// ■ Is it secure? (XSS, injection, auth)
// ■ Are there tests? Do they cover important cases?
// ■ Is it consistent with existing codebase patterns?
// ■ Are error messages helpful for debugging?

// Communication tips:
// - Think out loud – interviewers want to see your thought process
// - Ask clarifying questions before diving in
// - Discuss trade-offs (time vs space, simplicity vs performance)
// - Don't panic if stuck – explain what you're thinking
// - If you don't know something, say so and discuss how you'd find out
```


104. Real-World Implementation Patterns

Checklist: Debounce · Throttle · Deep Clone · Event Emitter · Flatten Array · Promise implementation · Curry · Memoize

104.1 Debounce

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
// With leading edge and cancel
function debounceAdvanced(fn, delay, { leading = false } = {}) {
  let timer;
  let isLeading = true;
  function debounced(...args) {
    if (leading && isLeading) {
      fn.apply(this, args);
      isLeading = false;
    }
    clearTimeout(timer);
    timer = setTimeout(() => {
      if (!leading) fn.apply(this, args);
      isLeading = true;
    }, delay);
  }
  debounced.cancel = () => { clearTimeout(timer); isLeading = true; };
  return debounced;
}
```

104.2 Throttle

```
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => (inThrottle = false), limit);
    }
  };
}
// Throttle with trailing call
function throttleTrailing(fn, limit) {
  let lastArgs, timer, lastTime = 0;
  return function(...args) {
    const now = Date.now();
    const remaining = limit - (now - lastTime);
    if (remaining <= 0) {
      fn.apply(this, args);
      lastTime = now;
    } else {
      lastArgs = args;
      clearTimeout(timer);
      timer = setTimeout(() => {
        fn.apply(this, lastArgs);
        lastTime = Date.now();
      }, remaining);
    }
  };
}
```

104.3 Deep Clone

```

function deepClone(obj, seen = new WeakMap()) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (obj instanceof Date) return new Date(obj);
  if (obj instanceof RegExp) return new RegExp(obj.source, obj.flags);
  if (obj instanceof Map) {
    const map = new Map();
    seen.set(obj, map);
    obj.forEach((v, k) => map.set(deepClone(k, seen), deepClone(v, seen)));
    return map;
  }
  if (obj instanceof Set) {
    const set = new Set();
    seen.set(obj, set);
    obj.forEach(v => set.add(deepClone(v, seen)));
    return set;
  }
  if (seen.has(obj)) return seen.get(obj); // handle circular references

  const clone = Array.isArray(obj) ? [] : Object.create(Object.getPrototypeOf(obj));
  seen.set(obj, clone);
  for (const key of Reflect.ownKeys(obj)) {
    clone[key] = deepClone(obj[key], seen);
  }
  return clone;
}
// In production, use: structuredClone(obj) or lodash.cloneDeep(obj)

```

104.4 Event Emitter

```

class EventEmitter {
  #listeners = new Map();

  on(event, callback) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, new Set());
    this.#listeners.get(event).add(callback);
    return () => this.off(event, callback); // return unsubscribe
  }

  once(event, callback) {
    const wrapper = (...args) => {
      callback(...args);
      this.off(event, wrapper);
    };
    return this.on(event, wrapper);
  }

  off(event, callback) {
    this.#listeners.get(event)?.delete(callback);
  }

  emit(event, ...args) {
    this.#listeners.get(event)?.forEach(cb => cb(...args));
  }
}

```

104.5 Flatten Array

```

// Recursive
function flatten(arr, depth = Infinity) {
  return depth > 0
    ? arr.reduce((acc, val) =>
      acc.concat(Array.isArray(val) ? flatten(val, depth - 1) : val), [])
    : arr.slice();
}

// Iterative (no recursion)
function flattenIterative(arr) {

```

```

const stack = [...arr];
const result = [];
while (stack.length) {
  const item = stack.pop();
  Array.isArray(item) ? stack.push(...item) : result.push(item);
}
return result.reverse();
}

```

```
// Built-in: arr.flat(Infinity)
```

104.6 Promise Implementation

```

class MyPromise {
  #state = 'pending'; // pending | fulfilled | rejected
  #value;
  #handlers = [];

  constructor(executor) {
    const resolve = (value) => {
      if (this.#state !== 'pending') return;
      this.#state = 'fulfilled';
      this.#value = value;
      this.#handlers.forEach(h => h.onFulfilled(value));
    };
    const reject = (reason) => {
      if (this.#state !== 'pending') return;
      this.#state = 'rejected';
      this.#value = reason;
      this.#handlers.forEach(h => h.onRejected(reason));
    };
    try { executor(resolve, reject); }
    catch (e) { reject(e); }
  }

  then(onFulfilled, onRejected) {
    return new MyPromise((resolve, reject) => {
      const handle = (fn, fallback) => (value) => {
        try {
          const result = (fn || fallback)(value);
          result instanceof MyPromise ? result.then(resolve, reject) : resolve(result);
        } catch (e) { reject(e); }
      };
      const handler = {
        onFulfilled: handle(onFulfilled, v => v),
        onRejected: handle(onRejected, e => { throw e; })
      };
      if (this.#state === 'pending') this.#handlers.push(handler);
      else if (this.#state === 'fulfilled') queueMicrotask(() => handler.onFulfilled(this.#value));
      else queueMicrotask(() => handler.onRejected(this.#value));
    });
  }

  catch(onRejected) { return this.then(null, onRejected); }

  static resolve(value) { return new MyPromise(r => r(value)); }
  static reject(reason) { return new MyPromise( (_, r) => r(reason)); }
}

```

104.7 Curry Function

```

function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function(...moreArgs) {

```

```

        return curried.apply(this, args.concat(moreArgs));
    };
}

```

```

const add = curry((a, b, c) => a + b + c);
add(1)(2)(3);    // 6
add(1, 2)(3);    // 6
add(1)(2, 3);    // 6
add(1, 2, 3);    // 6

```

104.8 Memoize

```

function memoize(fn) {
    const cache = new Map();
    return function(...args) {
        const key = JSON.stringify(args);
        if (cache.has(key)) return cache.get(key);
        const result = fn.apply(this, args);
        cache.set(key, result);
        return result;
    };
}

// LRU Memoize (bounded cache size)
function memoizeLRU(fn, maxSize = 100) {
    const cache = new Map();
    return function(...args) {
        const key = JSON.stringify(args);
        if (cache.has(key)) {
            const value = cache.get(key);
            cache.delete(key);    // move to end (most recently used)
            cache.set(key, value);
            return value;
        }
        const result = fn.apply(this, args);
        cache.set(key, result);
        if (cache.size > maxSize) {
            const oldest = cache.keys().next().value;
            cache.delete(oldest);    // evict least recently used
        }
        return result;
    };
}

```

105. Additional Interview Topics

Checklist: Web Components · Internationalization (i18n) · Error Tracking · SEO for SPAs · Micro-frontends · Monorepos · CI/CD

105.1 Web Components

```
// Custom Elements + Shadow DOM + HTML Templates
class MyCard extends HTMLElement {
  constructor() {
    super();
    const shadow = this.attachShadow({ mode: 'open' });
    shadow.innerHTML = `
      <style>
        :host { display: block; border: 1px solid #ddd; border-radius: 8px; padding: 16px; }
        :host([variant="primary"]) { border-color: #1a5c2e; }
        ::slotted(h3) { margin-top: 0; }
      </style>
      <slot name="header"></slot>
      <slot></slot>
    `;
  }

  static get observedAttributes() { return ['variant']; }
  attributeChangedCallback(name, oldVal, newVal) { /* react to changes */ }
  connectedCallback() { /* element added to DOM */ }
  disconnectedCallback() { /* element removed from DOM */ }
}

customElements.define('my-card', MyCard);
// Usage: <my-card variant="primary"><h3 slot="header">Title</h3>Content</my-card>
```

105.2 Internationalization (i18n)

```
// Angular i18n (built-in)
<h1 i18n="@@welcomeTitle">Welcome to our app</h1>
<p i18n>You have {{ count }} items in your cart.</p>

// JavaScript Intl API
new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' }).format(1234567.89);
// "₹12,34,567.89"

new Intl.DateTimeFormat('en-GB', { dateStyle: 'long' }).format(new Date());
// "15 March 2025"

new Intl.RelativeTimeFormat('en', { numeric: 'auto' }).format(-1, 'day');
// "yesterday"

new Intl.PluralRules('en').select(1); // "one"
new Intl.PluralRules('en').select(5); // "other"

// RTL (Right-to-Left) support
<html dir="auto" lang="ar">
// Use CSS logical properties: margin-inline-start instead of margin-left
```

105.3 Error Tracking & Monitoring

```
// Global error handlers
window.addEventListener('error', (event) => {
  reportError({ message: event.message, stack: event.error?.stack,
    filename: event.filename, line: event.lineno, col: event.colno });
});

window.addEventListener('unhandledrejection', (event) => {
  reportError({ message: event.reason?.message || 'Unhandled promise rejection',
    stack: event.reason?.stack });
});

// Angular global error handler
```

```

@Injectable()
class GlobalErrorHandler implements ErrorHandler {
  constructor(private logger: LoggingService) {}
  handleError(error: any) {
    this.logger.logError(error);
    // Send to Sentry, DataDog, etc.
  }
}
// providers: [{ provide: ErrorHandler, useClass: GlobalErrorHandler }]

// Source maps in production
// Upload source maps to error tracking service (Sentry, DataDog)
// NEVER serve source maps to users (security risk)

```

105.4 SEO for SPAs

```

// SPA SEO challenges:
// - Search engines may not execute JavaScript
// - Dynamic content not in initial HTML
// - No unique meta tags per page

// Solutions:
// 1. SSR (Angular Universal) – render on server, send full HTML
// 2. Pre-rendering – generate static HTML at build time
// 3. Dynamic meta tags
@Component({})
class ProductPage {
  constructor(private meta: Meta, private title: Title) {}

  ngOnInit() {
    this.title.setTitle('Product Name | My Store');
    this.meta.updateTag({ name: 'description', content: 'Product description...' });
    this.meta.updateTag({ property: 'og:title', content: 'Product Name' });
    this.meta.updateTag({ property: 'og:image', content: '/images/product.jpg' });
  }
}

// 4. Structured data (JSON-LD)
<script type="application/ld+json">{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "Product Name",
  "offers": { "@type": "Offer", "price": "29.99", "priceCurrency": "USD" }
}</script>

```

105.5 Micro-frontends

```

// Approaches:
// 1. Module Federation (Webpack 5) – runtime module sharing
// 2. Single-SPA – framework-agnostic orchestrator
// 3. iframes – simple isolation, limited interaction
// 4. Web Components – framework-agnostic, Shadow DOM isolation

// Module Federation example
// Remote app (team-products)
new ModuleFederationPlugin({
  name: 'products',
  filename: 'remoteEntry.js',
  exposes: {
    './ProductList': './src/app/product-list/product-list.module.ts'
  },
  shared: { '@angular/core': { singleton: true }, rxjs: { singleton: true } }
});

// Shell app
new ModuleFederationPlugin({
  name: 'shell',

```

```

remotes: {
  products: 'products@http://localhost:4201/remoteEntry.js'
},
shared: { '@angular/core': { singleton: true }, rxjs: { singleton: true } }
});

```

// Challenges: shared dependencies, routing, styling conflicts, communication

105.6 Monorepos

// Tools: Nx, Turborepo, Lerna, pnpm workspaces
 // Benefits: shared code, atomic changes, unified CI, consistent tooling

// Nx workspace structure

```

my-workspace/
  apps/
    web-app/
    admin-app/
    api/
  libs/
    shared/
      ui/           // shared UI components
      utils/        // shared utilities
      models/       // shared TypeScript interfaces
    feature/
      auth/         // auth feature library
      products/     // products feature library

```

// nx.json – define task dependencies

```

{
  "targetDefaults": {
    "build": { "dependsOn": ["^build"] }, // build deps first
    "test": { "dependsOn": ["build"] }
  }
}

```

// npx nx affected --target=test → only test what changed

105.7 CI/CD Pipeline

// GitHub Actions example

```

name: CI/CD Pipeline
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: 'npm' }
      - run: npm ci # install exact versions from lockfile
      - run: npm run lint # lint check
      - run: npm run test -- --ci # unit tests
      - run: npm run build -- --prod # production build
      - run: npm run e2e # E2E tests

  deploy:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/upload-artifact@v4

```

```
with: { name: dist, path: dist/ }  
# Deploy to hosting (Vercel, AWS, Azure, etc.)
```

■ Interview Questions — System Design & Architecture

Q: How would you design a real-time collaborative editor?

A: Use **Operational Transformation (OT)** or **CRDTs** for conflict resolution. WebSocket for real-time sync. Each user's cursor position broadcast to others. Operations: insert, delete at position. Server maintains canonical document state. Offline support: queue operations, apply on reconnect. Libraries: Yjs (CRDT), ShareDB (OT). Cursor presence using ephemeral WebSocket messages.

Q: How do you decide between SSR, SSG, and CSR?

A: **SSR** (Server-Side Rendering): dynamic content, SEO-critical, personalized pages (e.g., product pages, dashboards). **SSG** (Static Site Generation): content doesn't change often, public pages (e.g., blog, docs, landing pages). **CSR** (Client-Side Rendering): behind auth, interactive apps, real-time features (e.g., admin panels, chat). Many apps use a mix: SSG for marketing, SSR for product pages, CSR for authenticated areas.

106. Angular Change Detection

Checklist: Zone.js · Default vs OnPush · markForCheck · detectChanges · async pipe · Signals · runOutsideAngular · ApplicationRef.tick

106.1 How Zone.js Works

```
// Zone.js monkey-patches ALL async APIs:
// setTimeout, setInterval, Promise.then, addEventListener,
// XMLHttpRequest, fetch, requestAnimationFrame, MutationObserver

// When any async callback fires:
// 1. Zone.js intercepts it
// 2. Notifies Angular via NgZone
// 3. Angular runs change detection on the ENTIRE component tree

// This means EVERY click, timer, HTTP response triggers full tree check!
// For large apps this is a huge performance bottleneck.
```

106.2 Default vs OnPush Strategy

```
// DEFAULT – checks component on EVERY change detection cycle
@Component({
  changeDetection: ChangeDetectionStrategy.Default // ← default
})

// OnPush – only checks when:
// 1. @Input() reference changes (not mutation!)
// 2. Event handler fires within the component
// 3. Async pipe emits a new value
// 4. markForCheck() is called manually
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush // ← use this!
})
export class ProductListComponent {
  @Input() products: Product[] = [];

  // ■ This WON'T trigger change detection with OnPush:
  addProductWrong(p: Product) {
    this.products.push(p); // mutation, same reference!
  }

  // ■ This WILL trigger change detection:
  addProductRight(p: Product) {
    this.products = [...this.products, p]; // new reference!
  }
}

// Rule: With OnPush, ALWAYS treat data as immutable
// Arrays: use spread [...arr, newItem], filter(), map()
// Objects: use spread { ...obj, key: newValue }
```

106.3 Manual Change Detection Controls

```
@Component({
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class DashboardComponent {
  data: any;

  constructor(
    private cdr: ChangeDetectorRef,
    private ngZone: NgZone,
    private ws: WebSocketService
  ) {}

  ngOnInit() {
```

```

    // markForCheck – marks this component + ancestors dirty
    // (checked on next CD cycle)
    this.ws.messages$.subscribe(msg => {
      this.data = msg;
      this.cdr.markForCheck(); // ← tell Angular to check this component
    });

    // detectChanges – runs CD immediately on this component + children
    // Use sparingly! Usually markForCheck is better.
    // this.cdr.detectChanges();

    // detach – remove component from CD tree entirely
    // reattach – add back to CD tree
    this.cdr.detach(); // component won't be checked anymore
    setInterval(() => {
      this.cdr.detectChanges(); // manually check every 5s
    }, 5000);
  }

  // runOutsideAngular – skip Zone.js for perf-heavy work
  startAnimation() {
    this.ngZone.runOutsideAngular(() => {
      // This won't trigger change detection!
      requestAnimationFrame(function animate() {
        // do animation work
        requestAnimationFrame(animate);
      });
    });
  }

  // When you need to come back to Angular zone:
  this.ngZone.run(() => {
    this.status = 'Animation complete';
  });
}
}

```

106.4 Angular Signals (Angular 16+)

```

import { signal, computed, effect } from '@angular/core';

// Signal – reactive primitive, replaces Zone.js for CD
const count = signal(0);
count(); // read: 0
count.set(5); // write: 5
count.update(v => v + 1); // update based on previous: 6

// Computed – derived signal (lazy, memoized)
const doubled = computed(() => count() * 2); // 12
// Only recalculates when count changes

// Effect – side effect that auto-tracks dependencies
effect(() => {
  console.log(`Count is now: ${count()}`);
  // Re-runs whenever count changes
  // Auto-cleanup on component destroy
});

// Signals in components
@Component({
  template: `
    <p>Count: {{ count() }}</p>
    <p>Double: {{ doubled() }}</p>
    <button (click)="increment()">+1</button>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})

```

```

export class CounterComponent {
  count = signal(0);
  doubled = computed(() => this.count() * 2);

  increment() { this.count.update(v => v + 1); }
}

// Signal vs Observable
// Signal:    synchronous, always has a value, simpler API
// Observable: async streams, can be empty, operators for complex flows
// Use Signals for: UI state, computed values, simple reactivity
// Use RxJS for: HTTP, WebSocket, complex async flows, operators

```

■ Interview Questions — Change Detection

Q: Your app with 500 components is sluggish. How do you diagnose and fix?

A: (1) Use Angular DevTools Profiler to see which components re-render and how long. (2) Switch all components to `OnPush` change detection. (3) Use `async` pipe for observables (auto-marks for check). (4) Use `trackBy` for `*ngFor`. (5) Memoize expensive computations with pure pipes or `computed()` signals. (6) Use `runOutsideAngular` for animations/timers that don't affect the UI. (7) Lazy-load feature modules. (8) Use virtual scrolling for long lists.

Q: What's the difference between `markForCheck` and `detectChanges`?

A: `markForCheck()` marks the component and all ancestors as dirty — they'll be checked on the *next* CD cycle. It's safe and non-disruptive. `detectChanges()` runs CD *immediately* on the component and its children, bypassing the normal cycle. It can cause performance issues if overused and can lead to "Expression has changed after it was checked" errors.

107. Angular Dependency Injection

Checklist: Providers · injection tokens · providedIn · hierarchical injectors · useClass/useValue/useFactory/useExisting · forRoot/forChild · tree-shakable providers · inject() function

```
// Provider strategies
@Injectable({
  providedIn: 'root' // tree-shakable singleton (recommended!)
})
export class AuthService { }

// providedIn options:
// 'root'      - app-wide singleton
// 'any'       - unique instance per lazy-loaded module
// 'platform'  - shared across all apps on the page (micro-frontends)
// SomeModule - singleton within that module's injector

// Provider types
providers: [
  // useClass - create instance of different class
  { provide: LoggerService, useClass: BetterLoggerService },

  // useValue - provide a static value
  { provide: API_URL, useValue: 'https://api.example.com' },

  // useFactory - compute at runtime
  {
    provide: DataService,
    useFactory: (http: HttpClient, config: ConfigService) => {
      return config.useMock
        ? new MockDataService()
        : new RealDataService(http);
    },
    deps: [HttpClient, ConfigService]
  },

  // useExisting - alias to another provider
  { provide: AbstractLogger, useExisting: ConsoleLoggerService }
]

// InjectionToken - type-safe tokens for non-class dependencies
export const API_CONFIG = new InjectionToken<ApiConfig>('api.config', {
  providedIn: 'root',
  factory: () => ({ baseUrl: '/api', timeout: 5000 })
});

// inject() function (Angular 14+) - replaces constructor injection
@Component({ ... })
export class UserComponent {
  private http = inject(HttpClient);
  private config = inject(API_CONFIG);
  private router = inject(Router);
  private user = inject(UserService, { optional: true }); // nullable

  // No constructor needed!
}

// Hierarchical injectors - child overrides parent
@Component({
  providers: [{ provide: LoggerService, useClass: VerboseLogger }]
  // This component and its children get VerboseLogger
  // Rest of the app still uses default LoggerService
})
```

108. Angular Lifecycle Hooks

Checklist: constructor · ngOnChanges · ngOnInit · ngDoCheck · ngAfterContentInit/Checked · ngAfterViewInit/Checked · ngOnDestroy · execution order · parent-child order

```
@Component({ ... })
export class LifecycleComponent implements
  OnChanges, OnInit, DoCheck,
  AfterContentInit, AfterContentChecked,
  AfterViewInit, AfterViewChecked, OnDestroy {

  @Input() data: any;

  // 1. constructor – DI happens here, inputs NOT available yet
  constructor() { }

  // 2. ngOnChanges – called BEFORE ngOnInit and every @Input change
  //    Only for @Input() properties. Receives SimpleChanges object.
  ngOnChanges(changes: SimpleChanges) {
    if (changes['data'] && !changes['data'].firstChange) {
      console.log('data changed:', changes['data'].previousValue,
        '→', changes['data'].currentValue);
    }
  }

  // 3. ngOnInit – component initialized. Inputs ARE available.
  //    Called ONCE. Best place for initialization logic.
  ngOnInit() {
    this.loadData(); // ■ Do API calls here, not in constructor
  }

  // 4. ngDoCheck – custom change detection. Called EVERY CD cycle!
  //    Use sparingly – runs very frequently.
  ngDoCheck() { }

  // 5. ngAfterContentInit – projected content (ng-content) initialized
  //    Called ONCE after first ngDoCheck
  ngAfterContentInit() { }

  // 6. ngAfterContentChecked – after every check of projected content
  ngAfterContentChecked() { }

  // 7. ngAfterViewInit – view (template + child views) initialized
  //    @ViewChild is available here, NOT in ngOnInit!
  @ViewChild('chart') chartRef: ElementRef;
  ngAfterViewInit() {
    this.initChart(this.chartRef.nativeElement); // ■ safe here
  }

  // 8. ngAfterViewChecked – after every check of the view
  ngAfterViewChecked() { }

  // 9. ngOnDestroy – cleanup! Unsubscribe, detach listeners
  private destroy$ = new Subject<void>();
  ngOnDestroy() {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

// Execution order (parent P, child C):
// P.constructor → P.ngOnChanges → P.ngOnInit → P.ngDoCheck
// → C.constructor → C.ngOnChanges → C.ngOnInit → C.ngDoCheck
// → C.ngAfterContentInit → C.ngAfterContentChecked
// → C.ngAfterViewInit → C.ngAfterViewChecked
// → P.ngAfterContentInit → P.ngAfterContentChecked
```

```
// → P.ngAfterViewInit → P.ngAfterViewChecked
```

109. RxJS Deep Dive (Operators)

Checklist: Observables · Subjects · switchMap · mergeMap · concatMap · exhaustMap · combineLatest · forkJoin · takeUntil · shareReplay · debounceTime · distinctUntilChanged · catchError · retry · tap · BehaviorSubject · ReplaySubject

109.1 Observable vs Promise

```
// Observable: lazy, cancellable, multiple values, operators
// Promise: eager, not cancellable, single value
```

```
const obs$ = new Observable(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  setTimeout(() => subscriber.next(3), 1000);
  // subscriber.complete();
  // subscriber.error(new Error('fail'));
});
```

```
const sub = obs$.subscribe({
  next: val => console.log(val),
  error: err => console.error(err),
  complete: () => console.log('done')
});
```

```
sub.unsubscribe(); // ← cancel! Promises can't do this
```

109.2 Subjects — Observable + Observer

```
// Subject – multicast, no initial value
const subject = new Subject<number>();
subject.subscribe(v => console.log('A:', v));
subject.next(1); // A: 1
subject.subscribe(v => console.log('B:', v));
subject.next(2); // A: 2, B: 2 (B missed value 1!)
```

```
// BehaviorSubject – has initial value, emits last value to new subscribers
const behavior$ = new BehaviorSubject<string>('initial');
behavior$.subscribe(v => console.log('A:', v)); // A: initial
behavior$.next('hello'); // A: hello
behavior$.subscribe(v => console.log('B:', v)); // B: hello (gets last value!)
behavior$.getValue(); // 'hello' – synchronous read
```

```
// ReplaySubject – replays N last values to new subscribers
const replay$ = new ReplaySubject<number>(3); // buffer last 3
replay$.next(1); replay$.next(2); replay$.next(3); replay$.next(4);
replay$.subscribe(v => console.log(v)); // 2, 3, 4
```

```
// AsyncSubject – only emits the LAST value, and only on complete()
const async$ = new AsyncSubject<number>();
async$.next(1); async$.next(2); async$.next(3);
async$.subscribe(v => console.log(v)); // nothing yet...
async$.complete(); // NOW logs: 3
```

109.3 Flattening Operators — THE Most Asked Topic

```
// switchMap – cancel previous, use latest only
// Use for: search/typeahead, route params, latest value
this.searchInput$.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(query => this.api.search(query))
  // If user types again before response, previous HTTP is CANCELLED
).subscribe(results => this.results = results);
```

```
// mergeMap (flatMap) – all run in parallel, no cancellation
// Use for: parallel independent requests, logging, analytics
```

```

this.items$.pipe(
  mergeMap(item => this.api.process(item), 3) // max 3 concurrent
).subscribe();

// concatMap – sequential, one at a time, preserves order
// Use for: ordered operations, file uploads one-by-one
this.files$.pipe(
  concatMap(file => this.api.upload(file))
  // Waits for each upload to complete before starting next
).subscribe();

// exhaustMap – ignore new emissions while current is running
// Use for: login button, form submit (prevent double-click!)
this.loginClick$.pipe(
  exhaustMap(() => this.auth.login(this.credentials))
  // Clicking again while login is in-flight does NOTHING
).subscribe();

// SUMMARY TABLE:
// switchMap → cancel previous → typeahead, route params
// mergeMap → all parallel → parallel API calls
// concatMap → one by one → ordered operations
// exhaustMap → ignore new → prevent double-submit

```

109.4 Combination Operators

```

// combineLatest – emit when ANY source emits (needs all to emit once first)
combineLatest([
  this.route.params,
  this.route.queryParams,
  this.store.select(selectFilters)
]).pipe(
  map(([params, query, filters]) => ({ ...params, ...query, ...filters }))
).subscribe(combined => this.loadData(combined));

// forkJoin – emit when ALL complete (like Promise.all)
forkJoin({
  user: this.api.getUser(id),
  orders: this.api.getOrders(id),
  settings: this.api.getSettings(id)
}).subscribe(({ user, orders, settings }) => {
  // All three responses available here
});
// ■■■ If ANY fails, entire forkJoin fails! Use catchError inside each.

// withLatestFrom – get latest value of another observable
this.saveClick$.pipe(
  withLatestFrom(this.form.valueChanges),
  map([click, formValue]) => formValue),
  exhaustMap(value => this.api.save(value))
).subscribe();

// merge – combine multiple into one (interleaved)
merge(this.ws.messages$, this.polling$, this.push$).subscribe();

// zip – pair values by index (nth from each source)
zip(names$, scores$).subscribe(([name, score]) => {});

```

109.5 Error Handling & Retry

```

// catchError – handle errors, return fallback observable
this.api.getData().pipe(
  catchError(error => {
    this.logger.log(error);
    return of([]); // fallback: empty array
    // or: return EMPTY; (complete without emitting)
    // or: return throwError(() => new Error('Custom msg'));
  })
)

```



```

    })
  ).subscribe();

// retry – retry N times
this.api.getData().pipe(
  retry(3, // retry up to 3 times
    catchError(err => of([])) // after 3 failures, fallback
  ).subscribe();

// retryWhen with delay (exponential backoff)
this.api.getData().pipe(
  retry({
    count: 3,
    delay: (error, retryCount) => {
      const delay = Math.pow(2, retryCount) * 1000;
      console.log(`Retry ${retryCount} in ${delay}ms`);
      return timer(delay);
    }
  })
).subscribe();

```

109.6 Memory Leak Prevention

```

// ■ BAD – manual subscribe without cleanup → MEMORY LEAK
ngOnInit() {
  this.userService.user$.subscribe(user => this.user = user);
  // This subscription lives FOREVER after component is destroyed!
}

// ■ Method 1: takeUntil pattern
private destroy$ = new Subject<void>();
ngOnInit() {
  this.userService.user$.pipe(
    takeUntil(this.destroy$)
  ).subscribe(user => this.user = user);
}
ngOnDestroy() {
  this.destroy$.next();
  this.destroy$.complete();
}

// ■ Method 2: async pipe (BEST – auto-subscribes + unsubscribes)
@Component({
  template: `
    <div *ngIf="user$ | async as user">
      Welcome, {{ user.name }}
    </div>
  `
})
export class UserComponent {
  user$ = this.userService.user$; // no subscribe needed!
}

// ■ Method 3: takeUntilDestroyed (Angular 16+)
export class UserComponent {
  constructor() {
    this.userService.user$.pipe(
      takeUntilDestroyed() // auto-unsubscribe on destroy!
    ).subscribe(user => this.user = user);
  }
}

// ■ Method 4: DestroyRef (Angular 16+)
export class UserComponent {
  private destroyRef = inject(DestroyRef);

  ngOnInit() {

```

```

    const sub = this.data$.subscribe(d => this.data = d);
    this.destroyRef.onDestroy(() => sub.unsubscribe());
  }
}

```

■ Interview Questions — RxJS

Q: You have a search input. User types fast. How do you optimize API calls?

A: `searchInput.valueChanges.pipe(debounceTime(300), distinctUntilChanged(), filter(q => q.length >= 2), switchMap(q => api.search(q).pipe(catchError(() => of([]))))).debounceTime` waits for pause. `distinctUntilChanged` skips duplicate queries. `switchMap` cancels previous HTTP if new keystroke arrives. `catchError` inside `switchMap` prevents the outer observable from dying on error.

Q: When would you use `mergeMap` over `switchMap`?

A: Use `mergeMap` when you need ALL results (parallel, order doesn't matter) — e.g., sending analytics events, processing a batch of items. Use `switchMap` when only the LATEST matters and older results are stale — e.g., typeahead search, route param changes. Use `concatMap` when order matters — e.g., sequential file uploads. Use `exhaustMap` to prevent duplicate work — e.g., login/submit buttons.

110. Angular Router

Checklist: Route config · lazy loading · guards (canActivate, canDeactivate, resolve, canMatch) · route params/query params · child/nested routes · router events · preloading strategies

```
// Route configuration
const routes: Routes = [
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },

  // Eager route with guard
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [authGuard],
    resolve: { stats: dashboardResolver }
  },

  // Lazy-loaded with functional guard (modern approach)
  {
    path: 'products',
    loadChildren: () => import('./products/routes').then(m => m.PRODUCT_ROUTES),
    canMatch: [() => inject(AuthService).isAuthenticated()]
  },

  // Lazy-load standalone component
  {
    path: 'profile',
    loadComponent: () => import('./profile.component').then(c => c.ProfileComponent)
  },

  // Nested routes
  {
    path: 'admin',
    component: AdminLayoutComponent,
    canActivate: [authGuard, roleGuard('admin')],
    children: [
      { path: '', redirectTo: 'users', pathMatch: 'full' },
      { path: 'users', component: UserManagementComponent },
      { path: 'users/:id', component: UserDetailComponent },
      { path: 'settings', component: AdminSettingsComponent }
    ]
  },

  // Wildcard – must be last!
  { path: '**', component: NotFoundComponent }
];

// Functional guards (Angular 15+) – replace class-based guards
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isLoggedIn()) return true;
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: state.url } });
};

// Role guard factory
export function roleGuard(requiredRole: string): CanActivateFn {
  return () => {
    const auth = inject(AuthService);
    return auth.hasRole(requiredRole) || inject(Router).createUrlTree(['/forbidden']);
  };
}

// Resolver – fetch data before route activates
export const dashboardResolver: ResolveFn<DashboardStats> = () => {
  return inject(DashboardService).getStats().pipe(
```

```

        catchError(() => {
            inject(Router).navigate(['/error']);
            return EMPTY;
        })
    );
};

// canDeactivate – warn on unsaved changes
export const unsavedChangesGuard: CanDeactivateFn<{ hasUnsavedChanges: () => boolean }> =
    (component) => {
        if (component.hasUnsavedChanges()) {
            return confirm('You have unsaved changes. Leave anyway?');
        }
        return true;
    };

// Preloading strategies
RouterModule.forRoot(routes, {
    preloadingStrategy: PreloadAllModules // preload ALL lazy modules after initial load

    // Or: QuicklinkStrategy – preload only visible links (via quicklink library)
})

```

111. Angular Forms (Reactive vs Template)

Checklist: FormBuilder · FormGroup/FormControl/FormArray · validators · custom validators · async validators · dynamic forms · template-driven vs reactive · valueChanges · patchValue/setValue · strong typing

```
// REACTIVE FORMS – recommended for complex forms

// Strongly-typed form (Angular 14+)
interface LoginForm {
  email: FormControl<string>;
  password: FormControl<string>;
  rememberMe: FormControl<boolean>;
}

@Component({
  template: `
    <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
      <input formControlName="email" />
      <div *ngIf="loginForm.controls.email.errors?.['required']">
        Email is required
      </div>
      <div *ngIf="loginForm.controls.email.errors?.['email']">
        Invalid email format
      </div>
      <div *ngIf="loginForm.controls.email.errors?.['emailTaken']">
        Email already registered
      </div>

      <input formControlName="password" type="password" />
      <label>
        <input formControlName="rememberMe" type="checkbox" />
        Remember me
      </label>
      <button [disabled]="loginForm.invalid">Login</button>
    </form>
  `
})
export class LoginComponent {
  loginForm = new FormGroup<LoginForm>({
    email: new FormControl('', {
      nullable: true,
      validators: [Validators.required, Validators.email],
      asyncValidators: [this.emailExistsValidator()]
    }),
    password: new FormControl('', {
      nullable: true,
      validators: [Validators.required, Validators.minLength(8)]
    }),
    rememberMe: new FormControl(false, { nullable: true })
  });

  // Custom sync validator
  static passwordStrength(control: AbstractControl): ValidationErrors | null {
    const value = control.value;
    const hasUpper = /[A-Z]/.test(value);
    const hasLower = /[a-z]/.test(value);
    const hasDigit = /\d/.test(value);
    const hasSpecial = /[!@#$$%^&*]/.test(value);
    const valid = hasUpper && hasLower && hasDigit && hasSpecial;
    return valid ? null : { weakPassword: true };
  }

  // Custom async validator – check if email exists on server
  emailExistsValidator(): AsyncValidatorFn {
    return (control: AbstractControl): Observable<ValidationErrors | null> => {
      return this.authService.checkEmail(control.value).pipe(

```

```

        map(exists => exists ? { emailTaken: true } : null),
        catchError(() => of(null))
    );
};
}

// Cross-field validator (password confirmation)
static passwordMatch(group: AbstractControl): ValidationErrors | null {
    const password = group.get('password')?.value;
    const confirm = group.get('confirmPassword')?.value;
    return password === confirm ? null : { passwordMismatch: true };
}

// FormArray – dynamic number of fields (e.g., add skills)
skills = new FormArray<FormControl<string>>([]);
addSkill() { this.skills.push(new FormControl('', { nullable: true })); }
removeSkill(i: number) { this.skills.removeAt(i); }

onSubmit() {
    if (this.loginForm.valid) {
        const value = this.loginForm.getRawValue(); // typed!
        // value.email: string, value.password: string, value.rememberMe: boolean
    }
}
}

```

112. Angular HTTP & Interceptors

Checklist: HttpClient · typed responses · interceptors (class & functional) · error handling · caching · retry · request/response transformation · XSRF protection

```
// HttpClient basics
@Injectable({ providedIn: 'root' })
export class ProductService {
  private url = '/api/products';

  constructor(private http: HttpClient) {}

  getAll(): Observable<Product[]> {
    return this.http.get<Product[]>(this.url);
  }

  getById(id: string): Observable<Product> {
    return this.http.get<Product>(`${this.url}/${id}`);
  }

  create(product: CreateProductDto): Observable<Product> {
    return this.http.post<Product>(this.url, product);
  }

  update(id: string, product: Partial<Product>): Observable<Product> {
    return this.http.patch<Product>(`${this.url}/${id}`, product);
  }

  delete(id: string): Observable<void> {
    return this.http.delete<void>(`${this.url}/${id}`);
  }

  search(params: SearchParams): Observable<PaginatedResponse<Product>> {
    return this.http.get<PaginatedResponse<Product>>(this.url, {
      params: new HttpParams()
        .set('query', params.query)
        .set('page', params.page.toString())
        .set('limit', params.limit.toString())
    });
  }
}

// Functional interceptor (Angular 15+) – MODERN APPROACH
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const token = auth.getToken();

  if (token) {
    req = req.clone({
      headers: { Authorization: `Bearer ${token}` }
    });
  }
  return next(req);
};

// Error interceptor with refresh token
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      if (error.status === 401) {
        return auth.refreshToken().pipe(
          switchMap(newToken => {
            const cloned = req.clone({

```

```

        setHeaders: { Authorization: `Bearer ${newToken}` }
    });
    return next(cloned);
  }},
  catchError(() => {
    auth.logout();
    router.navigate(['/login']);
    return throwError(() => error);
  })
);
}
return throwError(() => error);
})
);
};

// Loading interceptor
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loading = inject(LoadingService);

  loading.show();
  return next(req).pipe(finally(() => loading.hide()));
};

// Register interceptors
export const appConfig: ApplicationConfig = {
  providers: [
    provideHttpClient(
      withInterceptors([authInterceptor, errorInterceptor, loadingInterceptor])
    )
  ]
};

```


113. Angular Standalone Components & Modern APIs

Checklist: Standalone components · bootstrapping without NgModule · provideRouter · provideHttpClient · input/output/model signals · viewChild/contentChild signals · deferrable views · control flow (@if, @for, @switch)

```
// Standalone component – no NgModule needed!
@Component({
  standalone: true,
  selector: 'app-product-card',
  imports: [CommonModule, RouterLink], // import what you need
  template: `
    <!-- New control flow (Angular 17+) -->
    @if (product) {
      <div class="card">
        <h3>{{ product.name }}</h3>
        <p>{{ product.price | currency }}</p>
        @switch (product.status) {
          @case ('active') { <span class="badge green">Active</span> }
          @case ('draft') { <span class="badge gray">Draft</span> }
          @default { <span class="badge">Unknown</span> }
        }
      </div>
    } @else {
      <p>Loading...</p>
    }
  `
})
export class ProductCardComponent {
  // Signal-based inputs (Angular 17.1+)
  product = input.required<Product>();
  highlighted = input(false); // optional with default

  // Signal-based output
  addToCart = output<Product>();

  // model() – two-way binding signal (Angular 17.2+)
  quantity = model(1);
}

// Usage:
// <app-product-card [product]="p" [(quantity)]="qty"
//   (addToCart)="onAdd($event)" />

// Deferrable views – lazy-load heavy components
@Component({
  template: `
    @defer (on viewport) {
      <app-heavy-chart [data]="chartData" />
    } @placeholder {
      <div class="skeleton">Loading chart...</div>
    } @loading (minimum 500ms) {
      <app-spinner />
    } @error {
      <p>Failed to load chart.</p>
    }
  `
})
// Triggers: on viewport, on idle, on interaction, on hover, on timer(5s), when condition

// Bootstrapping without NgModule
// main.ts
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimations(),
  ]
})
```

```
    { provide: ErrorHandler, useClass: GlobalErrorHandler }  
  ]  
});
```

114. Angular Pipes & Directives

Checklist: Built-in pipes · pure vs impure pipes · custom pipes · structural directives · attribute directives · host binding · host listener

```
// PURE PIPE – only re-evaluates when input value OR reference changes
// Angular caches the result. Much better for performance.
@Pipe({ name: 'filterBy', standalone: true, pure: true })
export class FilterByPipe implements PipeTransform {
  transform(items: any[], field: string, value: string): any[] {
    if (!items || !value) return items;
    return items.filter(item =>
      item[field].toLowerCase().includes(value.toLowerCase())
    );
  }
}

// ■ Use with OnPush + immutable data: products = [...products, newProduct]
// Usage: *ngFor="let p of products | filterBy:'name':searchTerm"

// IMPURE PIPE – re-evaluates on EVERY change detection cycle!
@Pipe({ name: 'filterBy', pure: false })
// ■ Avoid! Runs dozens of times per second. Use computed() or component logic instead.

// CUSTOM DIRECTIVE – attribute directive
@Directive({
  selector: '[appTooltip]',
  standalone: true
})
export class TooltipDirective {
  @Input('appTooltip') text = '';
  private tooltipEl: HTMLElement | null = null;

  @HostListener('mouseenter')
  show() {
    this.tooltipEl = document.createElement('div');
    this.tooltipEl.className = 'tooltip';
    this.tooltipEl.textContent = this.text;
    document.body.appendChild(this.tooltipEl);
    // position near host element...
  }

  @HostListener('mouseleave')
  hide() {
    this.tooltipEl?.remove();
    this.tooltipEl = null;
  }
}

// STRUCTURAL DIRECTIVE – modifies DOM structure
@Directive({ selector: '[appRepeat]', standalone: true })
export class RepeatDirective {
  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) {}

  @Input() set appRepeat(times: number) {
    this.viewContainer.clear();
    for (let i = 0; i < times; i++) {
      this.viewContainer.createEmbeddedView(this.templateRef, { index: i });
    }
  }
}

// Usage: <div *appRepeat="5">Repeated 5 times</div>
```

115. Content Projection & Templates

Checklist: ng-content · select attribute · multi-slot projection · ng-template · ng-container · ngTemplateOutlet · dynamic components

```
// Single-slot content projection
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content></ng-content>
    </div>
  `
})
// Usage: <app-card>Any content here!</app-card>

// Multi-slot projection – powerful composition pattern!
@Component({
  selector: 'app-dialog',
  template: `
    <div class="dialog">
      <header><ng-content select="[dialog-title]"></ng-content></header>
      <main><ng-content></ng-content></main>
      <footer><ng-content select="[dialog-actions]"></ng-content></footer>
    </div>
  `
})
// Usage:
// <app-dialog>
//   <h2 dialog-title>Confirm Delete</h2>
//   <p>Are you sure you want to delete this item?</p>
//   <div dialog-actions>
//     <button (click)="cancel()">Cancel</button>
//     <button (click)="confirm()">Delete</button>
//   </div>
// </app-dialog>

// ng-template + ngTemplateOutlet – render templates dynamically
@Component({
  template: `
    <app-data-table [data]="users" [columns]="columns">
      <!-- Custom cell template -->
      <ng-template #statusTpl let-row>
        <span [class]="'badge ' + row.status">{{ row.status }}</span>
      </ng-template>
      <ng-template #actionsTpl let-row>
        <button (click)="edit(row)">Edit</button>
        <button (click)="delete(row)">Delete</button>
      </ng-template>
    </app-data-table>
  `
})

// Dynamic component loading (Angular 13+)
@Component({
  template: `<ng-container #host></ng-container>`
})
export class DynamicLoaderComponent {
  @ViewChild('host', { read: ViewContainerRef }) host!: ViewContainerRef;

  async loadWidget(type: string) {
    this.host.clear();
    const component = await this.getComponent(type);
    const ref = this.host.createComponent(component);
    ref.instance.data = this.widgetData;
  }
}
```

```
private async getComponent(type: string) {  
  switch (type) {  
    case 'chart': return (await import('./chart.component')).ChartComponent;  
    case 'table': return (await import('./table.component')).TableComponent;  
    default: throw new Error(`Unknown widget: ${type}`);  
  }  
}
```

116. Angular Performance Optimization

Checklist: trackBy · OnPush · pure pipes · lazy loading · @defer · virtual scrolling · bundle size · source map analysis · tree shaking · zone-less Angular

```
// PERFORMANCE CHECKLIST – apply all of these for production apps

// 1. trackBy for ngFor – prevents re-creating DOM elements
@for (product of products; track product.id) {
  <app-product-card [product]="product" />
}
// Without trackBy: entire list re-renders on data change
// With trackBy: only changed items re-render

// 2. OnPush everywhere + immutable data patterns
// Already covered in section 106

// 3. Virtual scrolling for large lists
import { ScrollingModule } from '@angular/cdk/scrolling';

<cdk-virtual-scroll-viewport itemSize="72" class="list-viewport">
  <div *cdkVirtualFor="let item of items; trackBy: trackById">
    {{ item.name }}
  </div>
</cdk-virtual-scroll-viewport>
// Only renders visible items! 10,000 items? Only ~20 DOM nodes.

// 4. Lazy loading (already covered in routing)
// 5. @defer for heavy components (covered in section 113)

// 6. Bundle analysis
// ng build --stats-json
// npx webpack-bundle-analyzer dist/your-app/stats.json
// Look for: large dependencies, duplicate code, unused imports

// 7. Angular budgets in angular.json
"budgets": [
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "1mb" },
  { "type": "anyComponentStyle", "maximumWarning": "4kb", "maximumError": "8kb" }
]

// 8. Preloading strategies
// PreloadAllModules – load all lazy chunks after initial load
// Custom: preload based on user role, viewport links, or prediction

// 9. Image optimization
<img ngSrc="hero.jpg" width="800" height="400" priority />
// NgOptimizedImage: automatic lazy loading, srcset, priority hints

// 10. Zone-less Angular (bleeding edge)
bootstrapApplication(AppComponent, {
  providers: [provideExperimentalZonelessChangeDetection()]
});
// Requires all components to use Signals or manual CD
```

■ Interview Questions — Angular Performance

Q: Your Angular app takes 8 seconds to load. Walk me through your optimization approach.

A: (1) Measure: Lighthouse audit, performance.getEntries(), bundle analyzer. (2) **Code splitting:** lazy-load routes, @defer for below-fold components. (3) **Bundle size:** remove unused imports, replace moment.js with date-fns, tree-shake lodash. (4) **Loading:** preload critical fonts, preconnect to API, SSR for FCP. (5) **Images:** NgOptimizedImage, WebP/AVIF, responsive srcset. (6) **Caching:** service worker, HTTP cache headers, stale-while-revalidate. (7) **Runtime:** OnPush everywhere, virtual scroll for lists, trackBy. (8) Set budgets in CI to prevent regression.

117. Angular Testing

Checklist: TestBed · ComponentFixture · provide mock services · fakeAsync/tick/flush · HttpClientTestingModule · RouterTestingModule · component harness · integration tests

```
// Component test with TestBed
describe('ProductListComponent', () => {
  let component: ProductListComponent;
  let fixture: ComponentFixture<ProductListComponent>;
  let mockProductService: jasmine.SpyObj<ProductService>;

  beforeEach(async () => {
    mockProductService = jasmine.createSpyObj('ProductService', ['getAll', 'delete']);
    mockProductService.getAll.and.returnValue(of([
      { id: '1', name: 'Laptop', price: 1200 },
      { id: '2', name: 'Phone', price: 800 }
    ]));

    await TestBed.configureTestingModule({
      imports: [ProductListComponent], // standalone component
      providers: [
        { provide: ProductService, useValue: mockProductService }
      ]
    }).compileComponents();

    fixture = TestBed.createComponent(ProductListComponent);
    component = fixture.componentInstance;
    fixture.detectChanges(); // triggers ngOnInit
  });

  it('should display products', () => {
    const cards = fixture.nativeElement.querySelectorAll('.product-card');
    expect(cards.length).toBe(2);
    expect(cards[0].textContent).toContain('Laptop');
  });

  it('should call delete and refresh list', fakeAsync(() => {
    mockProductService.delete.and.returnValue(of(void 0));
    mockProductService.getAll.and.returnValue(of([
      { id: '2', name: 'Phone', price: 800 }
    ]));

    component.deleteProduct('1');
    tick(); // resolve the observable

    expect(mockProductService.delete).toHaveBeenCalledWith('1');
    fixture.detectChanges();
    const cards = fixture.nativeElement.querySelectorAll('.product-card');
    expect(cards.length).toBe(1);
  }));
});

// HTTP testing
describe('ProductService', () => {
  let service: ProductService;
  let httpMock: HttpClientTestingModule;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [
        ProductService,
        provideHttpClient(),
        provideHttpClientTesting()
      ]
    });
    service = TestBed.inject(ProductService);
  });
```

```
    httpMock = TestBed.inject(HttpTestingController);
  });

  afterEach(() => httpMock.verify()); // no outstanding requests

  it('should fetch products', () => {
    const mockProducts = [{ id: '1', name: 'Laptop' }];
    service.getAll().subscribe(products => {
      expect(products).toEqual(mockProducts);
    });
    const req = httpMock.expectOne('/api/products');
    expect(req.request.method).toBe('GET');
    req.flush(mockProducts);
  });
});
```


118. State Management with NgRx

Checklist: Store · Actions · Reducers · Selectors · Effects · Entity adapter · Component Store · SignalStore

```
// NgRx – Redux pattern for Angular

// 1. ACTIONS – describe events
export const ProductActions = createActionGroup({
  source: 'Products',
  events: {
    'Load Products': emptyProps(),
    'Load Products Success': props<{ products: Product[] }>(),
    'Load Products Failure': props<{ error: string }>(),
    'Delete Product': props<{ id: string }>(),
    'Delete Product Success': props<{ id: string }>(),
  }
});

// 2. REDUCER – how state changes
interface ProductState {
  products: Product[];
  loading: boolean;
  error: string | null;
}

const initialState: ProductState = { products: [], loading: false, error: null };

export const productReducer = createReducer(
  initialState,
  on(ProductActions.loadProducts, state => ({ ...state, loading: true, error: null })),
  on(ProductActions.loadProductsSuccess, (state, { products }) =>
    ({ ...state, products, loading: false })),
  on(ProductActions.loadProductsFailure, (state, { error }) =>
    ({ ...state, loading: false, error })),
  on(ProductActions.deleteProductSuccess, (state, { id }) =>
    ({ ...state, products: state.products.filter(p => p.id !== id) })),
);

// 3. SELECTORS – derive data from state (memoized!)
export const selectProductState = createFeatureSelector<ProductState>('products');
export const selectAllProducts = createSelector(selectProductState, s => s.products);
export const selectLoading = createSelector(selectProductState, s => s.loading);
export const selectExpensiveProducts = createSelector(
  selectAllProducts,
  products => products.filter(p => p.price > 1000)
);

// 4. EFFECTS – side effects (API calls)
@Injectable()
export class ProductEffects {
  loadProducts$ = createEffect(() =>
    this.actions$.pipe(
      ofType(ProductActions.loadProducts),
      exhaustMap(() =>
        this.productService.getAll().pipe(
          map(products => ProductActions.loadProductsSuccess({ products })),
          catchError(error =>
            of(ProductActions.loadProductsFailure({ error: error.message })))
        )
      )
    )
  );

  constructor(private actions$: Actions, private productService: ProductService) {}
}
```

```

// 5. COMPONENT – use store
@Component({
  template: `
    @if (loading$ | async) { <app-spinner /> }
    @for (product of products$ | async; track product.id) {
      <app-product-card [product]="product" (delete)="onDelete($event)" />
    }
  `
})
export class ProductListComponent {
  products$ = this.store.select(selectAllProducts);
  loading$ = this.store.select(selectLoading);

  constructor(private store: Store) {
    this.store.dispatch(ProductActions.loadProducts());
  }

  onDelete(id: string) {
    this.store.dispatch(ProductActions.deleteProduct({ id }));
  }
}

// NgRx SignalStore (modern alternative) – simpler, signal-based
export const ProductStore = signalStore(
  withState({ products: [] as Product[], loading: false }),
  withComputed(({ products }) => ({
    expensiveProducts: computed(() => products().filter(p => p.price > 1000)),
    total: computed(() => products().length)
  })),
  withMethods((store, productService = inject(ProductService)) => ({
    async loadProducts() {
      patchState(store, { loading: true });
      const products = await firstValueFrom(productService.getAll());
      patchState(store, { products, loading: false });
    }
  })))
);

```

119. Angular Best Practices & Patterns

Checklist: Smart vs dumb components · service layer patterns · error handling strategy · environment configs · barrel exports · feature modules · shared modules

```
// SMART vs DUMB components (Container vs Presentational)

// SMART (Container) – knows about services, state, business logic
@Component({
  template: `
    <app-product-filters [categories]="categories"
      (filterChange)="onFilter($event)" />
    <app-product-grid [products]="filteredProducts$ | async"
      (addToCart)="onAddToCart($event)" />
  `
})
export class ProductPageComponent {
  // Handles: data fetching, state management, business logic
  private store = inject(ProductStore);
  filteredProducts$ = this.store.filteredProducts$;
}

// DUMB (Presentational) – only @Input and @Output, no injected services
@Component({
  template: `
    @for (product of products; track product.id) {
      <div class="card" (click)="addToCart.emit(product)">
        {{ product.name }} – {{ product.price | currency }}
      </div>
    }
  `
  ,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ProductGridComponent {
  @Input() products: Product[] = [];
  @Output() addToCart = new EventEmitter<Product>();
  // Easy to test, reusable, predictable
}

// SERVICE LAYER PATTERN
// API Service → Domain Service → Component
@Injectable({ providedIn: 'root' })
export class OrderApiService {
  // Only HTTP calls, no business logic
  getOrders = () => this.http.get<Order[]>('/api/orders');
}

@Injectable({ providedIn: 'root' })
export class OrderService {
  // Business logic, caching, transformation
  private cache = new Map<string, Order>();

  getOrderWithTotal(id: string): Observable<OrderWithTotal> {
    return this.api.getOrder(id).pipe(
      map(order => ({
        ...order,
        total: order.items.reduce((sum, i) => sum + i.price * i.quantity, 0),
        tax: this.calculateTax(order),
        status: this.mapStatus(order.rawStatus)
      })),
      tap(order => this.cache.set(id, order))
    );
  }
}

// ENVIRONMENT-BASED FEATURE FLAGS
```

```
@Injectable({ providedIn: 'root' })
export class FeatureFlagService {
  private flags = signal(environment.featureFlags);

  isEnabled(flag: string): boolean {
    return this.flags()[flag] ?? false;
  }
}

// Usage in template:
// @if (featureFlags.isEnabled('newCheckout')) {
//   <app-new-checkout />
// } @else {
//   <app-legacy-checkout />
// }
```

120. Common Angular Mistakes & Fixes

Checklist: Memory leaks · ExpressionChangedAfterItHasBeenChecked · circular dependencies · over-subscribing · template performance · AOT issues

```
// MISTAKE 1: Memory leaks from subscriptions
// ■
ngOnInit() { this.dataService.data$.subscribe(d => this.data = d); }
// ■
ngOnInit() {
  this.dataService.data$.pipe(takeUntilDestroyed()).subscribe(d => this.data = d);
}
// ■ BEST: use async pipe in template
// {{ data$ | async }}
```



```
// MISTAKE 2: ExpressionChangedAfterItHasBeenChecked
// Happens when you change state DURING change detection
// ■ Changing parent state in ngAfterViewInit of child
ngAfterViewInit() {
  this.parentService.title = 'New Title'; // ERROR in dev mode!
}
// ■ Fix: defer the change
ngAfterViewInit() {
  setTimeout(() => this.parentService.title = 'New Title');
  // OR: this.cdr.detectChanges();
}
```



```
// MISTAKE 3: Not using trackBy with ngFor
// ■ Entire list re-renders every time
<div *ngFor="let item of items">{{ item.name }}</div>
// ■
<div *ngFor="let item of items; trackBy: trackById">{{ item.name }}</div>
trackById = (index: number, item: any) => item.id;
// ■ Angular 17 @for:
// @for (item of items; track item.id) { ... }
```



```
// MISTAKE 4: Function calls in templates
// ■ getFullName() called on EVERY change detection cycle!
<p>{{ getFullName() }}</p>
// ■ Use a pipe
<p>{{ user | fullName }}</p>
// ■ Or compute in component
fullName = computed(() => `${this.user().first} ${this.user().last}`);
```



```
// MISTAKE 5: Importing entire libraries
// ■ Imports ALL of lodash (70KB gzipped!)
import _ from 'lodash';
// ■ Import only what you need (2KB)
import debounce from 'lodash-es/debounce';
// ■ Or use native
const unique = [...new Set(arr)]; // no lodash needed
```



```
// MISTAKE 6: Not handling HTTP errors
// ■ Error kills the observable stream
this.api.getData().subscribe(data => this.data = data);
// ■
this.api.getData().pipe(
  catchError(error => {
    this.errorService.handle(error);
    return of(fallbackData);
  })
).subscribe(data => this.data = data);
```



```
// MISTAKE 7: Circular dependencies
// Module A imports Module B, Module B imports Module A → ERROR
// ■ Fix: extract shared code into a third module
```

// ■ Fix: use injection tokens instead of direct imports

■ Interview Questions — Angular Architecture

Q: How would you migrate a large Angular app from Zone.js to Signals?

A: Gradual migration, not big-bang. (1) Start by adding `OnPush` to all components (prerequisite). (2) Replace simple component state with `signal()` and `computed()`. (3) Replace `BehaviorSubjects` in services with signals where appropriate. (4) Use `toSignal()` to convert observables to signals at boundaries. (5) Keep RxJS for complex async flows (HTTP, WebSocket, operators). (6) Test each module independently. (7) Eventually enable `provideExperimentalZonelessChangeDetection()` when all components are ready.

Q: Explain your approach to structuring a large Angular enterprise application.

A: Feature-based folder structure. **Core module:** singleton services (auth, logging, interceptors) — imported once in root. **Shared module:** reusable components, pipes, directives — imported by feature modules. **Feature modules:** each has its own components, services, routes, state — lazy-loaded independently. Smart/dumb component split: containers handle logic, presentational components are pure `@Input/@Output`. API layer separated from business logic layer. NgRx (or SignalStore) per feature slice. Strict typing, no `any`. ESLint + Prettier + Husky enforced.

121. Tricky JavaScript Output Questions

Checklist: Hoisting traps · closure traps · this binding puzzles · async execution order · type coercion · scope chains · event loop micro/macro tasks

121.1 Hoisting & TDZ Traps

```
// Q1: What's the output?
console.log(a);    // undefined (var is hoisted, initialized as undefined)
console.log(b);    // ReferenceError: Cannot access 'b' before initialization (TDZ!)
var a = 1;
let b = 2;

// Q2: What's the output?
var x = 1;
function foo() {
  console.log(x);  // undefined — NOT 1! var x below is hoisted inside foo
  var x = 2;
  console.log(x);  // 2
}
foo();

// Q3: What's the output?
function foo() { return 1; }
var foo;
console.log(typeof foo); // "function"
// Function declarations are hoisted ABOVE var declarations
// var foo doesn't overwrite because it has no assignment

// Q4: What's the output?
var foo = function() { return 1; };
function foo() { return 2; }
console.log(foo()); // 1
// function foo(){} is hoisted to top, then var foo = function(){} overwrites it
```

121.2 Closure Traps — The Classic Loop Problem

```
// Q5: What's the output?
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 3, 3, 3 (NOT 0, 1, 2!)
// Reason: var i is function-scoped, all closures share same i.
// By the time setTimeout fires, loop is done, i === 3.

// Fix 1: use let (block-scoped, each iteration gets its own i)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Output: 0, 1, 2 ■

// Fix 2: IIFE (creates new scope per iteration)
for (var i = 0; i < 3; i++) {
  (function(j) {
    setTimeout(() => console.log(j), 100);
  })(i);
}
// Output: 0, 1, 2 ■

// Q6: What's the output?
function createCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    getCount: () => count
  };
};
```

```

}
const counter = createCounter();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 2
// Both methods close over the SAME count variable

```

121.3 this Binding Puzzles

```

// Q7: What's the output?
const obj = {
  name: 'Sid',
  greet: function() { console.log(this.name); },
  greetArrow: () => { console.log(this.name); }
};
obj.greet();           // "Sid" - this = obj (implicit binding)
obj.greetArrow();      // undefined - arrow inherits this from outer scope (window/module)

// Q8: What's the output?
const obj2 = {
  name: 'Sid',
  greet() {
    console.log(this.name);
  }
};
const fn = obj2.greet;
fn();                  // undefined (or error in strict mode)
// When you extract a method, `this` binding is lost!
// Fix: const fn = obj2.greet.bind(obj2);

// Q9: What's the output?
function Person(name) {
  this.name = name;
  this.sayHi = function() {
    console.log(this.name);
  };
  this.sayHiArrow = () => {
    console.log(this.name);
  };
}
const p = new Person('Sid');
const sayHi = p.sayHi;
const sayHiArrow = p.sayHiArrow;
sayHi();               // undefined (lost this)
sayHiArrow();          // "Sid" ■ (arrow captures this from constructor scope!)

// Q10: What's the output?
const obj3 = {
  x: 42,
  getX: function() { return this.x; }
};
const unboundGetX = obj3.getX;
console.log(unboundGetX()); // undefined
const boundGetX = unboundGetX.bind(obj3);
console.log(boundGetX());   // 42
console.log(obj3.getX.call({ x: 99 })); // 99 (explicit binding)

```

121.4 Event Loop & Async Execution Order

```

// Q11: What's the output ORDER?
console.log('1');

setTimeout(() => console.log('2'), 0);

Promise.resolve().then(() => console.log('3'));

console.log('4');

```



```

// Output: 1, 4, 3, 2
// Explanation:
// 1. Synchronous: log '1'
// 2. setTimeout → macrotask queue
// 3. Promise.then → microtask queue
// 4. Synchronous: log '4'
// 5. Microtasks run first → log '3'
// 6. Macrotasks → log '2'

// Q12: What's the output ORDER?
async function foo() {
  console.log('A');
  await Promise.resolve();
  console.log('B');
}

console.log('C');
foo();
console.log('D');

// Output: C, A, D, B
// C: synchronous
// foo() starts: A is synchronous
// await pauses foo, schedules B as microtask
// D: synchronous
// B: microtask runs

// Q13: What's the output ORDER?
setTimeout(() => console.log('timeout'), 0);
Promise.resolve()
  .then(() => console.log('promise 1'))
  .then(() => console.log('promise 2'));
queueMicrotask(() => console.log('microtask'));
console.log('sync');

// Output: sync, promise 1, microtask, promise 2, timeout
// sync → microtasks (promise 1, microtask) → promise 2 (chained) → timeout

// Q14: What's the output?
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
Promise.resolve().then(() => console.log('promise'));
// Output: promise, 3, 3, 3
// Promise (microtask) runs before setTimeout (macrotask)

```

121.5 Type Coercion Gotchas

```

// Q15: What are these?
console.log([] + []);           // "" (both convert to "", then string concat)
console.log([] + {});          // "[object Object]"
console.log({} + []);           // 0 (in some contexts) or "[object Object]" (depends)
console.log(true + true);       // 2
console.log(true + false);      // 1
console.log('5' - 3);           // 2 (- converts string to number)
console.log('5' + 3);           // "53" (+ prefers string concat)
console.log(null + 1);          // 1 (null → 0)
console.log(undefined + 1);      // NaN (undefined → NaN)
console.log('2' > '12');        // true! (string comparison: '2' > '1')
console.log(NaN === NaN);       // false (NaN is not equal to anything)
console.log(0.1 + 0.2 === 0.3); // false (floating point: 0.30000000000000004)

// Q16: What's the output?
console.log(typeof null);        // "object" (historical bug, never fixed)
console.log(typeof undefined);   // "undefined"
console.log(typeof NaN);         // "number" (NaN is a number!)

```

```

console.log(typeof []);           // "object"
console.log(typeof function(){}); // "function"

// Q17: Equality weirdness
console.log(null == undefined);  // true (loose equality special case)
console.log(null === undefined); // false
console.log('' == false);        // true (both coerce to 0)
console.log('' === false);       // false (different types)
console.log([] == false);        // true ([] → '' → 0, false → 0)
console.log([1] == 1);           // true ([1] → '1' → 1)

```

■ Master Rule for Output Questions

Step-by-step approach: (1) Identify ALL synchronous code — runs first. (2) Identify microtasks (Promise.then, queueMicrotask, MutationObserver) — runs after sync. (3) Identify macrotasks (setTimeout, setInterval, I/O) — runs last. (4) For closures — ask "what is the variable's value AT THE TIME the function executes, not when it's defined?" (5) For this — ask "how is the function CALLED?" (method call → object, standalone → undefined/window, arrow → lexical, new → new object, bind/call/apply → explicit).

122. Implement JS Built-ins (Polyfills)

Checklist: Promise.all · Promise.race · Promise.any · Array.map · Array.filter · Array.reduce · Array.flat · Function.bind · call/apply · Object.assign · Object.create · Array.from

```
// IMPLEMENT Promise.all
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let completed = 0;
    const total = promises.length;

    if (total === 0) return resolve([]);

    promises.forEach((promise, index) => {
      Promise.resolve(promise) // handle non-promise values
        .then(value => {
          results[index] = value; // maintain order!
          completed++;
          if (completed === total) resolve(results);
        })
        .catch(reject); // first rejection rejects the whole thing
    });
  });
}

// IMPLEMENT Promise.race
function promiseRace(promises) {
  return new Promise((resolve, reject) => {
    promises.forEach(promise => {
      Promise.resolve(promise).then(resolve, reject);
      // First to settle (resolve or reject) wins
    });
  });
}

// IMPLEMENT Promise.any
function promiseAny(promises) {
  return new Promise((resolve, reject) => {
    const errors = [];
    let rejectedCount = 0;
    promises.forEach((promise, index) => {
      Promise.resolve(promise).then(resolve, error => {
        errors[index] = error;
        rejectedCount++;
        if (rejectedCount === promises.length) {
          reject(new AggregateError(errors, 'All promises rejected'));
        }
      });
    });
  });
}

// IMPLEMENT Array.prototype.map
Array.prototype.myMap = function(callback, thisArg) {
  const result = [];
  for (let i = 0; i < this.length; i++) {
    if (i in this) { // handle sparse arrays
      result.push(callback.call(thisArg, this[i], i, this));
    }
  }
  return result;
};

// IMPLEMENT Array.prototype.filter
Array.prototype.myFilter = function(callback, thisArg) {
```

```

    const result = [];
    for (let i = 0; i < this.length; i++) {
        if (i in this && callback.call(thisArg, this[i], i, this)) {
            result.push(this[i]);
        }
    }
    return result;
};

// IMPLEMENT Array.prototype.reduce
Array.prototype.myReduce = function(callback, initialValue) {
    let accumulator;
    let startIndex;

    if (arguments.length >= 2) {
        accumulator = initialValue;
        startIndex = 0;
    } else {
        if (this.length === 0) throw new TypeError('Reduce of empty array with no initial value');

        accumulator = this[0];
        startIndex = 1;
    }

    for (let i = startIndex; i < this.length; i++) {
        if (i in this) {
            accumulator = callback(accumulator, this[i], i, this);
        }
    }
    return accumulator;
};

// IMPLEMENT Function.prototype.bind
Function.prototype.myBind = function(context, ...args) {
    const fn = this;
    return function(...moreArgs) {
        return fn.apply(context, [...args, ...moreArgs]);
    };
};

// IMPLEMENT Function.prototype.call
Function.prototype.myCall = function(context, ...args) {
    context = context ?? globalThis;
    const sym = Symbol('fn');
    context[sym] = this;
    const result = context[sym](...args);
    delete context[sym];
    return result;
};

// IMPLEMENT Object.create
function objectCreate(proto, propertiesObject) {
    function F() {}
    F.prototype = proto;
    const obj = new F();
    if (propertiesObject) {
        Object.defineProperties(obj, propertiesObject);
    }
    return obj;
}

// IMPLEMENT new keyword
function myNew(Constructor, ...args) {
    const obj = Object.create(Constructor.prototype);
    const result = Constructor.apply(obj, args);
    return result instanceof Object ? result : obj;
}

```

}

123. Machine Coding — Common UI Components

Checklist: Star rating · Accordion · Countdown timer · Infinite scroll · Type-ahead / Autocomplete · Modal · Tabs · Carousel · Drag & Drop sortable list · Tic-tac-toe

123.1 Star Rating Component

```
// Pure JavaScript – no framework
class StarRating {
  constructor(container, { maxStars = 5, rating = 0, onChange } = {}) {
    this.container = container;
    this.maxStars = maxStars;
    this.rating = rating;
    this.onChange = onChange;
    this.render();
  }

  render() {
    this.container.innerHTML = '';
    this.container.style.cssText = 'display:inline-flex;gap:4px;cursor:pointer;font-size:28px;';

    for (let i = 1; i <= this.maxStars; i++) {
      const star = document.createElement('span');
      star.textContent = i <= this.rating ? '★' : '■';
      star.style.color = i <= this.rating ? '#f59e0b' : '#d1d5db';
      star.style.transition = 'transform 0.15s, color 0.15s';

      star.addEventListener('click', () => {
        this.rating = this.rating === i ? 0 : i; // click same star to clear
        this.onChange?.(this.rating);
        this.render();
      });

      star.addEventListener('mouseenter', () => {
        this.highlight(i);
      });

      this.container.appendChild(star);
    }

    this.container.addEventListener('mouseleave', () => this.render());
  }

  highlight(upTo) {
    const stars = this.container.children;
    for (let i = 0; i < stars.length; i++) {
      stars[i].textContent = i < upTo ? '★' : '■';
      stars[i].style.color = i < upTo ? '#f59e0b' : '#d1d5db';
      if (i < upTo) stars[i].style.transform = 'scale(1.15)';
    }
  }
}

// Usage: new StarRating(document.getElementById('stars'), { rating: 3, onChange: r => console.log(r) })
```

123.2 Countdown Timer

```
class CountdownTimer {
  constructor(container, targetDate) {
    this.container = container;
    this.targetDate = new Date(targetDate).getTime();
    this.interval = null;
    this.start();
  }

  start() {
    this.update(); // immediate first render
  }
}
```

```

    this.interval = setInterval(() => this.update(), 1000);
  }

  update() {
    const now = Date.now();
    const diff = this.targetDate - now;

    if (diff <= 0) {
      clearInterval(this.interval);
      this.container.textContent = '■ Time is up!';
      return;
    }

    const days = Math.floor(diff / (1000 * 60 * 60 * 24));
    const hours = Math.floor((diff % (1000 * 60 * 60 * 24)) / (1000 * 60 * 60));
    const mins = Math.floor((diff % (1000 * 60 * 60)) / (1000 * 60));
    const secs = Math.floor((diff % (1000 * 60)) / 1000);

    this.container.innerHTML = [
      this.box(days, 'Days'), this.box(hours, 'Hours'),
      this.box(mins, 'Minutes'), this.box(secs, 'Seconds')
    ].join('');
  }

  box(val, label) {
    return `<div style="display:inline-block;text-align:center;margin:0 10px">
      <div style="font-size:2.5rem;font-weight:bold">${String(val).padStart(2, '0')}</div>
      <div style="font-size:0.8rem;color:#666">${label}</div>
    </div>`;
  }

  destroy() { clearInterval(this.interval); }
}

```

123.3 LRU Cache — Most Common Data Structure Question

```

class LRUCache {
  constructor(capacity) {
    this.capacity = capacity;
    this.cache = new Map(); // Map preserves insertion order
  }

  get(key) {
    if (!this.cache.has(key)) return -1;
    // Move to end (most recently used)
    const value = this.cache.get(key);
    this.cache.delete(key);
    this.cache.set(key, value);
    return value;
  }

  put(key, value) {
    if (this.cache.has(key)) {
      this.cache.delete(key); // remove old position
    } else if (this.cache.size >= this.capacity) {
      // Evict least recently used (first item in Map)
      const oldestKey = this.cache.keys().next().value;
      this.cache.delete(oldestKey);
    }
    this.cache.set(key, value); // add at end (most recent)
  }
}

// Test:
const cache = new LRUCache(2);
cache.put(1, 'a');

```

```
cache.put(2, 'b');  
cache.get(1);      // 'a' – moves 1 to most recent  
cache.put(3, 'c'); // evicts key 2 (least recently used)  
cache.get(2);      // -1 (evicted!)  
cache.get(3);      // 'c'
```


124. Advanced Coding Challenges

Checklist: Implement setInterval using setTimeout · Implement pipe/compose · Implement JSON.stringify · Implement instanceof · Implement Object.assign · Implement Array.from · Implement flat with depth · Intersection of arrays

```
// IMPLEMENT setInterval using setTimeout (frequently asked!)
function mySetInterval(callback, delay) {
  let id = { cancelled: false };

  function tick() {
    if (!id.cancelled) {
      callback();
      setTimeout(tick, delay);
    }
  }

  setTimeout(tick, delay);
  return id;
}

function myClearInterval(id) { id.cancelled = true; }

// IMPLEMENT pipe (left-to-right function composition)
function pipe(...fns) {
  return (input) => fns.reduce((acc, fn) => fn(acc), input);
}

const transform = pipe(
  x => x * 2,
  x => x + 10,
  x => `Result: ${x}`
);
transform(5); // "Result: 20"

// IMPLEMENT compose (right-to-left function composition)
function compose(...fns) {
  return (input) => fns.reduceRight((acc, fn) => fn(acc), input);
}

// IMPLEMENT instanceof
function myInstanceOf(obj, Constructor) {
  if (obj === null || typeof obj !== 'object') return false;
  let proto = Object.getPrototypeOf(obj);
  while (proto !== null) {
    if (proto === Constructor.prototype) return true;
    proto = Object.getPrototypeOf(proto);
  }
  return false;
}

// IMPLEMENT JSON.stringify (simplified but covers main cases)
function myStringify(value) {
  if (value === null) return 'null';
  if (value === undefined) return undefined;
  if (typeof value === 'boolean') return String(value);
  if (typeof value === 'number') {
    if (!isFinite(value)) return 'null';
    return String(value);
  }
  if (typeof value === 'string') return ` "${value}" `;
  if (Array.isArray(value)) {
    const items = value.map(v => myStringify(v) ?? 'null');
    return `[${items.join(',')}]`;
  }
  if (typeof value === 'object') {
    const entries = Object.entries(value)
      .filter(([k, v]) => v !== undefined && typeof v !== 'function')
      .map(([k, v]) => ` "${k}":${myStringify(v)} `);
  }
}
```

```

    return `${entries.join(',')}`;
  }
}

// IMPLEMENT Array.prototype.flat with depth
function flatDeep(arr, depth = 1) {
  if (depth <= 0) return arr.slice();
  return arr.reduce((acc, val) =>
    acc.concat(Array.isArray(val) ? flatDeep(val, depth - 1) : val), []);
}
flatDeep([1, [2, [3, [4]]]], 2); // [1, 2, 3, [4]]
flatDeep([1, [2, [3, [4]]]], Infinity); // [1, 2, 3, 4]

// IMPLEMENT debounce with leading + trailing + cancel + flush
function debounce(fn, delay, { leading = false, trailing = true } = {}) {
  let timer = null;
  let lastArgs = null;

  function debounced(...args) {
    lastArgs = args;
    const callNow = leading && !timer;

    clearTimeout(timer);
    timer = setTimeout(() => {
      timer = null;
      if (trailing && lastArgs) fn.apply(this, lastArgs);
      lastArgs = null;
    }, delay);

    if (callNow) fn.apply(this, args);
  }

  debounced.cancel = () => { clearTimeout(timer); timer = null; lastArgs = null; };
  debounced.flush = () => {
    if (timer) { clearTimeout(timer); timer = null; fn.apply(this, lastArgs); lastArgs = null; }
  };

  return debounced;
}

```

125. Advanced TypeScript Patterns

Checklist: Conditional types with infer · template literal types · branded types · module augmentation · recursive types · discriminated union exhaustiveness · const assertions

```
// CONDITIONAL TYPES with infer – extract types
type ReturnOf<T> = T extends (...args: any[]) => infer R ? R : never;
type T1 = ReturnOf<() => string>; // string
type T2 = ReturnOf<(x: number) => boolean>; // boolean

// Extract Promise value
type Unwrap<T> = T extends Promise<infer U> ? Unwrap<U> : T;
type T3 = Unwrap<Promise<Promise<string>>>; // string (recursive unwrap!)

// Extract array element type
type ElementOf<T> = T extends (infer E)[] ? E : never;
type T4 = ElementOf<string[]>; // string

// BRANDED TYPES – prevent accidental type mixups
type UserId = string & { readonly __brand: 'UserId' };
type OrderId = string & { readonly __brand: 'OrderId' };

function createUserId(id: string): UserId { return id as UserId; }
function createOrderId(id: string): OrderId { return id as OrderId; }

function getUser(id: UserId) { /* ... */ }
function getOrder(id: OrderId) { /* ... */ }

const userId = createUserId('user-123');
const orderId = createOrderId('order-456');
getUser(userId); // ■
getUser(orderId); // ■ Compile error! OrderId is not assignable to UserId

// TEMPLATE LITERAL TYPES
type HTTPMethod = 'GET' | 'POST' | 'PUT' | 'DELETE';
type ApiRoute = '/users' | '/products' | '/orders';
type Endpoint = `${HTTPMethod} ${ApiRoute}`;
// 'GET /users' | 'GET /products' | 'GET /orders' | 'POST /users' | ...

// Event handler type
type EventName = 'click' | 'focus' | 'blur';
type Handler = `on${Capitalize<EventName>}`; // 'onClick' | 'onFocus' | 'onBlur'

// EXHAUSTIVE SWITCH – compile error if case is missing
type Shape = { kind: 'circle'; r: number } | { kind: 'square'; s: number };

function area(shape: Shape): number {
  switch (shape.kind) {
    case 'circle': return Math.PI * shape.r ** 2;
    case 'square': return shape.s ** 2;
    default: {
      const _exhaustive: never = shape; // ← ERROR if case is missing!
      return _exhaustive;
    }
  }
}

// const ASSERTION – narrowest possible type
const config = {
  endpoint: '/api',
  retries: 3,
  methods: ['GET', 'POST']
} as const;
// type: { readonly endpoint: "/api"; readonly retries: 3; readonly methods: readonly ["GET", "POST"] }
// Without as const: { endpoint: string; retries: number; methods: string[] }
```

```

// RECURSIVE TYPES
type DeepReadonly<T> = {
  readonly [K in keyof T]: T[K] extends object ? DeepReadonly<T[K]> : T[K];
};

type DeepPartial<T> = {
  [K in keyof T]?: T[K] extends object ? DeepPartial<T[K]> : T[K];
};

type JSONValue =
  | string | number | boolean | null
  | JSONValue[]
  | { [key: string]: JSONValue };

// BUILDER PATTERN with TypeScript (type-safe)
class QueryBuilder<Selected extends string = never> {
  select<F extends string>(field: F): QueryBuilder<Selected | F> {
    return this as any;
  }

  execute(): Record<Selected, unknown> {
    return {} as any;
  }
}

const result = new QueryBuilder()
  .select('name')
  .select('email')
  .execute();
// type: { name: unknown; email: unknown } ← only selected fields!

```

126. Frontend Low-Level Design (LLD)

Checklist: Design a real-time notification system · Design a form builder · Design a drag-and-drop kanban · Design an image gallery with zoom · Design a rich text editor · Design a rate limiter

```
// LLD: Real-Time Notification System

// Requirements:
// - Multiple notification types (info, warning, error, success)
// - Auto-dismiss with configurable timeout
// - Stack with max visible (newest replaces oldest)
// - Persistent notifications (don't auto-dismiss)
// - Action buttons (undo, retry, view)
// - Sound/vibration for urgent notifications
// - Queue system: if max visible, queue up the rest

interface Notification {
  id: string;
  type: 'info' | 'warning' | 'error' | 'success';
  title: string;
  message: string;
  duration?: number; // ms, 0 = persistent
  actions?: NotificationAction[];
  persistent?: boolean;
  priority?: 'low' | 'normal' | 'high';
  timestamp: number;
}

interface NotificationAction {
  label: string;
  handler: () => void;
  style?: 'primary' | 'secondary' | 'danger';
}

class NotificationService {
  private visible: Notification[] = [];
  private queue: Notification[] = [];
  private maxVisible = 5;
  private timers = new Map<string, number>();
  private subscribers = new Set<(notifications: Notification[]) => void>();

  show(options: Omit<Notification, 'id' | 'timestamp'>): string {
    const notification: Notification = {
      ...options,
      id: crypto.randomUUID(),
      timestamp: Date.now(),
      duration: options.duration ?? 5000
    };

    if (this.visible.length >= this.maxVisible) {
      // Remove lowest priority non-persistent notification
      const removable = this.visible.find(n => !n.persistent);
      if (removable) this.dismiss(removable.id);
      else { this.queue.push(notification); return notification.id; }
    }

    this.visible.push(notification);
    this.notify();

    if (notification.duration > 0 && !notification.persistent) {
      const timer = window.setTimeout(
        () => this.dismiss(notification.id),
        notification.duration
      );
      this.timers.set(notification.id, timer);
    }
  }
}
```

```

    return notification.id;
}

dismiss(id: string) {
  clearTimeout(this.timers.get(id));
  this.timers.delete(id);
  this.visible = this.visible.filter(n => n.id !== id);

  // Show queued notification
  if (this.queue.length > 0) {
    const next = this.queue.shift();
    this.visible.push(next);
  }
  this.notify();
}

subscribe(fn: (notifications: Notification[]) => void) {
  this.subscribers.add(fn);

  return () => this.subscribers.delete(fn);
}

private notify() {
  this.subscribers.forEach(fn => fn([...this.visible]));
}
}

// LLD: Token Bucket Rate Limiter (client-side)
class RateLimiter {
  private tokens: number;
  private lastRefill: number;

  constructor(
    private maxTokens: number,    // max burst
    private refillRate: number    // tokens per second
  ) {
    this.tokens = maxTokens;
    this.lastRefill = Date.now();
  }

  tryAcquire(): boolean {
    this.refill();
    if (this.tokens < 1) return false;
    this.tokens -= 1;
    return true;
  }

  private refill() {
    const now = Date.now();
    const elapsed = (now - this.lastRefill) / 1000;
    this.tokens = Math.min(this.maxTokens, this.tokens + elapsed * this.refillRate);
    this.lastRefill = now;
  }
}

// Usage: const limiter = new RateLimiter(10, 2); // 10 burst, 2/sec refill
// if (limiter.tryAcquire()) { makeApiCall(); }
// else { showRateLimitMessage(); }

```

127. Frontend System Design Interview Patterns

Checklist: Design approach framework · component hierarchy · data flow · API design · state management · error handling · performance · accessibility · edge cases

```
// FRAMEWORK FOR ANY FRONTEND SYSTEM DESIGN QUESTION

// Step 1: REQUIREMENTS CLARIFICATION (5 min)
// - Functional requirements: What features?
// - Non-functional: Performance, accessibility, offline support?
// - Scale: How many users? How much data?
// - Platform: Desktop? Mobile? Both?
// - Browser support?

// Step 2: HIGH-LEVEL ARCHITECTURE (5 min)
// Draw: Component tree, data flow, API endpoints
// Identify: Pages, shared components, services, stores

// Step 3: DATA MODEL & API (5 min)
// Define: TypeScript interfaces for all entities
// API endpoints: REST or GraphQL schema
// State shape: What goes in global store vs local state vs URL?

// Step 4: COMPONENT DESIGN (10 min)
// Smart vs Dumb components
// Content projection / composition patterns
// Reusability considerations

// Step 5: KEY INTERACTIONS (10 min)
// Walk through critical user flows
// Error states, loading states, empty states
// Optimistic updates, retry logic

// Step 6: OPTIMIZATIONS (5 min)
// Performance: lazy loading, virtual scroll, memoization
// Caching strategy
// Bundle size considerations

// EXAMPLE: Design a Twitter-like Feed

// Data model:
interface Tweet {
  id: string;
  author: User;
  content: string;
  media?: Media[];
  likes: number;
  retweets: number;
  replies: number;
  isLiked: boolean;
  isRetweeted: boolean;
  createdAt: string;
}

// Component hierarchy:
// FeedPage (smart)
//   ■■■ ComposeBox (smart – handles post creation)
//   ■■■ FeedList (dumb – receives tweets)
//     ■ ■■■ TweetCard (dumb – single tweet)
//       ■ ■ ■■■ UserAvatar
//       ■ ■ ■■■ TweetContent (handles text, links, media)
//       ■ ■ ■■■ TweetActions (like, retweet, reply buttons)
//       ■ ■ ■■■ TweetThread (if it's a thread)
//       ■ ■■■ InfiniteScrollSentinel
//     ■■■ TrendingSidebar
//     ■■■ SuggestedFollows
```

```
// State management:
// - Feed items → SignalStore (paginated, cursor-based)
// - Current user → global store (auth)
// - New tweet form → local state (component)
// - Filters/tab → URL state (router)
// - Like/retweet counts → optimistic update pattern

// Performance:
// - Virtual scrolling (1000s of tweets, only render ~20 DOM nodes)
// - Image lazy loading (IntersectionObserver)
// - Optimistic updates for likes (instant UI, background API)
// - Debounced search
// - WebSocket for real-time new tweet count badge

// Edge cases:
// - Network failure during post → retry with saved draft
// - Duplicate rapid likes → exhaustMap prevents double-request
// - Content longer than limit → show "Read more" truncation

// - Deleted tweets → handle 404 gracefully
// - Rate limiting → show message, queue requests
```


128. Web Performance Case Studies

Checklist: Diagnosing slow FCP · fixing layout shifts · optimizing bundle size · handling slow APIs · memory leak detection

```
// CASE STUDY 1: Slow First Contentful Paint (FCP > 4s)
// Diagnosis steps:
// 1. Run Lighthouse → see what's blocking
// 2. Check Network waterfall → large blocking resources?
// 3. Check Coverage tab → unused CSS/JS %

// Common fixes:
// - Inline critical CSS, defer non-critical
// - Preconnect to API: <link rel="preconnect" href="https://api.example.com">
// - Preload key resources: <link rel="preload" href="/fonts/main.woff2" as="font">
// - Remove render-blocking scripts (add defer/async)
// - Server-side render the above-the-fold content
// - Compress assets (Brotli > gzip, 20-30% smaller)

// CASE STUDY 2: Layout Shift (CLS > 0.25)
// Diagnosis: DevTools Performance tab → Layout Shift rows
// Common causes + fixes:
// 1. Images without dimensions → add width/height or aspect-ratio
// 2. Web fonts causing FOUT → font-display: swap + preload
// 3. Dynamic content inserted above → reserve space with min-height
// 4. Ads/embeds without size → use <iframe width="..." height="...">

// CASE STUDY 3: Memory Leak
// Diagnosis: DevTools → Memory → Heap Snapshot → Compare snapshots
function detectLeak() {
  // Take snapshot 1
  // Perform action (open/close modal 10 times)
  // Take snapshot 2
  // Compare: if detached DOM nodes keep growing → LEAK!
}

// Common frontend memory leaks:
// 1. Unsubscribed observables / event listeners
// 2. Closures holding references to detached DOM
// 3. Global arrays/maps that grow without bounds
// 4. setInterval without clearInterval
// 5. Detached DOM trees (element removed but JS still references it)

// Fix pattern:
class Widget {
  private abortController = new AbortController();

  init() {
    window.addEventListener('resize', this.onResize, {
      signal: this.abortController.signal // auto-cleanup!
    });
  }

  destroy() {
    this.abortController.abort(); // removes ALL listeners at once
  }
}
```

129. Behavioral & Situational Questions

Checklist: Conflict resolution · technical debt decisions · mentoring · handling tight deadlines · production incidents · architectural decisions

```
// SITUATIONAL QUESTIONS – prepare stories using STAR method

// Q: "Tell me about a time you disagreed with a technical decision."
// Framework:
// Situation: What was the context and the disagreement?
// Task: What was your role?
// Action: How did you handle it? (data-driven arguments, POC, compromise?)
// Result: What happened? What did you learn?

// Q: "How do you handle technical debt?"
// Good answer framework:
// 1. Document it: maintain a tech debt register with impact scores
// 2. Prioritize: security debt > performance debt > code quality debt
// 3. Budget time: 20% of sprint for tech debt (negotiate with PM)
// 4. Prevent: enforce coding standards, PR reviews, automated linting
// 5. Track: measure code coverage, bundle size, Lighthouse scores over time

// Q: "A critical production bug is reported. Walk me through your approach."
// 1. ASSESS: Severity? Users affected? Data loss? Revenue impact?
// 2. COMMUNICATE: Inform stakeholders, set incident channel
// 3. MITIGATE: Can we feature-flag or rollback? Reduce blast radius
// 4. DIAGNOSE: Check error tracking (Sentry), logs, recent deployments
// 5. FIX: Minimal fix for production (hotfix), thorough fix for next release
// 6. POSTMORTEM: Root cause, timeline, what we'll do to prevent recurrence

// Q: "How do you mentor junior developers?"
// - Pair programming sessions on complex features
// - Code review as a teaching tool (explain WHY, not just WHAT)
// - Set up knowledge sharing sessions (tech talks, demos)
// - Create clear documentation and coding guidelines
// - Give increasing ownership: start small, build confidence
// - Regular 1:1s to understand their goals and blockers

// Q: "How do you make architectural decisions?"
// 1. Define the problem and constraints clearly
// 2. Research options (at least 2-3 alternatives)
// 3. Create a comparison matrix: pros, cons, effort, risk
// 4. Build a lightweight POC for risky choices
// 5. Write an Architecture Decision Record (ADR)
// 6. Get team buy-in through RFC process
// 7. Decide with data, document with ADRs
```

130. Advanced Frontend Security

Checklist: Prototype pollution · supply chain attacks · DOM clobbering · ReDoS · dependency auditing · subresource integrity · permissions policy

```
// PROTOTYPE POLLUTION – manipulate Object.prototype
// Attack: JSON.parse('{"__proto__": {"isAdmin": true}}')
// After pollution: ({}).isAdmin === true // DANGER!

// Prevention:
// 1. Use Object.create(null) for dictionaries (no prototype)
const safeMap = Object.create(null);

// 2. Freeze the prototype
Object.freeze(Object.prototype);

// 3. Validate input keys
function safeAssign(target, source) {
  for (const key of Object.keys(source)) {
    if (key === '__proto__' || key === 'constructor' || key === 'prototype') {
      continue; // skip dangerous keys
    }
    target[key] = source[key];
  }
  return target;
}

// SUPPLY CHAIN ATTACKS – compromised npm packages
// Prevention:
// 1. npm audit / pnpm audit (check for known vulnerabilities)
// 2. Lock file (package-lock.json) – pin exact versions
// 3. Use npm ci in CI (installs from lock file only)
// 4. Subresource Integrity for CDN scripts
// 5. Renovate/Dependabot for automated security updates
// 6. Review dependency tree: npm ls --all
// 7. Use Socket.dev or Snyk for supply chain scanning

// PERMISSIONS POLICY – control browser features
Permissions-Policy:
  camera=(), // disable camera
  microphone=(), // disable microphone
  geolocation=(self), // only our origin
  payment=(self "https://pay.example.com")

// DOM CLOBBERING – HTML elements override global properties
// Attack: <img name="location" /> → window.location is now the img!
// Prevention: always use explicit references, CSP, sanitize HTML inputs

// DEPENDENCY AUDIT CHECKLIST for production:
// ■ npm audit --production (zero critical/high vulnerabilities)
// ■ Check bundle for eval(), Function(), innerHTML usage
// ■ Review postinstall scripts in dependencies
// ■ Pin dependency versions in lock file
// ■ Enable 2FA on npm account (if publishing)
// ■ Monitor with Snyk/Socket.dev/GitHub Dependabot
```

■ Final Interview Power Tips

Before the interview:

- Review your past projects — be ready to explain architecture, challenges, and trade-offs for ANY project on your resume.
- Practice explaining concepts out loud — interviewers evaluate communication as much as knowledge.
- Know your numbers: "I reduced bundle size from 2.4MB to 800KB" beats "I optimized the build."

- Prepare 2-3 stories for behavioral questions (conflict, mentoring, production incident, technical decision).

During the interview:

- Think out loud — silence is worse than a wrong direction (they can redirect you).
- Ask clarifying questions before coding — shows maturity.
- Start with brute force, then optimize — shows systematic thinking.
- For system design: draw the component tree first, then dive into details.
- If stuck: "I'd approach this by..." (describe the concept even if you can't code it perfectly).
- Always discuss trade-offs: "We could use X, but Y might be better here because..."

131. Coding Challenges — Closures & Currying

Concept Focus: Lexical scope · closure over variables · partial application · infinite currying · function composition via closures

131.1 Basic — Counter with Private State

```
// Create a counter using closures (no class, no global variable)
function createCounter(initial = 0) {
  let count = initial;
  return {
    increment() { return ++count; },
    decrement() { return --count; },
    getCount() { return count; },
    reset() { count = initial; return count; }
  };
}

const counter = createCounter(10);
console.log(counter.increment()); // 11
console.log(counter.increment()); // 12
console.log(counter.decrement()); // 11
console.log(counter.reset()); // 10
console.log(counter.count); // undefined – truly private!
```

131.2 Basic — once() Function (run only first time)

```
// Create once() – the function only executes the first time
function once(fn) {
  let called = false;
  let result;
  return function(...args) {
    if (!called) {
      called = true;
      result = fn.apply(this, args);
    }
    return result;
  };
}

const initialize = once(() => {
  console.log('DB connected');
  return { status: 'connected' };
});

initialize(); // logs "DB connected", returns {status:'connected'}
initialize(); // no log, returns same {status:'connected'}
initialize(); // no log, returns same {status:'connected'}
```

131.3 Intermediate — Simple Curry (fixed arity)

```
// Implement curry() that transforms f(a,b,c) → f(a)(b)(c)
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function(...nextArgs) {
      return curried.apply(this, [...args, ...nextArgs]);
    };
  };
}

// Usage:
const add = (a, b, c) => a + b + c;
const curriedAdd = curry(add);

console.log(curriedAdd(1)(2)(3)); // 6
```

```

console.log(curriedAdd(1, 2)(3)); // 6
console.log(curriedAdd(1)(2, 3)); // 6
console.log(curriedAdd(1, 2, 3)); // 6

```

131.4 Intermediate — Infinite Currying (sum(1)(2)(3)...)()

```

// sum(1)(2)(3)() → 6 – call with no args to get result
function sum(a) {
  return function(b) {
    if (b === undefined) return a;
    return sum(a + b);
  };
}

console.log(sum(1)(2)(3)()); // 6
console.log(sum(5)(10)(15)(20)()); // 50
console.log(sum(1)()); // 1

// Variant: using toString/valueOf for automatic coercion
function sumAuto(a) {
  function inner(b) {
    return sumAuto(a + b);
  }
  inner.valueOf = () => a;
  inner.toString = () => String(a);
  return inner;
}

console.log(+sumAuto(1)(2)(3)); // 6 – the + triggers valueOf
console.log(`${sumAuto(1)(2)}`); // "3" – template literal triggers toString

```

131.5 Advanced — Generic Partial Application

```

// partial(fn, ...presetArgs) – like bind without "this"
// Support placeholder _ for skipping arguments
const _ = Symbol('placeholder');

function partial(fn, ...presetArgs) {
  return function(...laterArgs) {
    const args = [];
    let laterIdx = 0;

    // Fill in preset args, replacing placeholders
    for (const arg of presetArgs) {
      if (arg === _) {
        args.push(laterArgs[laterIdx++]);
      } else {
        args.push(arg);
      }
    }
    // Append remaining later args
    while (laterIdx < laterArgs.length) {
      args.push(laterArgs[laterIdx++]);
    }

    return fn(...args);
  };
}

// Usage:
const divide = (a, b) => a / b;
const halve = partial(divide, _, 2); // skip first arg, preset second
console.log(halve(10)); // 5

const greet = (greeting, name, punct) => `${greeting}, ${name}${punct}`;
const hello = partial(greet, 'Hello');
console.log(hello('World', '!')); // "Hello, World!"

```

```
const exclaim = partial(greet, _, _, '!!');
console.log(exclaim('Hi', 'Dev')); // "Hi, Dev!"
```

131.6 Advanced — Compose & Pipe with Curried Functions

```
// Compose: right-to-left execution  compose(f, g, h)(x) = f(g(h(x)))
// Pipe:    left-to-right execution  pipe(h, g, f)(x) = f(g(h(x)))

const compose = (...fns) => (x) =>
  fns.reduceRight((acc, fn) => fn(acc), x);

const pipe = (...fns) => (x) =>
  fns.reduce((acc, fn) => fn(acc), x);

// Real-world example: data transformation pipeline
const trim      = (s) => s.trim();
const lower     = (s) => s.toLowerCase();
const split     = (sep) => (s) => s.split(sep); // curried!
const join      = (sep) => (arr) => arr.join(sep);
const capitalize = (s) => s[0].toUpperCase() + s.slice(1);
const map       = (fn) => (arr) => arr.map(fn);

const slugify = pipe(
  trim,
  lower,
  split(/\s+/),
  join('-')
);
console.log(slugify(' Hello World Example ')); // "hello-world-example"

const titleCase = pipe(
  trim,
  lower,
  split(/\s+/),
  map(capitalize),
  join(' ')
);
console.log(titleCase(' hello world ')); // "Hello World"
```

131.7 Advanced — Memoize with Closure (LRU-style)

```
// Memoize that handles multiple arguments, uses Map for O(1) lookup
function memoize(fn, maxSize = 100) {
  const cache = new Map();

  function memoized(...args) {
    const key = JSON.stringify(args); // simple key strategy

    if (cache.has(key)) {
      // Move to end (most recently used)
      const value = cache.get(key);
      cache.delete(key);
      cache.set(key, value);
      return value;
    }

    const result = fn.apply(this, args);
    cache.set(key, result);

    // Evict oldest if over capacity
    if (cache.size > maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
    }

    return result;
  }
}
```

```

    memoized.cache = cache;
    memoized.clear = () => cache.clear();
    return memoized;
}

// Usage:
const expensiveFib = memoize(function fib(n) {
  if (n <= 1) return n;
  return expensiveFib(n - 1) + expensiveFib(n - 2);
});

console.log(expensiveFib(50)); // 12586269025 – instant!
console.log(expensiveFib.cache.size); // entries cached

```

■ Interview Tip — Closures & Currying:

- **Q: What is a closure?** A function that retains access to its lexical scope even when executed outside that scope.
- **Q: Curry vs Partial Application?** Curry transforms $f(a,b,c)$ into $f(a)(b)(c)$. Partial fixes some args upfront:
 $\text{partial}(f, 1) \rightarrow f(1, b, c)$.
- **Q: When is currying useful?** Reusable config (loggers, validators), point-free function composition, event handler factories.
- **Q: Name a closure pitfall.** Closing over loop variables (var in for-loop), accidental memory retention of large objects.

132. Coding Challenges — Debounce & Throttle

Concept Focus: Timer management · leading/trailing edge · cancel/flush · requestAnimationFrame throttle · real-world patterns

132.1 Basic — Simple Debounce

```
// debounce(fn, delay) – delays execution until after delay ms of silence
function debounce(fn, delay) {
  let timerId;
  return function(...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// Usage: search input
const searchInput = document.getElementById('search');
const handleSearch = debounce((e) => {
  console.log('API call:', e.target.value);
}, 300);
searchInput?.addEventListener('input', handleSearch);
```

132.2 Basic — Simple Throttle

```
// throttle(fn, limit) – executes at most once per limit ms
function throttle(fn, limit) {
  let inThrottle = false;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => {
        inThrottle = false;
      }, limit);
    }
  };
}

// Usage: scroll handler
const handleScroll = throttle(() => {
  console.log('Scroll position:', window.scrollY);
}, 200);
window.addEventListener('scroll', handleScroll);
```

132.3 Intermediate — Debounce with Leading & Trailing Options

```
// Full-featured debounce: leading edge, trailing edge, or both
function debounce(fn, delay, { leading = false, trailing = true } = {}) {
  let timerId;
  let lastArgs;

  function debounced(...args) {
    lastArgs = args;
    const callNow = leading && !timerId;

    clearTimeout(timerId);
    timerId = setTimeout(() => {
      timerId = null;
      if (trailing && lastArgs) {
        fn.apply(this, lastArgs);
        lastArgs = null;
      }
    }, delay);

    if (callNow) {

```

```

        fn.apply(this, args);
        lastArgs = null; // prevent trailing call for same invocation
    }
}

debounced.cancel = () => {
    clearTimeout(timerId);
    timerId = null;
    lastArgs = null;
};

debounced.flush = () => {
    if (timerId && lastArgs) {
        fn.apply(null, lastArgs);
        debounced.cancel();
    }
};

return debounced;
}

// Leading-only: fires immediately, ignores subsequent calls within delay
const btnHandler = debounce(submitForm, 1000, { leading: true, trailing: false });

// Both: fires immediately AND after delay of silence
const autosave = debounce(saveData, 2000, { leading: true, trailing: true });

```

132.4 Intermediate — Throttle with Trailing Call

```

// Throttle that also fires on the trailing edge (last call isn't lost)
function throttle(fn, limit) {
    let lastTime = 0;
    let timerId;

    return function(...args) {
        const now = Date.now();
        const remaining = limit - (now - lastTime);

        clearTimeout(timerId);

        if (remaining <= 0) {
            // Enough time has passed – execute immediately
            lastTime = now;
            fn.apply(this, args);
        } else {
            // Schedule trailing call
            timerId = setTimeout(() => {
                lastTime = Date.now();
                fn.apply(this, args);
            }, remaining);
        }
    };
}

// This ensures the LAST call always fires, even if it comes during throttle window
const resize = throttle(() => recalcLayout(), 150);
window.addEventListener('resize', resize);

```

132.5 Advanced — requestAnimationFrame Throttle

```

// Best for visual updates – syncs with browser's paint cycle (~16ms / 60fps)
function rafThrottle(fn) {
    let rafId = null;
    let lastArgs = null;

    function throttled(...args) {
        lastArgs = args;
    }
}

```

```

    if (rafId === null) {
      rafId = requestAnimationFrame(() => {
        fn.apply(this, lastArgs);
        rafId = null;
        lastArgs = null;
      });
    }
  }

  throttled.cancel = () => {
    if (rafId !== null) {
      cancelAnimationFrame(rafId);
      rafId = null;
      lastArgs = null;
    }
  };

  return throttled;
}

// Usage: smooth drag or scroll animations
const onMouseMove = rafThrottle((e) => {
  tooltip.style.transform = `translate(${e.clientX}px, ${e.clientY}px)`;
});
document.addEventListener('mousemove', onMouseMove);

```

132.6 Advanced — Debounce with maxWait (Lodash-style)

```

// If continuous calls keep resetting the debounce, maxWait ensures
// the function fires at least once within maxWait ms
function debounce(fn, delay, { maxWait = Infinity, leading = false } = {}) {
  let timerId, maxTimerId;
  let lastArgs, lastThis;
  let lastCallTime;

  function invoke() {
    const args = lastArgs;
    const thisArg = lastThis;
    lastArgs = lastThis = undefined;
    clearTimeout(timerId);
    clearTimeout(maxTimerId);
    timerId = maxTimerId = null;
    fn.apply(thisArg, args);
  }

  function debounced(...args) {
    lastArgs = args;
    lastThis = this;
    lastCallTime = Date.now();

    const callNow = leading && !timerId;

    clearTimeout(timerId);
    timerId = setTimeout(invoke, delay);

    // Set maxWait timer if not already running
    if (maxWait !== Infinity && !maxTimerId) {
      maxTimerId = setTimeout(invoke, maxWait);
    }

    if (callNow) invoke();
  }

  debounced.cancel = () => {
    clearTimeout(timerId);
    clearTimeout(maxTimerId);
  };
}

```

```
        timerId = maxTimerId = null;
        lastArgs = lastThis = undefined;
    };

    return debounced;
}

// Usage: autocomplete that fires at most every 3s even if user keeps typing
const search = debounce(fetchResults, 300, { maxWait: 3000 });
```

■ Interview Tip — Debounce vs Throttle:

- **Debounce:** Wait until user STOPS doing something. Use for: search input, form validation, window resize end.
- **Throttle:** Execute at regular intervals WHILE user is doing something. Use for: scroll events, mouse move, game loops.
- **maxWait:** Hybrid — debounce with guaranteed max delay. Lodash's `_.debounce` supports this.
- **RAF throttle:** Best for visual/paint updates — automatically aligns to 60fps.
- **Common follow-up:** "Implement cancel and flush methods" — shows you understand lifecycle management.

133. Coding Challenges — Promises & Async Patterns

Concept Focus: Promise constructor · chaining · error handling · Promise combinators · async iteration · concurrency control · retry patterns

133.1 Basic — Promisify a Callback Function

```
// Convert callback-style function to Promise-based
function promisify(fn) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      fn(...args, (err, result) => {
        if (err) reject(err);
        else resolve(result);
      });
    });
  };
}

// Usage:
const fs = require('fs');
const readFileAsync = promisify(fs.readFile);
readFileAsync('config.json', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

133.2 Basic — Sleep / Delay Promise

```
// sleep(ms) – returns a promise that resolves after ms
const sleep = (ms) => new Promise(resolve => setTimeout(resolve, ms));

// Timeout wrapper – race between promise and timer
function withTimeout(promise, ms, message = 'Timeout') {
  const timeout = new Promise((_, reject) =>
    setTimeout(() => reject(new Error(message)), ms)
  );
  return Promise.race([promise, timeout]);
}

// Usage:
async function fetchWithTimeout(url) {
  try {
    const data = await withTimeout(fetch(url), 5000, 'Request timed out');
    return data.json();
  } catch (e) {
    console.error(e.message);
  }
}
```

133.3 Intermediate — Sequential Async Execution

```
// Execute async functions one after another (NOT parallel)
// Method 1: reduce
async function sequential(tasks) {
  return tasks.reduce(async (prevPromise, task) => {
    const results = await prevPromise;
    const result = await task();
    return [...results, result];
  }, Promise.resolve([]));
}

// Method 2: for-of (cleaner)
async function sequential2(tasks) {
  const results = [];
  for (const task of tasks) {
    results.push(await task());
  }
}
```

```

    }
    return results;
}

// Usage:
const apis = [
  () => fetch('/api/users').then(r => r.json()),
  () => fetch('/api/posts').then(r => r.json()),
  () => fetch('/api/comments').then(r => r.json()),
];
const [users, posts, comments] = await sequential(apis);

```

133.4 Intermediate — Retry with Exponential Backoff

```

// Retry a promise-returning function with exponential backoff
async function retry(fn, { retries = 3, delay = 1000, backoff = 2 } = {}) {
  let lastError;

  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      return await fn();
    } catch (err) {
      lastError = err;
      if (attempt < retries) {
        const waitTime = delay * Math.pow(backoff, attempt);
        console.log(`Retry ${attempt + 1}/${retries} in ${waitTime}ms...`);
        await new Promise(r => setTimeout(r, waitTime));
      }
    }
  }

  throw lastError;
}

// Usage:
const data = await retry(
  () => fetch('https://flaky-api.com/data').then(r => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json();
  }),
  { retries: 3, delay: 500, backoff: 2 }
);
// Waits: 500ms → 1000ms → 2000ms before giving up

```

133.5 Advanced — Parallel with Concurrency Limit

```

// Run N promises at a time (like p-limit / Promise pool)
async function parallelLimit(tasks, concurrency) {
  const results = new Array(tasks.length);
  let currentIndex = 0;

  async function worker() {
    while (currentIndex < tasks.length) {
      const index = currentIndex++;
      results[index] = await tasks[index]();
    }
  }

  // Create N workers
  const workers = Array.from({ length: Math.min(concurrency, tasks.length) }, worker);
  await Promise.all(workers);

  return results;
}

// Usage: download 100 images, 5 at a time
const imageURLs = Array.from({ length: 100 }, (_, i) => `https://img.com/${i}.jpg`);

```

```
const tasks = imageURLs.map(url => () => fetch(url).then(r => r.blob()));

const images = await parallelLimit(tasks, 5);
// Only 5 fetches run simultaneously!
```

133.6 Advanced — Implement Promise.allSettled

```
// Returns array of {status, value/reason} for ALL promises (never rejects)
function allSettled(promises) {
  return Promise.all(
    promises.map(p =>
      Promise.resolve(p)
        .then(value => ({ status: 'fulfilled', value }))
        .catch(reason => ({ status: 'rejected', reason }))
    )
  );
}

// Usage:
const results = await allSettled([
  fetch('/api/fast'),           // succeeds
  fetch('/api/broken'),         // fails
  Promise.resolve(42),          // succeeds
]);
// [
//   { status: 'fulfilled', value: Response },
//   { status: 'rejected', reason: Error },
//   { status: 'fulfilled', value: 42 }
// ]
```

133.7 Advanced — Async Queue (process tasks in FIFO order)

```
// Process async tasks one at a time in the order they're added
class AsyncQueue {
  #queue = [];
  #processing = false;

  enqueue(asyncFn) {
    return new Promise((resolve, reject) => {
      this.#queue.push({ asyncFn, resolve, reject });
      this.#process();
    });
  }

  async #process() {
    if (this.#processing) return;
    this.#processing = true;

    while (this.#queue.length > 0) {
      const { asyncFn, resolve, reject } = this.#queue.shift();
      try {
        const result = await asyncFn();
        resolve(result);
      } catch (err) {
        reject(err);
      }
    }

    this.#processing = false;
  }

  get size() { return this.#queue.length; }
}

// Usage: ensure API calls happen sequentially
const queue = new AsyncQueue();
```

```
// These all return promises, but execute one at a time
queue.enqueue(() => saveUser(data));
queue.enqueue(() => sendNotification(userId));
queue.enqueue(() => updateAnalytics('user_created'));
```

133.8 Expert — Cancellable Promise with AbortController

```
// Make any async operation cancellable
function cancellable(asyncFn) {
  const controller = new AbortController();

  const promise = new Promise(async (resolve, reject) => {
    controller.signal.addEventListener('abort', () => {
      reject(new DOMException('Cancelled', 'AbortError'));
    });

    try {
      const result = await asyncFn(controller.signal);
      if (!controller.signal.aborted) {
        resolve(result);
      }
    } catch (err) {
      if (!controller.signal.aborted) {
        reject(err);
      }
    }
  });

  return { promise, cancel: () => controller.abort() };
}

// Usage:
const { promise, cancel } = cancellable(async (signal) => {
  const resp = await fetch('/api/data', { signal });
  return resp.json();
});

// Cancel after 2 seconds if still running
setTimeout(cancel, 2000);

try {
  const data = await promise;
} catch (e) {
  if (e.name === 'AbortError') console.log('Request was cancelled');
}
```

■ Interview Tip — Promises & Async:

- **Q: Promise.all vs allSettled vs race vs any?** all = fail-fast on first rejection. allSettled = wait for all, no rejection. race = first to settle wins. any = first to fulfill wins (ignores rejections).
- **Q: How to limit concurrency?** Worker pool pattern — create N workers that pull from a shared task queue.
- **Q: Retry strategy?** Exponential backoff with jitter (randomized delay) to avoid thundering herd.
- **Common mistake:** Forgetting to handle errors in Promise.all causes unhandledrejection.

134. Coding Challenges — Array & Object Manipulation

Concept Focus: map/filter/reduce patterns · flat/groupBy · deep clone · object diff · immutable updates · custom iterators

134.1 Basic — Group By Property

```
// groupBy(array, keyFn) – group items by a computed key
function groupBy(arr, keyFn) {
  return arr.reduce((groups, item) => {
    const key = typeof keyFn === 'function' ? keyFn(item) : item[keyFn];
    (groups[key] ??= []).push(item);
    return groups;
  }, {});
}

const users = [
  { name: 'Alice', role: 'admin' },
  { name: 'Bob', role: 'user' },
  { name: 'Carol', role: 'admin' },
  { name: 'Dave', role: 'user' },
];

console.log(groupBy(users, 'role'));
// {
//   admin: [{name:'Alice',role:'admin'}, {name:'Carol',role:'admin'}],
//   user:  [{name:'Bob',role:'user'}, {name:'Dave',role:'user'}]
// }

// With function key:
console.log(groupBy([1,2,3,4,5,6], n => n % 2 === 0 ? 'even' : 'odd'));
// { odd: [1,3,5], even: [2,4,6] }
```

134.2 Basic — Flatten Array to Depth

```
// flat(arr, depth) – flatten nested arrays to specified depth
function flat(arr, depth = 1) {
  if (depth <= 0) return arr.slice();

  return arr.reduce((acc, item) => {
    if (Array.isArray(item)) {
      acc.push(...flat(item, depth - 1));
    } else {
      acc.push(item);
    }
    return acc;
  }, []);
}

console.log(flat([1, [2, [3, [4]]]], 1)); // [1, 2, [3, [4]]]
console.log(flat([1, [2, [3, [4]]]], 2)); // [1, 2, 3, [4]]
console.log(flat([1, [2, [3, [4]]]], Infinity)); // [1, 2, 3, 4]

// Iterative version (no recursion, no stack overflow):
function flatIterative(arr, depth = 1) {
  let result = [...arr];
  for (let d = 0; d < depth; d++) {
    const next = [];
    let didFlatten = false;
    for (const item of result) {
      if (Array.isArray(item)) {
        next.push(...item);
        didFlatten = true;
      } else {
        next.push(item);
      }
    }
    if (!didFlatten) break;
    result = next;
  }
}
```

```

    result = next;
    if (!didFlatten) break;
  }
  return result;
}

```

134.3 Intermediate — Deep Clone (handle all types)

```

// Deep clone that handles: objects, arrays, Date, RegExp, Map, Set, circular refs
function deepClone(obj, seen = new WeakMap()) {
  // Primitives + functions
  if (obj === null || typeof obj !== 'object') return obj;

  // Circular reference
  if (seen.has(obj)) return seen.get(obj);

  // Special types
  if (obj instanceof Date) return new Date(obj.getTime());
  if (obj instanceof RegExp) return new RegExp(obj.source, obj.flags);
  if (obj instanceof Map) {
    const map = new Map();
    seen.set(obj, map);
    obj.forEach((v, k) => map.set(deepClone(k, seen), deepClone(v, seen)));
    return map;
  }
  if (obj instanceof Set) {
    const set = new Set();
    seen.set(obj, set);
    obj.forEach(v => set.add(deepClone(v, seen)));
    return set;
  }

  // Arrays and plain objects
  const clone = Array.isArray(obj) ? [] : Object.create(Object.getPrototypeOf(obj));
  seen.set(obj, clone);

  for (const key of Reflect.ownKeys(obj)) {
    clone[key] = deepClone(obj[key], seen);
  }

  return clone;
}

// Test circular reference:
const a = { x: 1 };
a.self = a;
const b = deepClone(a);
console.log(b.self === b); // true (correctly cloned circular ref)
console.log(b.self === a); // false (different object)

```

134.4 Intermediate — Object Diff (find changes between two objects)

```

// diff(oldObj, newObj) – returns what changed, added, removed
function diff(oldObj, newObj, path = '') {
  const changes = [];
  const allKeys = new Set([...Object.keys(oldObj), ...Object.keys(newObj)]);

  for (const key of allKeys) {
    const fullPath = path ? `${path}.${key}` : key;
    const oldVal = oldObj[key];
    const newVal = newObj[key];

    if (!(key in oldObj)) {
      changes.push({ type: 'added', path: fullPath, value: newVal });
    } else if (!(key in newObj)) {
      changes.push({ type: 'removed', path: fullPath, oldValue: oldVal });
    } else if (typeof oldVal === 'object' && typeof newVal === 'object')

```

```

        && oldVal !== null && newVal !== null
        && !Array.isArray(oldVal) && !Array.isArray(newVal)) {
      changes.push(...diff(oldVal, newVal, fullPath));
    } else if (oldVal !== newVal) {
      changes.push({ type: 'changed', path: fullPath, oldValue: oldVal, newValue: newVal });
    }
  }
}

return changes;
}

const before = { name: 'Alice', age: 25, address: { city: 'NYC', zip: '10001' } };
const after = { name: 'Alice', age: 26, address: { city: 'LA', zip: '10001' }, role: 'admin' };

console.log(diff(before, after));
// [
//   { type: 'changed', path: 'age', oldValue: 25, newValue: 26 },
//   { type: 'changed', path: 'address.city', oldValue: 'NYC', newValue: 'LA' },
//   { type: 'added', path: 'role', value: 'admin' }
// ]

```

134.5 Advanced — Immutable Update Helper (like immer's produce)

```

// setIn(obj, path, value) – returns NEW object with value set at path
function setIn(obj, pathStr, value) {
  const path = pathStr.split('.');

  function helper(current, index) {
    if (index === path.length) return value;

    const key = path[index];
    const isArray = Array.isArray(current);
    const clone = isArray ? [...current] : { ...current };

    clone[key] = helper(current?.[key] ?? {}, index + 1);
    return clone;
  }

  return helper(obj, 0);
}

// deleteIn – returns NEW object with key removed at path
function deleteIn(obj, pathStr) {
  const path = pathStr.split('.');
  const lastKey = path.pop();

  if (path.length === 0) {
    const { [lastKey]: _, ...rest } = obj;
    return rest;
  }

  const parent = path.reduce((o, k) => o?.[k], obj);
  if (!parent) return obj;

  return setIn(obj, path.join('.'), (() => {
    const { [lastKey]: _, ...rest } = parent;
    return rest;
  })());
}

const state = { user: { name: 'Alice', settings: { theme: 'dark' } } };
const next = setIn(state, 'user.settings.theme', 'light');

console.log(next.user.settings.theme); // 'light'
console.log(state.user.settings.theme); // 'dark' – original unchanged
console.log(next === state);           // false

```

```
console.log(next.user.name === state.user.name); // true – shared ref!
```

134.6 Advanced — Custom Iterable + Lazy Evaluation

```
// Create a lazy range that supports chaining (like Lodash lazy)
```

```
class LazySeq {
  constructor(iterable) {
    this._iterable = iterable;
  }

  static range(start, end, step = 1) {
    return new LazySeq(function*() {
      for (let i = start; i < end; i += step) yield i;
    }());
  }

  static from(arr) {
    return new LazySeq(arr[Symbol.iterator]());
  }

  map(fn) {
    const source = this._iterable;
    return new LazySeq(function*() {
      for (const item of source) yield fn(item);
    }());
  }

  filter(fn) {
    const source = this._iterable;
    return new LazySeq(function*() {
      for (const item of source) {
        if (fn(item)) yield item;
      }
    }());
  }

  take(n) {
    const source = this._iterable;
    return new LazySeq(function*() {
      let count = 0;
      for (const item of source) {
        if (count++ >= n) return;
        yield item;
      }
    }());
  }

  toArray() {
    return [...this._iterable];
  }

  [Symbol.iterator]() {
    return this._iterable[Symbol.iterator]();
  }
}

// None of these intermediate steps create full arrays!
const result = LazySeq.range(1, 100000000)
  .filter(n => n % 2 === 0) // lazy
  .map(n => n * n)          // lazy
  .take(5)                 // lazy – stops after 5
  .toArray();              // only NOW does it execute
```

```
console.log(result); // [4, 16, 36, 64, 100]
```

■ Interview Tip — Array & Object:

- **Q: structuredClone vs JSON.parse(JSON.stringify)?** structuredClone handles circular refs, Date, Map, Set, ArrayBuffer. JSON method loses them all and fails on circular refs.
- **Q: Why immutable updates?** Reference equality checks (===) become O(1) change detection. Crucial for React/Angular OnPush.
- **Q: Lazy vs eager evaluation?** Lazy processes items one at a time through the chain. Eager creates intermediate arrays. Lazy = less memory for large datasets.

135. Coding Challenges — Recursion & Tree Traversal

Concept Focus: Base cases · tree DFS/BFS · nested data structures · DOM traversal · flatten/unflatten · recursive backtracking

135.1 Basic — Flatten Nested Object to Dot Notation

```
// { a: { b: { c: 1 } } } → { 'a.b.c': 1 }
function flattenObject(obj, prefix = '', result = {}) {
  for (const [key, value] of Object.entries(obj)) {
    const path = prefix ? `${prefix}.${key}` : key;

    if (typeof value === 'object' && value !== null && !Array.isArray(value)) {
      flattenObject(value, path, result);
    } else {
      result[path] = value;
    }
  }
  return result;
}

// Reverse: unflatten
function unflattenObject(flat) {
  const result = {};
  for (const [path, value] of Object.entries(flat)) {
    const keys = path.split('.');
    let current = result;
    for (let i = 0; i < keys.length - 1; i++) {
      current[keys[i]] ??= {};
      current = current[keys[i]];
    }
    current[keys[keys.length - 1]] = value;
  }
  return result;
}

const nested = { user: { name: 'Alice', address: { city: 'NYC', zip: '10001' } } };
const flat = flattenObject(nested);
console.log(flat);
// { 'user.name': 'Alice', 'user.address.city': 'NYC', 'user.address.zip': '10001' }

console.log(unflattenObject(flat));
// Original nested structure restored!
```

135.2 Intermediate — DOM Tree Traversal (DFS & BFS)

```
// DFS – depth-first traversal of DOM tree
function dfsDOM(node, callback) {
  callback(node);
  for (const child of node.children) {
    dfsDOM(child, callback);
  }
}

// BFS – breadth-first traversal of DOM tree
function bfsDOM(root, callback) {
  const queue = [root];
  while (queue.length > 0) {
    const node = queue.shift();
    callback(node);
    for (const child of node.children) {
      queue.push(child);
    }
  }
}

// Find all text nodes
```

```

function getAllTextNodes(root) {
  const texts = [];
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT);
  while (walker.nextNode()) {
    const text = walker.currentNode.textContent.trim();
    if (text) texts.push(text);
  }
  return texts;
}

// Get DOM depth
function getDepth(node) {
  if (!node.children.length) return 0;
  return 1 + Math.max(...[...node.children].map(getDepth));
}

```

135.3 Intermediate — File System Tree (render like "tree" command)

```

// Given a file-system-like tree structure, render it like the 'tree' command
const fileSystem = {
  name: 'root',
  children: [
    { name: 'src', children: [
      { name: 'index.ts', children: [] },
      { name: 'utils', children: [
        { name: 'helpers.ts', children: [] },
        { name: 'constants.ts', children: [] },
      ] },
    ] },
    { name: 'package.json', children: [] },
    { name: 'README.md', children: [] },
  ],
};

function printTree(node, prefix = '', isLast = true) {
  const connector = isLast ? '■■■ ' : '■■■ ';
  const line = prefix + connector + node.name;
  console.log(line);

  const children = node.children || [];
  children.forEach((child, index) => {
    const childPrefix = prefix + (isLast ? ' ' : '■ ');
    printTree(child, childPrefix, index === children.length - 1);
  });
}

printTree(fileSystem, '', true);
// ■■■ root
//   ■■■ src
//     ■ ■■■ index.ts
//     ■ ■■■ utils
//       ■ ■■■ helpers.ts
//       ■ ■■■ constants.ts
//   ■■■ package.json
//   ■■■ README.md

```

135.4 Advanced — JSON Path Query (like \$.store.book[*].author)

```

// Simple JSONPath-like query engine
function jsonQuery(obj, path) {
  const tokens = path.replace('$', '').split('.').filter(Boolean);
  let results = [obj];

  for (const token of tokens) {
    const nextResults = [];

    for (const current of results) {

```

```

    if (token === '*') {
      // Wildcard – all values
      if (Array.isArray(current)) {
        nextResults.push(...current);
      } else if (typeof current === 'object' && current !== null) {
        nextResults.push(...Object.values(current));
      }
    } else if (token.includes '[' && token.includes ']') {
      // Array access: books[0] or books[*]
      const [key, indexStr] = token.split('[');
      const idx = indexStr.replace(']', '');
      const arr = current[key];
      if (!Array.isArray(arr)) continue;
      if (idx === '*') nextResults.push(...arr);
      else nextResults.push(arr[parseInt(idx)]);
    } else {
      if (current && current[token] !== undefined) {
        nextResults.push(current[token]);
      }
    }
  }
}

results = nextResults;
}

return results;
}

const data = {
  store: {
    books: [
      { title: 'JS Guide', author: 'Alice', price: 29 },
      { title: 'CSS Magic', author: 'Bob', price: 19 },
      { title: 'TS Deep', author: 'Carol', price: 35 },
    ]
  }
};

console.log(jsonQuery(data, '$.store.books[*].author'));
// ['Alice', 'Bob', 'Carol']

console.log(jsonQuery(data, '$.store.books[0].title'));
// ['JS Guide']

```

135.5 Advanced — Recursive Backtracking (generate permutations)

```

// Generate all permutations of an array
function permutations(arr) {
  if (arr.length <= 1) return [arr];

  const result = [];
  for (let i = 0; i < arr.length; i++) {
    const rest = [...arr.slice(0, i), ...arr.slice(i + 1)];
    for (const perm of permutations(rest)) {
      result.push([arr[i], ...perm]);
    }
  }
  return result;
}

console.log(permutations([1, 2, 3]));
// [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

// Generate all subsets (power set)
function subsets(arr) {
  const result = [[]];

```



```

    for (const item of arr) {
      const len = result.length;
      for (let i = 0; i < len; i++) {
        result.push([...result[i], item]);
      }
    }
    return result;
  }
}

console.log(subsets([1, 2, 3]));
// [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]

// Generate balanced parentheses
function generateParens(n) {
  const result = [];

  function backtrack(current, open, close) {
    if (current.length === 2 * n) {
      result.push(current);
      return;
    }
    if (open < n) backtrack(current + '(', open + 1, close);
    if (close < open) backtrack(current + ')', open, close + 1);
  }

  backtrack('', 0, 0);
  return result;
}

console.log(generateParens(3));
// ['((()))', '(()())', '(()()())', '()()()', '()()()()']

```

■ Interview Tip — Recursion:

- **Q: How to avoid stack overflow?** (1) Convert to iterative with explicit stack/queue. (2) Tail-call optimization (limited support). (3) Trampoline pattern.
- **Q: DFS vs BFS for DOM?** DFS is natural for DOM (parent→child). BFS useful for level-order rendering, finding nearest element matching a condition.
- **Q: Time complexity of permutations?** $O(n!)$ — can't do better since there ARE $n!$ permutations.

136. Coding Challenges — Event Emitter & Observer Pattern

Concept Focus: Pub-sub pattern · event namespacing · once listeners · wildcard events · typed events · RxJS-like Observable from scratch

136.1 Basic — Simple EventEmitter

```
// Core EventEmitter – on, off, emit
class EventEmitter {
  #events = new Map();

  on(event, listener) {
    if (!this.#events.has(event)) {
      this.#events.set(event, []);
    }
    this.#events.get(event).push(listener);
    return this; // allow chaining
  }

  off(event, listener) {
    const listeners = this.#events.get(event);
    if (listeners) {
      this.#events.set(event, listeners.filter(l => l !== listener && l._original !== listener));
    }
    return this;
  }

  emit(event, ...args) {
    const listeners = this.#events.get(event) || [];
    listeners.forEach(l => l(...args));
    return listeners.length > 0;
  }
}

const emitter = new EventEmitter();
const handler = (msg) => console.log(`Received: ${msg}`);
emitter.on('message', handler);
emitter.emit('message', 'Hello'); // "Received: Hello"
emitter.off('message', handler);
emitter.emit('message', 'Hello'); // nothing
```

136.2 Intermediate — once() + removeAllListeners()

```
class EventEmitter {
  #events = new Map();

  on(event, listener) {
    if (!this.#events.has(event)) this.#events.set(event, []);
    this.#events.get(event).push(listener);
    return this;
  }

  once(event, listener) {
    const wrapper = (...args) => {
      this.off(event, wrapper);
      listener(...args);
    };
    wrapper._original = listener; // for off() to find it
    return this.on(event, wrapper);
  }

  off(event, listener) {
    const listeners = this.#events.get(event);
    if (listeners) {
      this.#events.set(event, listeners.filter(l => l !== listener && l._original !== listener));
    }
  }
}
```

```

    return this;
  }

  emit(event, ...args) {
    const listeners = [...(this.#events.get(event) || [])]; // copy to handle once()
    listeners.forEach(l => l(...args));
    return listeners.length > 0;
  }

  removeAllListeners(event) {
    if (event) this.#events.delete(event);
    else this.#events.clear();
    return this;
  }

  listenerCount(event) {
    return (this.#events.get(event) || []).length;
  }
}

const ee = new EventEmitter();
ee.once('connect', () => console.log('Connected!'));
ee.emit('connect'); // "Connected!"
ee.emit('connect'); // nothing – listener was auto-removed

```

136.3 Advanced — Observable from Scratch (simplified RxJS)

```

// Simplified Observable with map, filter, subscribe, unsubscribe
class Observable {
  constructor(subscribeFn) {
    this._subscribeFn = subscribeFn;
  }

  subscribe(observer) {
    // Normalize: accept function or { next, error, complete }
    const obs = typeof observer === 'function'
      ? { next: observer, error: () => {}, complete: () => {} }
      : { next: observer.next || (() => {}),
        error: observer.error || (() => {}),
        complete: observer.complete || (() => {}) };

    let isUnsubscribed = false;
    const safeObserver = {
      next: (v) => { if (!isUnsubscribed) obs.next(v); },
      error: (e) => { if (!isUnsubscribed) { obs.error(e); isUnsubscribed = true; } },
      complete: () => { if (!isUnsubscribed) { obs.complete(); isUnsubscribed = true; } },
    };

    const teardown = this._subscribeFn(safeObserver);

    return {
      unsubscribe() {
        isUnsubscribed = true;
        if (typeof teardown === 'function') teardown();
      }
    };
  }
}

// Operators return NEW Observables (immutable chain)
map(fn) {
  return new Observable((observer) => {
    return this.subscribe({
      next: (v) => observer.next(fn(v)),
      error: (e) => observer.error(e),
      complete: () => observer.complete(),
    }).unsubscribe;
  });
}

```

```

    });
  }

  filter(fn) {
    return new Observable((observer) => {
      return this.subscribe({
        next: (v) => { if (fn(v)) observer.next(v); },
        error: (e) => observer.error(e),
        complete: () => observer.complete(),
      }).unsubscribe();
    });
  }

  take(n) {
    return new Observable((observer) => {
      let count = 0;
      const sub = this.subscribe({
        next: (v) => {
          observer.next(v);
          if (++count >= n) {
            observer.complete();
            sub.unsubscribe();
          }
        },
        error: (e) => observer.error(e),
        complete: () => observer.complete(),
      });
      return () => sub.unsubscribe();
    });
  }

  // Static creators
  static fromEvent(element, eventName) {
    return new Observable((observer) => {
      const handler = (e) => observer.next(e);
      element.addEventListener(eventName, handler);
      return () => element.removeEventListener(eventName, handler);
    });
  }

  static interval(ms) {
    return new Observable((observer) => {
      let i = 0;
      const id = setInterval(() => observer.next(i++), ms);
      return () => clearInterval(id);
    });
  }
}

// Usage (mirrors RxJS patterns):
const clicks$ = Observable.fromEvent(document, 'click')
  .map(e => ({ x: e.clientX, y: e.clientY }))
  .filter(pos => pos.x > 100);

const sub = clicks$.subscribe({
  next: pos => console.log('Click at', pos),
  complete: () => console.log('Done'),
});

// After 10 seconds, unsubscribe
setTimeout(() => sub.unsubscribe(), 10000);

// Interval with take:
Observable.interval(1000)
  .map(n => n * 2)

```

```
.take(5)
.subscribe({
  next: v => console.log(v),      // 0, 2, 4, 6, 8
  complete: () => console.log('Done!'),
});
```

■ Interview Tip — Event Emitter & Observer:

- **Q: EventEmitter vs Observable?** EventEmitter is push-based, imperative. Observable is lazy — nothing happens until subscribe(). Observable supports operators (map, filter) natively.
- **Q: Memory leak risk?** Forgetting to call off()/unsubscribe(). Always clean up in component destroy.
- **Q: Why copy listeners array before emit?** A listener might call off() during emit, modifying the array mid-iteration.

137. Coding Challenges — this Binding & Prototypes

Concept Focus: call/apply/bind · prototype chain · inheritance without class · method borrowing · new keyword · Object.create patterns

137.1 Basic — Implement Function.prototype.myBind

```
// myBind – returns a new function with 'this' bound + partial args
Function.prototype.myBind = function(context, ...boundArgs) {
  const fn = this;

  return function boundFunction(...callArgs) {
    // Handle 'new' – don't bind context when called as constructor
    if (new.target) {
      return new fn(...boundArgs, ...callArgs);
    }
    return fn.apply(context, [...boundArgs, ...callArgs]);
  };
};

function greet(greeting, punct) {
  return `${greeting}, ${this.name}${punct}`;
}

const bob = { name: 'Bob' };
const greetBob = greet.myBind(bob, 'Hello');
console.log(greetBob('!')); // "Hello, Bob!"
console.log(greetBob('.')'); // "Hello, Bob."
```

137.2 Basic — Implement Function.prototype.myCall & myApply

```
// myCall – invoke function with specified 'this' and args
Function.prototype.myCall = function(context, ...args) {
  context = context ?? globalThis; // null/undefined → global
  context = Object(context);        // primitives → wrapper object

  const sym = Symbol('fn');         // unique key to avoid collision
  context[sym] = this;
  const result = context[sym](...args);
  delete context[sym];
  return result;
};

// myApply – same but takes array of args
Function.prototype.myApply = function(context, args = []) {
  return this.myCall(context, ...args);
};

function introduce(age) {
  return `I'm ${this.name}, age ${age}`;
}

console.log(introduce.myCall({ name: 'Alice' }, 25)); // "I'm Alice, age 25"
console.log(introduce.myApply({ name: 'Bob' }, [30])); // "I'm Bob, age 30"
```

137.3 Intermediate — Implement new Keyword

```
// myNew(Constructor, ...args) – simulates the 'new' operator
function myNew(Constructor, ...args) {
  // 1. Create new object with Constructor's prototype
  const obj = Object.create(Constructor.prototype);

  // 2. Execute constructor with 'this' = new object
  const result = Constructor.apply(obj, args);

  // 3. If constructor returns an object, use that; otherwise use our obj
  return result instanceof Object ? result : obj;
}
```

```

}

function Person(name, age) {
  this.name = name;
  this.age = age;
}
Person.prototype.greet = function() {
  return `Hi, I'm ${this.name}`;
};

const p = myNew(Person, 'Alice', 25);
console.log(p.name);    // 'Alice'
console.log(p.greet()); // "Hi, I'm Alice"
console.log(p instanceof Person); // true ✓

```

137.4 Intermediate — Prototypal Inheritance without Class

```

// Classical OOP via prototypes (pre-ES6 style – still asked in interviews!)
function Animal(name) {
  this.name = name;
}
Animal.prototype.speak = function() {
  return `${this.name} makes a sound.`;
};

function Dog(name, breed) {
  Animal.call(this, name); // Super constructor call
  this.breed = breed;
}

// Set up inheritance chain
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog; // Fix constructor reference

Dog.prototype.speak = function() {
  return `${this.name} barks!`;
};

Dog.prototype.fetch = function(item) {
  return `${this.name} fetches the ${item}`;
};

const rex = new Dog('Rex', 'Labrador');
console.log(rex.speak());           // "Rex barks!"
console.log(rex.fetch('ball'));     // "Rex fetches the ball"
console.log(rex instanceof Dog);    // true
console.log(rex instanceof Animal); // true

// Verify prototype chain:
console.log(Object.getPrototypeOf(rex) === Dog.prototype); // true
console.log(Object.getPrototypeOf(Dog.prototype) === Animal.prototype); // true

```

137.5 Advanced — Implement instanceof

```

// myInstanceOf – check if obj's prototype chain includes Constructor.prototype
function myInstanceOf(obj, Constructor) {
  if (obj === null || typeof obj !== 'object') return false;

  let proto = Object.getPrototypeOf(obj);
  const target = Constructor.prototype;

  while (proto !== null) {
    if (proto === target) return true;
    proto = Object.getPrototypeOf(proto);
  }

  return false;
}

```

```

}

console.log(myInstanceof(rex, Dog));    // true
console.log(myInstanceof(rex, Animal)); // true
console.log(myInstanceof(rex, Array));  // false
console.log(myInstanceof([], Array));    // true
console.log(myInstanceof([], Object));   // true

```

137.6 Advanced — Method Chaining Builder with Proxies

```

// Create a chainable query builder using Proxy
function createQueryBuilder() {
  const query = { table: '', conditions: [], columns: '', orderBy: '', limit: null };

  const builder = new Proxy({}, {
    get(target, prop) {
      if (prop === 'build') {
        return () => {
          let sql = `SELECT ${query.columns} FROM ${query.table}`;
          if (query.conditions.length) sql += ` WHERE ${query.conditions.join(' AND ')} `;
          if (query.orderBy) sql += ` ORDER BY ${query.orderBy} `;
          if (query.limit) sql += ` LIMIT ${query.limit} `;
          return sql;
        };
      }
      if (prop === 'select') return (cols) => { query.columns = cols; return builder; };
      if (prop === 'from') return (table) => { query.table = table; return builder; };
      if (prop === 'where') return (cond) => { query.conditions.push(cond); return builder; };
      if (prop === 'order') return (col) => { query.orderBy = col; return builder; };
      if (prop === 'limit') return (n) => { query.limit = n; return builder; };
    }
  });

  return builder;
}

const sql = createQueryBuilder()
  .select('name, email')
  .from('users')
  .where('age > 18')
  .where('active = true')
  .order('name ASC')
  .limit(10)
  .build();

console.log(sql);
// SELECT name, email FROM users WHERE age > 18 AND active = true ORDER BY name ASC LIMIT 10

```

■ Interview Tip — this & Prototypes:

- **Q: 4 rules for this?** (1) new → new object. (2) call/apply/bind → specified object. (3) obj.method() → obj. (4) standalone → globalThis (or undefined in strict mode).
- **Q: Arrow functions and this?** Arrows don't have their own this — they inherit from lexical scope. Cannot be used as constructors.
- **Q: Why Object.create(Parent.prototype) instead of new Parent()?** new Parent() runs the constructor, which might have side effects or require args. Object.create just sets up the prototype link.

138. Coding Challenges — Scope, Hoisting & Execution Context

Concept Focus: var/let/const scoping · TDZ · IIFE patterns · block scoping · function vs block scope · module scope

138.1 — IIFE Data Privacy Module

```
// Create a module with private state using IIFE
const UserModule = (function() {
  // Private
  let users = [];
  let nextId = 1;

  // Private helper
  function validate(user) {
    if (!user.name || typeof user.name !== 'string') {
      throw new Error('Invalid name');
    }
    if (!user.email || !user.email.includes('@')) {
      throw new Error('Invalid email');
    }
  }

  // Public API
  return {
    addUser(name, email) {
      const user = { id: nextId++, name, email };
      validate(user);
      users.push(user);
      return { ...user }; // return copy, not reference
    },
    getUser(id) {
      const user = users.find(u => u.id === id);
      return user ? { ...user } : null;
    },
    getAllUsers() {
      return users.map(u => ({ ...u })); // return copies
    },
    removeUser(id) {
      const idx = users.findIndex(u => u.id === id);
      if (idx !== -1) return users.splice(idx, 1)[0];
      return null;
    },
    get count() { return users.length; }
  };
})();

UserModule.addUser('Alice', 'alice@test.com');
UserModule.addUser('Bob', 'bob@test.com');
console.log(UserModule.count); // 2
console.log(UserModule.getAllUsers()); // [{id:1,...}, {id:2,...}]
console.log(typeof users); // 'undefined' – truly private
```

138.2 — Block Scoping Puzzles

```
// Puzzle 1: What's logged?
{
  var a = 1; // function-scoped → leaks out of block
  let b = 2; // block-scoped → stays inside
  const c = 3; // block-scoped → stays inside
}
console.log(a); // 1
// console.log(b); // ReferenceError
// console.log(c); // ReferenceError

// Puzzle 2: switch-case scoping trap
function processCommand(cmd) {
```

```

switch(cmd) {
  case 'start':
    let result = 'started'; // This let is scoped to the switch BLOCK
    break;
  case 'stop':
    // let result = 'stopped'; // SyntaxError! 'result' already declared in same block
    // Fix: wrap each case in its own block
    break;
}
}

// Fixed version:
function processCommandFixed(cmd) {
  switch(cmd) {
    case 'start': {
      let result = 'started'; // scoped to this { } block
      console.log(result);
      break;
    }
    case 'stop': {
      let result = 'stopped'; // different scope – no conflict
      console.log(result);
      break;
    }
  }
}

// Puzzle 3: for-loop var vs let with setTimeout (classic!)
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var:', i), 0);
}
// var: 3, var: 3, var: 3

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log('let:', i), 0);
}
// let: 0, let: 1, let: 2

```

138.3 — Temporal Dead Zone Challenges

```

// TDZ: let/const exist from block start but can't be accessed before declaration

// Challenge 1: typeof in TDZ
console.log(typeof undeclaredVar); // 'undefined' – no error for undeclared
// console.log(typeof tdzVar);      // ReferenceError! – different from undeclared
// let tdzVar = 10;

// Challenge 2: Default param TDZ
// function bad(a = b, b = 1) {} // ReferenceError! 'b' not yet initialized when 'a' evaluates
function good(a = 1, b = a) { return [a, b]; } // OK – a is already initialized
console.log(good()); // [1, 1]

// Challenge 3: Class TDZ
const instance = new MyClass(); // ReferenceError – class declarations have TDZ too!
class MyClass { }

// But function declarations DON'T have TDZ (they're fully hoisted):
const result = hoistedFn(); // Works fine!
function hoistedFn() { return 'I am hoisted'; }

```

139. Coding Challenges — Event Loop & Async Execution Order

Concept Focus: Microtasks vs macrotasks · Promise resolution timing · queueMicrotask · MutationObserver · requestAnimationFrame order · async/await desugaring

139.1 — Predict the Output (Classic Event Loop)

```
// Challenge 1: What's the order?
console.log('1');

setTimeout(() => console.log('2'), 0);

Promise.resolve().then(() => console.log('3'));

console.log('4');

// Answer: 1, 4, 3, 2
// Explanation:
// - '1' → sync, runs immediately
// - setTimeout → schedules macrotask (task queue)
// - Promise.then → schedules microtask (microtask queue)
// - '4' → sync, runs immediately
// - Call stack empty → drain microtask queue → '3'
// - Then process next macrotask → '2'
```

139.2 — Complex Event Loop (nested microtasks & macrotasks)

```
// Challenge 2: What's the order?
console.log('start');

setTimeout(() => {
  console.log('timeout 1');
  Promise.resolve().then(() => console.log('promise inside timeout'));
}, 0);

Promise.resolve().then(() => {
  console.log('promise 1');
  setTimeout(() => console.log('timeout inside promise'), 0);
});

Promise.resolve().then(() => console.log('promise 2'));

console.log('end');

// Answer: start, end, promise 1, promise 2, timeout 1, promise inside timeout, timeout inside p
// Step-by-step:
// 1. "start" – sync
// 2. setTimeout cb → macrotask queue
// 3. Promise.then cb → microtask queue
// 4. Promise.then cb → microtask queue
// 5. "end" – sync
// 6. Drain microtasks: "promise 1" (schedules new macrotask), then "promise 2"
// 7. Next macrotask: "timeout 1" (schedules microtask)
// 8. Drain microtasks: "promise inside timeout"
// 9. Next macrotask: "timeout inside promise"
```

139.3 — async/await Execution Order

```
// Challenge 3: async/await is syntactic sugar over Promises
async function foo() {
  console.log('foo start');
  await bar(); // everything after await goes into microtask
  console.log('foo end'); // this is like .then(() => console.log('foo end'))
}

async function bar() {
```

```

    console.log('bar');
  }

  console.log('script start');
  foo();
  console.log('script end');

  // Answer: script start, foo start, bar, script end, foo end
  // Why? 'await bar()' resolves synchronously (bar returns resolved promise),
  // but the code AFTER await is still scheduled as a microtask.

```

139.4 — Build Your Own Task Scheduler

```

// Scheduler that respects micro/macro task ordering
class TaskScheduler {
  #microTasks = [];
  #macroTasks = [];

  addMicrotask(fn) {
    this.#microTasks.push(fn);
    // queueMicrotask runs BEFORE next macrotask
    queueMicrotask(() => this.#processMicrotasks());
  }

  addMacrotask(fn, delay = 0) {
    this.#macroTasks.push(fn);
    setTimeout(() => {
      const task = this.#macroTasks.shift();
      if (task) task();
    }, delay);
  }

  #processMicrotasks() {
    while (this.#microTasks.length) {
      const task = this.#microTasks.shift();
      task();
    }
  }

  // Implement setImmediate-like behavior (next tick)
  nextTick(fn) {
    if (typeof MessageChannel !== 'undefined') {
      const channel = new MessageChannel();
      channel.port1.onmessage = fn;
      channel.port2.postMessage(undefined);
    } else {
      setTimeout(fn, 0);
    }
  }
}

// Implement setInterval using setTimeout (common interview question!)
function mySetInterval(fn, delay) {
  let cancelled = false;

  function tick() {
    if (!cancelled) {
      fn();
      setTimeout(tick, delay);
    }
  }

  setTimeout(tick, delay);

  return {
    clear() { cancelled = true; }
  }
}

```

```
    };
  }
```

```
const interval = mySetInterval(() => console.log('tick'), 1000);
setTimeout(() => interval.clear(), 5500); // stops after 5 ticks
```

139.5 — requestAnimationFrame vs setTimeout vs Microtask Ordering

```
// Challenge: What's the order?
setTimeout(() => console.log('setTimeout'), 0);
requestAnimationFrame(() => console.log('rAF'));
Promise.resolve().then(() => console.log('microtask'));
queueMicrotask(() => console.log('queueMicrotask'));

// Typical answer: microtask, queueMicrotask, rAF, setTimeout
// BUT: rAF timing varies – it runs before the next paint, which is
// usually before setTimeout, but NOT guaranteed to come before setTimeout
// in all browsers. The ONLY guarantee is:
// Microtasks always run before any macrotask/rAF

// Rule of thumb for priority:
// 1. Synchronous code
// 2. Microtasks (Promise.then, queueMicrotask, MutationObserver)
// 3. requestAnimationFrame (before paint)
// 4. Macrotasks (setTimeout, setInterval, I/O, UI events)
```

■ Interview Tip — Event Loop:

- **Key rule:** After each task (macrotask), ALL microtasks drain before the next task runs.
- **async/await:** Code before first await is SYNCHRONOUS. Code after await becomes a microtask.
- **Common gotcha:** Promise constructor callback is synchronous! Only .then/.catch/.finally are async.
- **Node.js difference:** process.nextTick runs BEFORE Promise microtasks. setImmediate runs after I/O.

140. Coding Challenges — String Manipulation & RegEx

Concept Focus: String parsing · template engines · regex patterns · Unicode handling · common string algorithms

140.1 Basic — String Compression

```
// "aaabbbccca" → "a3b3c3a1"
function compress(str) {
  if (!str) return '';
  let result = '';
  let count = 1;

  for (let i = 1; i <= str.length; i++) {
    if (i < str.length && str[i] === str[i - 1]) {
      count++;
    } else {
      result += str[i - 1] + count;
      count = 1;
    }
  }

  return result.length < str.length ? result : str;
}

console.log(compress('aaabbbccca')); // "a3b3c3a1"
console.log(compress('abcdef')); // "abcdef" (compression is longer)

// Decompress: "a3b3c3a1" → "aaabbbccca"
function decompress(str) {
  return str.replace(/([a-zA-Z])(\d+)/g, (_, char, count) =>
    char.repeat(parseInt(count))
  );
}
console.log(decompress('a3b3c3a1')); // "aaabbbccca"
```

140.2 Intermediate — Simple Template Engine

```
// "Hello {{name}}, you have {{count}} messages" + data → filled string
function template(str, data) {
  return str.replace(/\{\{(\w+(?:\.\w+)*)\}\}/g, (match, path) => {
    const value = path.split('.').reduce((obj, key) => obj?.[key], data);
    return value !== undefined ? value : match;
  });
}

const result = template('Hello {{user.name}}, you have {{count}} messages', {
  user: { name: 'Alice' },
  count: 5
});
console.log(result); // "Hello Alice, you have 5 messages"

// Advanced version with conditionals and loops
function advancedTemplate(str, data) {
  // Handle {{#if condition}}...{{/if}}
  str = str.replace(/\{\{#if (\w+)\}\}([\s\S]*?)\{\{/if\}\}/g,
    (_, key, content) => data[key] ? content : ''
  );

  // Handle {{#each items}}...{{/each}}
  str = str.replace(/\{\{#each (\w+)\}\}([\s\S]*?)\{\{/each\}\}/g,
    (_, key, content) => {
      return (data[key] || []).map(item =>
        content.replace(/\{\{this(?:\.\w+)\}\}/g, (_, prop) =>
          prop ? item[prop] : item
        )
      ).join('');
    }
  );
}
```

```

    }
  );

  // Handle simple {{variable}}
  str = str.replace(/\{\{(\w+(?:\.\w+)*)\}\}/g, (match, path) => {
    const value = path.split('.').reduce((obj, key) => obj?.[key], data);
    return value !== undefined ? String(value) : match;
  });

  return str;
}

```

140.3 Intermediate — Parse Query String / Build Query String

```

// "?name=Alice&age=25&tags=js&tags=ts" → { name: 'Alice', age: '25', tags: ['js', 'ts'] }
function parseQueryString(qs) {
  const params = {};
  const search = qs.startsWith('?') ? qs.slice(1) : qs;
  if (!search) return params;

  for (const pair of search.split('&')) {
    const [rawKey, rawValue = ''] = pair.split('=');
    const key = decodeURIComponent(rawKey);
    const value = decodeURIComponent(rawValue);

    if (key in params) {
      params[key] = [].concat(params[key], value);
    } else {
      params[key] = value;
    }
  }

  return params;
}

// Reverse: build query string from object
function buildQueryString(params) {
  const parts = [];
  for (const [key, value] of Object.entries(params)) {
    const values = [].concat(value);
    for (const v of values) {
      parts.push(`${encodeURIComponent(key)}=${encodeURIComponent(v)}`);
    }
  }
  return parts.length ? `?${parts.join('&')}` : '';
}

console.log(parseQueryString('?name=Alice&tags=js&tags=ts'));
// { name: 'Alice', tags: ['js', 'ts'] }

console.log(buildQueryString({ name: 'Alice', tags: ['js', 'ts'] }));
// "?name=Alice&tags=js&tags=ts"

```

140.4 Advanced — Regex: Validate Complex Patterns

```

// Email validation (simplified but practical)
const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;

// Password: 8+ chars, uppercase, lowercase, digit, special char
const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;

// URL parsing with named groups
const urlRegex = /^(?<protocol>https?):\/\/(?<host>[^\n]+)(?::(?<port>\d+))?(?<path>\/[^\?#]*)?$/;

const match = 'https://example.com:8080/api/users?page=1#top'.match(urlRegex);
console.log(match.groups);
// { protocol: 'https', host: 'example.com', port: '8080',

```

```
// path: '/api/users', query: 'page=1', fragment: 'top' }

// Highlight search terms (wrap in <mark>)
function highlightText(text, searchTerm) {
  const escaped = searchTerm.replace(/[\.*+?^${}()|[\]\$\\"/g, '\\$&');
  const regex = new RegExp(`(${escaped})`, 'gi');
  return text.replace(regex, '<mark>$1</mark>');
}

console.log(highlightText('JavaScript is great for scripting', 'script'));
// "Java<mark>Script</mark> is great for <mark>script</mark>ing"
```

140.5 Advanced — Longest Palindromic Substring

```
// Expand-around-center approach — O(n²)
function longestPalindrome(s) {
  if (s.length < 2) return s;
  let start = 0, maxLen = 1;

  function expandAroundCenter(left, right) {
    while (left >= 0 && right < s.length && s[left] === s[right]) {
      const len = right - left + 1;
      if (len > maxLen) {
        start = left;
        maxLen = len;
      }
      left--;
      right++;
    }
  }

  for (let i = 0; i < s.length; i++) {
    expandAroundCenter(i, i); // odd-length palindromes
    expandAroundCenter(i, i + 1); // even-length palindromes
  }

  return s.substring(start, start + maxLen);
}

console.log(longestPalindrome('babad')); // "bab" or "aba"
console.log(longestPalindrome('racecar')); // "racecar"
```

■ Interview Tip — Strings & RegEx:

- **Q: Why escape regex special chars?** Characters like `.` `*` `+` `?` have special meaning. Use `str.replace(/[\.*+?^${}()|[\]\$\\"/g, '\\$&')` to escape user input.
- **Q: String immutability in JS?** Strings are immutable — every operation creates a new string. For many concatenations, use `array + join()`.
- **Q: Named groups in regex?** (`?<name>pattern`) — accessed via `match.groups.name`. Much more readable than positional `$1`, `$2`.

141. Coding Challenges — Data Structures in JavaScript

Concept Focus: Stack · Queue · LinkedList · HashMap · Trie · Heap — all implemented from scratch in JS

141.1 — Stack (with min/max tracking)

```
class MinMaxStack {
  #stack = [];
  #minStack = [];
  #maxStack = [];

  push(val) {
    this.#stack.push(val);
    this.#minStack.push(this.#minStack.length === 0 ? val : Math.min(val, this.min()));
    this.#maxStack.push(this.#maxStack.length === 0 ? val : Math.max(val, this.max()));
  }

  pop() {
    this.#minStack.pop();
    this.#maxStack.pop();
    return this.#stack.pop();
  }

  peek() { return this.#stack[this.#stack.length - 1]; }
  min() { return this.#minStack[this.#minStack.length - 1]; }
  max() { return this.#maxStack[this.#maxStack.length - 1]; }
  get size() { return this.#stack.length; }
  get isEmpty() { return this.#stack.length === 0; }
}

const stack = new MinMaxStack();
stack.push(5); stack.push(2); stack.push(8); stack.push(1);
console.log(stack.min()); // 1
console.log(stack.max()); // 8
stack.pop(); // removes 1
console.log(stack.min()); // 2 — still 0(1)!
```

141.2 — Queue (with two stacks)

```
// Implement Queue using two Stacks — amortized O(1) operations
class QueueWithStacks {
  #inStack = [];
  #outStack = [];

  enqueue(val) {
    this.#inStack.push(val);
  }

  dequeue() {
    if (this.#outStack.length === 0) {
      // Transfer all from inStack to outStack (reverses order)
      while (this.#inStack.length > 0) {
        this.#outStack.push(this.#inStack.pop());
      }
    }
    return this.#outStack.pop();
  }

  peek() {
    if (this.#outStack.length === 0) {
      while (this.#inStack.length > 0) {
        this.#outStack.push(this.#inStack.pop());
      }
    }
    return this.#outStack[this.#outStack.length - 1];
  }
}
```

```

    get size() { return this.#inStack.length + this.#outStack.length; }
    get isEmpty() { return this.size === 0; }
}

const q = new QueueWithStacks();
q.enqueue(1); q.enqueue(2); q.enqueue(3);
console.log(q.dequeue()); // 1 (FIFO)
console.log(q.dequeue()); // 2
q.enqueue(4);
console.log(q.dequeue()); // 3

```

141.3 — Linked List (with reverse)

```

class ListNode {
  constructor(val, next = null) {
    this.val = val;
    this.next = next;
  }
}

class LinkedList {
  #head = null;
  #size = 0;

  // Add to end
  append(val) {
    const node = new ListNode(val);
    if (!this.#head) { this.#head = node; }
    else {
      let current = this.#head;
      while (current.next) current = current.next;
      current.next = node;
    }
    this.#size++;
  }

  // Remove by value
  remove(val) {
    if (!this.#head) return false;
    if (this.#head.val === val) { this.#head = this.#head.next; this.#size--; return true; }

    let current = this.#head;
    while (current.next) {
      if (current.next.val === val) {
        current.next = current.next.next;
        this.#size--;
        return true;
      }
      current = current.next;
    }
    return false;
  }

  // Reverse in-place — O(n) time, O(1) space
  reverse() {
    let prev = null, current = this.#head;
    while (current) {
      const next = current.next;
      current.next = prev;
      prev = current;
      current = next;
    }
    this.#head = prev;
  }

  // Detect cycle (Floyd's Tortoise and Hare)

```

```

hasCycle() {
  let slow = this.#head, fast = this.#head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true;
  }
  return false;
}

// Find middle node
findMiddle() {
  let slow = this.#head, fast = this.#head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
  }
  return slow?.val;
}

toArray() {
  const arr = [];
  let current = this.#head;
  while (current) { arr.push(current.val); current = current.next; }
  return arr;
}

get size() { return this.#size; }
}

const ll = new LinkedList();
ll.append(1); ll.append(2); ll.append(3); ll.append(4);
console.log(ll.findMiddle()); // 3
ll.reverse();
console.log(ll.toArray()); // [4, 3, 2, 1]

```

141.4 — Trie (Prefix Tree)

```

class TrieNode {
  constructor() {
    this.children = new Map();
    this.isEnd = false;
  }
}

class Trie {
  #root = new TrieNode();

  insert(word) {
    let node = this.#root;
    for (const char of word) {
      if (!node.children.has(char)) {
        node.children.set(char, new TrieNode());
      }
      node = node.children.get(char);
    }
    node.isEnd = true;
  }

  search(word) {
    const node = this.#traverse(word);
    return node !== null && node.isEnd;
  }

  startsWith(prefix) {
    return this.#traverse(prefix) !== null;
  }
}

```

```

// Autocomplete: return all words with given prefix
autocomplete(prefix, limit = 10) {
  const node = this.#traverse(prefix);
  if (!node) return [];

  const results = [];
  const dfs = (current, path) => {
    if (results.length >= limit) return;
    if (current.isEnd) results.push(prefix + path);
    for (const [char, child] of current.children) {
      dfs(child, path + char);
    }
  };
  dfs(node, '');
  return results;
}

#traverse(str) {
  let node = this.#root;
  for (const char of str) {
    if (!node.children.has(char)) return null;
    node = node.children.get(char);
  }
  return node;
}
}

const trie = new Trie();
['apple', 'app', 'application', 'apply', 'banana', 'band'].forEach(w => trie.insert(w));
console.log(trie.search('app'));           // true
console.log(trie.search('ap'));            // false
console.log(trie.startsWith('ap'));        // true
console.log(trie.autocomplete('app'));      // ['app', 'apple', 'application', 'apply']

```

141.5 — Min Heap (Priority Queue)

```

class MinHeap {
  #heap = [];

  insert(val) {
    this.#heap.push(val);
    this.#bubbleUp(this.#heap.length - 1);
  }

  extractMin() {
    if (this.#heap.length === 0) return null;
    const min = this.#heap[0];
    const last = this.#heap.pop();
    if (this.#heap.length > 0) {
      this.#heap[0] = last;
      this.#sinkDown(0);
    }
    return min;
  }

  peek() { return this.#heap[0] ?? null; }
  get size() { return this.#heap.length; }

  #bubbleUp(idx) {
    while (idx > 0) {
      const parent = Math.floor((idx - 1) / 2);
      if (this.#heap[parent] <= this.#heap[idx]) break;
      [this.#heap[parent], this.#heap[idx]] = [this.#heap[idx], this.#heap[parent]];
      idx = parent;
    }
  }
}

```

```

    #sinkDown(idx) {
      const length = this.#heap.length;
      while (true) {
        let smallest = idx;
        const left = 2 * idx + 1, right = 2 * idx + 2;
        if (left < length && this.#heap[left] < this.#heap[smallest]) smallest = left;
        if (right < length && this.#heap[right] < this.#heap[smallest]) smallest = right;
        if (smallest === idx) break;
        [this.#heap[smallest], this.#heap[idx]] = [this.#heap[idx], this.#heap[smallest]];
        idx = smallest;
      }
    }
  }
}

// Usage: find K largest elements
function kLargest(arr, k) {
  const heap = new MinHeap();
  for (const num of arr) {
    heap.insert(num);
    if (heap.size > k) heap.extractMin();
  }
  const result = [];
  while (heap.size) result.push(heap.extractMin());
  return result;
}

console.log(kLargest([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5], 3));
// [5, 6, 9] – top 3 largest

```

■ Interview Tip — Data Structures:

- **Q: When to use a Trie?** Autocomplete, spell check, IP routing, dictionary problems. $O(L)$ lookup where L = word length.
- **Q: Queue vs Stack use cases?** Queue = BFS, task scheduling, print queue. Stack = DFS, undo/redo, expression parsing, call stack.
- **Q: Why implement Queue with 2 stacks?** Shows understanding of amortized analysis. Each element is moved at most twice $\rightarrow O(1)$ amortized per operation.

142. Coding Challenges — Design Patterns in Action

Concept Focus: Singleton · Factory · Strategy · Observer · Decorator · Middleware pattern · Dependency injection

142.1 — Singleton (multiple approaches)

```
// Approach 1: Closure-based
const Database = (() => {
  let instance;

  function createInstance(config) {
    return {
      config,
      query(sql) { return `Executing: ${sql}`; },
      connect() { console.log(`Connected to ${config.host}`); }
    };
  }

  return {
    getInstance(config) {
      if (!instance) instance = createInstance(config);
      return instance;
    }
  };
})();

const db1 = Database.getInstance({ host: 'localhost' });
const db2 = Database.getInstance({ host: 'remote' }); // ignored
console.log(db1 === db2); // true

// Approach 2: ES Module (modules are cached by default!)
// db.js
// const connection = createConnection(config);
// export default connection;
// Every import gets the SAME instance – natural singleton
```

142.2 — Strategy Pattern (payment processing)

```
// Different payment strategies, swappable at runtime
const paymentStrategies = {
  creditCard: {
    validate: (details) => details.number?.length === 16,
    process: (amount, details) => ({
      status: 'success',
      method: 'Credit Card',
      last4: details.number.slice(-4),
      amount
    })
  },
  upi: {
    validate: (details) => details.upiId?.includes('@'),
    process: (amount, details) => ({
      status: 'success',
      method: 'UPI',
      upiId: details.upiId,
      amount
    })
  },
  wallet: {
    validate: (details) => details.balance >= 0,
    process: (amount, details) => {
      if (details.balance < amount) {
        return { status: 'failed', reason: 'Insufficient balance' };
      }
    }
  }
}
```

```

        return { status: 'success', method: 'Wallet', amount };
    }
}
};

class PaymentProcessor {
    constructor(strategy) {
        this.strategy = paymentStrategies[strategy];
        if (!this.strategy) throw new Error(`Unknown strategy: ${strategy}`);
    }

    setStrategy(name) {
        this.strategy = paymentStrategies[name];
    }

    pay(amount, details) {
        if (!this.strategy.validate(details)) {
            throw new Error('Invalid payment details');
        }
        return this.strategy.process(amount, details);
    }
}

const processor = new PaymentProcessor('creditCard');
console.log(processor.pay(1000, { number: '4111111111111111' }));
// { status: 'success', method: 'Credit Card', last4: '1111', amount: 1000 }

processor.setStrategy('upi');
console.log(processor.pay(500, { upiId: 'user@paytm' }));
// { status: 'success', method: 'UPI', upiId: 'user@paytm', amount: 500 }

```

142.3 — Middleware Pattern (like Express.js)

```

// Build a middleware pipeline from scratch
class MiddlewarePipeline {
    #middlewares = [];

    use(fn) {
        this.#middlewares.push(fn);
        return this;
    }

    execute(context) {
        let index = 0;

        const next = () => {
            if (index < this.#middlewares.length) {
                const middleware = this.#middlewares[index++];
                middleware(context, next);
            }
        };

        next();
        return context;
    }
}

// Usage: HTTP request pipeline
const pipeline = new MiddlewarePipeline();

// Logger
pipeline.use((ctx, next) => {
    console.log(`${ctx.method} ${ctx.url}`);
    ctx.startTime = Date.now();
    next();
    console.log(`Completed in ${Date.now() - ctx.startTime}ms`);
});

```

```

});

// Auth check
pipeline.use((ctx, next) => {
  if (!ctx.headers?.authorization) {
    ctx.status = 401;
    ctx.body = 'Unauthorized';
    return; // don't call next – stops pipeline
  }
  ctx.user = { id: 1, name: 'Alice' };
  next();
});

// Route handler
pipeline.use((ctx, next) => {
  ctx.status = 200;
  ctx.body = `Hello, ${ctx.user.name}!`;
  next();
});

const result = pipeline.execute({
  method: 'GET',
  url: '/api/profile',
  headers: { authorization: 'Bearer xxx' }
});
console.log(result.body); // "Hello, Alice!"

```

142.4 — Decorator Pattern (function enhancement)

```

// Decorators wrap functions to add behavior without modifying them

// Logging decorator
function withLogging(fn, label = fn.name) {
  return function(...args) {
    console.log(`[${label}] called with:`, args);
    const result = fn.apply(this, args);
    console.log(`[${label}] returned:`, result);
    return result;
  };
}

// Timing decorator
function withTiming(fn) {
  return function(...args) {
    const start = performance.now();
    const result = fn.apply(this, args);
    console.log(`${fn.name} took ${performance.now() - start}.toFixed(2)}ms`);
    return result;
  };
}

// Retry decorator
function withRetry(fn, maxRetries = 3) {
  return async function(...args) {
    for (let attempt = 1; attempt <= maxRetries; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (err) {
        if (attempt === maxRetries) throw err;
        console.log(`Attempt ${attempt} failed, retrying...`);
      }
    }
  };
}

// Compose decorators

```



```
const fetchUser = withLogging(withTiming(withRetry(async (id) => {  
  const res = await fetch(`/api/users/${id}`);  
  if (!res.ok) throw new Error('Failed');  
  return res.json();  
})), 'fetchUser');
```

■ Interview Tip — Design Patterns:

- **Q: Singleton in JS?** ES Modules are natural singletons (cached). Explicit singleton only when you need lazy initialization or config-dependent creation.
- **Q: Strategy vs Factory?** Strategy = choose ALGORITHM at runtime. Factory = choose which OBJECT to create. Strategy composes behavior; Factory composes structure.
- **Q: Middleware pattern benefits?** Separation of concerns, composable, order-dependent pipeline. Used in Express, Redux, Angular interceptors.

143. Coding Challenges — DOM & Browser APIs

Concept Focus: Virtual DOM diff · event delegation · infinite scroll · drag & drop · intersection observer · web workers

143.1 — Event Delegation System

```
// Handle clicks on dynamic elements efficiently
class EventDelegator {
  #handlers = new Map();

  constructor(root) {
    this.root = root;
    this.root.addEventListener('click', this.#handleClick.bind(this));
  }

  on(selector, callback) {
    if (!this.#handlers.has(selector)) {
      this.#handlers.set(selector, []);
    }
    this.#handlers.get(selector).push(callback);
    return this;
  }

  off(selector, callback) {
    const handlers = this.#handlers.get(selector);
    if (handlers) {
      this.#handlers.set(selector, handlers.filter(h => h !== callback));
    }
    return this;
  }

  #handleClick(event) {
    for (const [selector, handlers] of this.#handlers) {
      const target = event.target.closest(selector);
      if (target && this.root.contains(target)) {
        handlers.forEach(h => h(event, target));
      }
    }
  }
}

// Usage: handle clicks on dynamic list items
const delegator = new EventDelegator(document.getElementById('app'));
delegator
  .on('.btn-delete', (e, el) => {
    el.closest('.item')?.remove();
  })
  .on('.btn-edit', (e, el) => {
    const item = el.closest('.item');
    console.log('Edit:', item.dataset.id);
  });
```

143.2 — Simple Virtual DOM Diff & Patch

```
// Minimal vDOM: create, diff, and patch
// vNode = { tag, props, children }

function createElement(tag, props = {}, ...children) {
  return { tag, props, children: children.flat() };
}

function render(vNode) {
  if (typeof vNode === 'string') return document.createTextNode(vNode);

  const el = document.createElement(vNode.tag);
```

```

    for (const [key, value] of Object.entries(vNode.props || {})) {
      if (key.startsWith('on')) {
        el.addEventListener(key.slice(2).toLowerCase(), value);
      } else {
        el.setAttribute(key, value);
      }
    }
  }

  vNode.children.forEach(child => el.appendChild(render(child)));
  return el;
}

function diff(oldVNode, newVNode) {
  // Node removed
  if (!newVNode) return (node) => { node.remove(); };

  // Text node changed
  if (typeof oldVNode === 'string' || typeof newVNode === 'string') {
    if (oldVNode !== newVNode) {
      return (node) => node.replaceWith(render(newVNode));
    }
    return () => {};
  }

  // Tag changed – replace entire node
  if (oldVNode.tag !== newVNode.tag) {
    return (node) => node.replaceWith(render(newVNode));
  }

  // Same tag – diff props and children
  return (node) => {
    // Update props
    const allProps = new Set([
      ...Object.keys(oldVNode.props || {}),
      ...Object.keys(newVNode.props || {})
    ]);
    for (const key of allProps) {
      if (key.startsWith('on')) continue;
      if (!(key in (newVNode.props || {}))) {
        node.removeAttribute(key);
      } else if ((oldVNode.props || {})[key] !== (newVNode.props || {})[key]) {
        node.setAttribute(key, newVNode.props[key]);
      }
    }
  }

  // Diff children
  const maxLen = Math.max(oldVNode.children.length, newVNode.children.length);
  for (let i = 0; i < maxLen; i++) {
    if (i >= oldVNode.children.length) {
      node.appendChild(render(newVNode.children[i]));
    } else if (i >= newVNode.children.length) {
      node.lastChild.remove();
    } else {
      diff(oldVNode.children[i], newVNode.children[i])(node.childNodes[i]);
    }
  }
};

// Usage:
const vApp1 = createElement('div', { class: 'app' },
  createElement('h1', {}, 'Hello'),
  createElement('p', {}, 'World')
);

const vApp2 = createElement('div', { class: 'app updated' },

```

```

    createElement('h1', {}, 'Hello!'),
    createElement('p', {}, 'World'),

    createElement('button', {}, 'Click me')
  );

  const rootEl = render(vApp1);
  document.body.appendChild(rootEl);
  diff(vApp1, vApp2)(rootEl); // patches in-place!

```

143.3 — Infinite Scroll with Intersection Observer

```

// Efficient infinite scroll – no scroll event listeners needed
class InfiniteScroll {
  #page = 1;
  #loading = false;
  #hasMore = true;
  #observer;

  constructor(container, loadMore) {
    this.container = container;
    this.loadMore = loadMore;

    // Sentinel element at bottom
    this.sentinel = document.createElement('div');
    this.sentinel.className = 'scroll-sentinel';
    this.container.appendChild(this.sentinel);

    this.#observer = new IntersectionObserver(
      (entries) => {
        if (entries[0].isIntersecting && !this.#loading && this.#hasMore) {
          this.#fetchNext();
        }
      },
      { rootMargin: '200px' } // trigger 200px before visible
    );

    this.#observer.observe(this.sentinel);
  }

  async #fetchNext() {
    this.#loading = true;
    this.#showLoader();

    try {
      const items = await this.loadMore(this.#page);

      if (items.length === 0) {
        this.#hasMore = false;
        this.#observer.disconnect();
        this.sentinel.textContent = 'No more items';
        return;
      }

      // Insert items BEFORE sentinel
      items.forEach(item => {
        this.container.insertBefore(item, this.sentinel);
      });

      this.#page++;
    } finally {
      this.#loading = false;
      this.#hideLoader();
    }
  }

  #showLoader() { this.sentinel.textContent = 'Loading...'; }

```

```

    #hideLoader() { this.sentinel.textContent = ''; }

    destroy() {
      this.#observer.disconnect();
      this.sentinel.remove();
    }
  }

  // Usage:
  const scroller = new InfiniteScroll(
    document.getElementById('feed'),
    async (page) => {
      const res = await fetch(`/api/posts?page=${page}`);
      const posts = await res.json();
      return posts.map(post => {
        const div = document.createElement('div');
        div.className = 'post';
        div.textContent = post.title;
        return div;
      });
    }
  );

```

■ Interview Tip — DOM Challenges:

- **Q: Why event delegation?** Single listener handles all current AND future child elements. $O(1)$ listeners instead of $O(n)$. Crucial for dynamic lists.
- **Q: Virtual DOM purpose?** Batch DOM mutations, minimize reflows/repaints. Diff algorithm determines minimal required changes.
- **Q: IntersectionObserver vs scroll event?** IO is non-blocking (runs off main thread), no need for debounce/throttle, battery-efficient. Scroll events are synchronous and need throttling.